

Edge Computing with AWS IoT Greengrass - *Part #2*

by Cagatay Sonmez, 30 Oct 2026

The Primary Sources Used in This Lecture

- https://aws.amazon.com/greengrass/
- https://docs.aws.amazon.com/greengrass/v2/developerguide
- https://docs.aws.amazon.com/greengrass/v2/developerguide/run-lambda-functions.html
- https://docs.aws.amazon.com/greengrass/v2/developerguide/interact-with-local-iot-devices.html
- https://docs.aws.amazon.com/greengrass/v2/developerguide/client-devices-tutorial.html

- All the source codes used in this Lecture can be found in the following GitHub repository:
  - https://github.com/CagataySonmez/AWS-IoT-Greengrass-Demo

AWS IoT Greengrass Architecture - Fill in the Blanks

Artifact

Thing Group

Core Device

IPC

Deployment

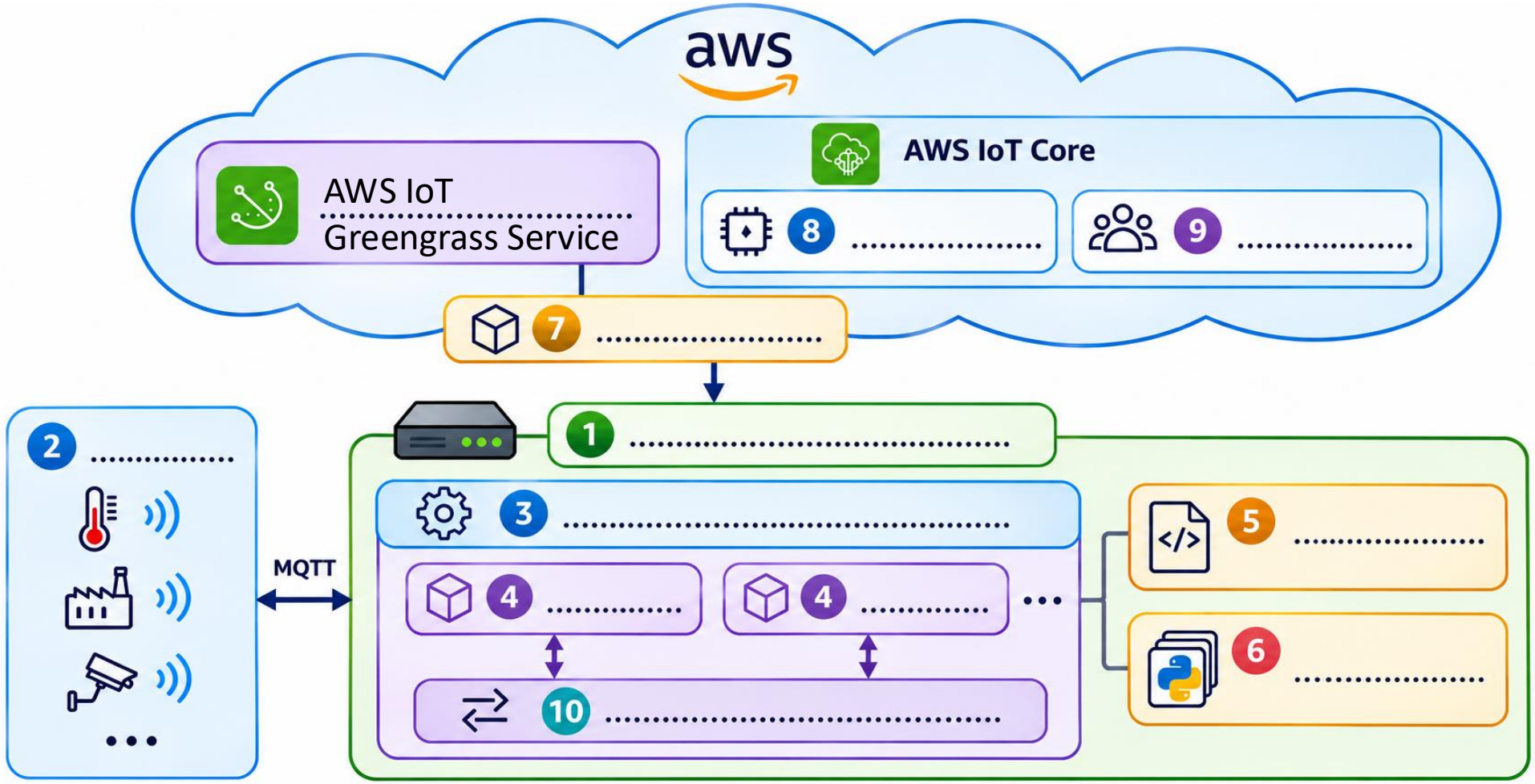
Client Device

Recipe

Nucleus

Thing

Component



AWS Lambda Functions on Core Devices

Run AWS Lambda Functions on Core Devices

- If you want to deploy an existing an application code in Lambda functions to core devices, you can import AWS Lambda functions as components that run on AWS IoT Greengrass core devices.
- Lambda functions include dependencies on the following components.
  - The **Lambda launcher** component: handles processes and environment configuration.
  - The **Lambda manager** component: handles inter-process communication and scaling.
  - The **Lambda runtimes** component: provides artifacts for each supported Lambda runtime.
- You don't need to define these components as dependencies when you import the function.
- When you deploy the Lambda function component, the deployment includes these Lambda component dependencies.

Lambda Function Requirements

- A Linux-based OS with Java Runtime Environment (JRE) version 8 or greater.
- Minimum 256 MB disk space available for the AWS IoT Greengrass Core software.
- Minimum 96 MB RAM allocated to the AWS IoT Greengrass Core software.
- The /tmp directory must be mounted with exec permissions.
- Device must have the mkfifo shell command.
- Device must run the programming language libraries that a Lambda function requires.
  - Python, Node.js, and Java runtimes
- All of the following shell commands:
  - ps -ax -o pid, ppid, sudo, sh, kill, cp, chmod, rm, ln, echo, exit, id, uname, grep

Lambda Function Lifecycle

On-demand functions

- On-demand functions start when they are invoked and stop when there are no tasks left to run.
- Each invocation of the function creates a separate container, also called a sandbox, to process invocations, unless an existing container is available for reuse.
- There is a "cold start" delay to create a new container.
- Multiple invocations of an on-demand function can run **simultaneously**.

Long-lived functions

- Long-lived (or pinned) functions start when the AWS IoT Greengrass Core software starts and run in a single container.
- The container is "always-on" (warm) and ready to process data instantly. The
- They start immediately without invocations. But you can still trigger handlers with invocations (e.g. MQTT message).
- Multiple invocations are **queued** until the AWS IoT Greengrass Core software runs earlier invocations.

On-demand vs Long-lived Functions

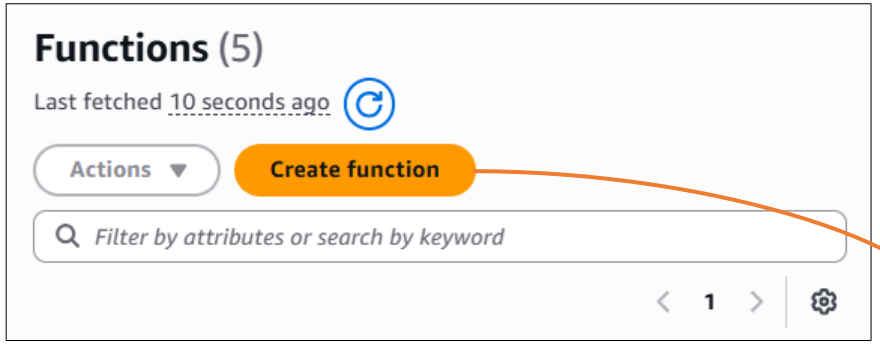
Feature	On-demand Function	Long-lived (Pinned) Function
Lifecycle Trigger	Starts only when invoked by an event (e.g., an MQTT message or an IPC call).	Starts automatically with the Greengrass Core component and runs continuously.
Execution Duration	**Short-lived.** The function's container is shut down after a period of inactivity.	**Persistent.** Starts with Greengrass and remains active until it stops or enters an error state.
State Management	Primarily **Stateless**. Cannot reliably maintain state in memory between invocations.	Primarily **Stateful**. Ideal for maintaining state, open connections, and managing in-memory caches.
Resource Usage	**Low / Intermittent**. Consumes CPU and memory only when active, making it efficient for resource-constrained devices.	**High / Constant**. Consumes resources continuously as long as the Greengrass Core is running.
Configuration	Configured with **Pinned: false**.	Configured with **Pinned: true** (default).
Ideal Use Cases	• Responding to sensor data triggers • Message transformation and routing • Simple, event-driven logic that runs infrequently	• Continuous data acquisition from local devices • Running a local ML inference model. • Managing a persistent connection to a local database or device.

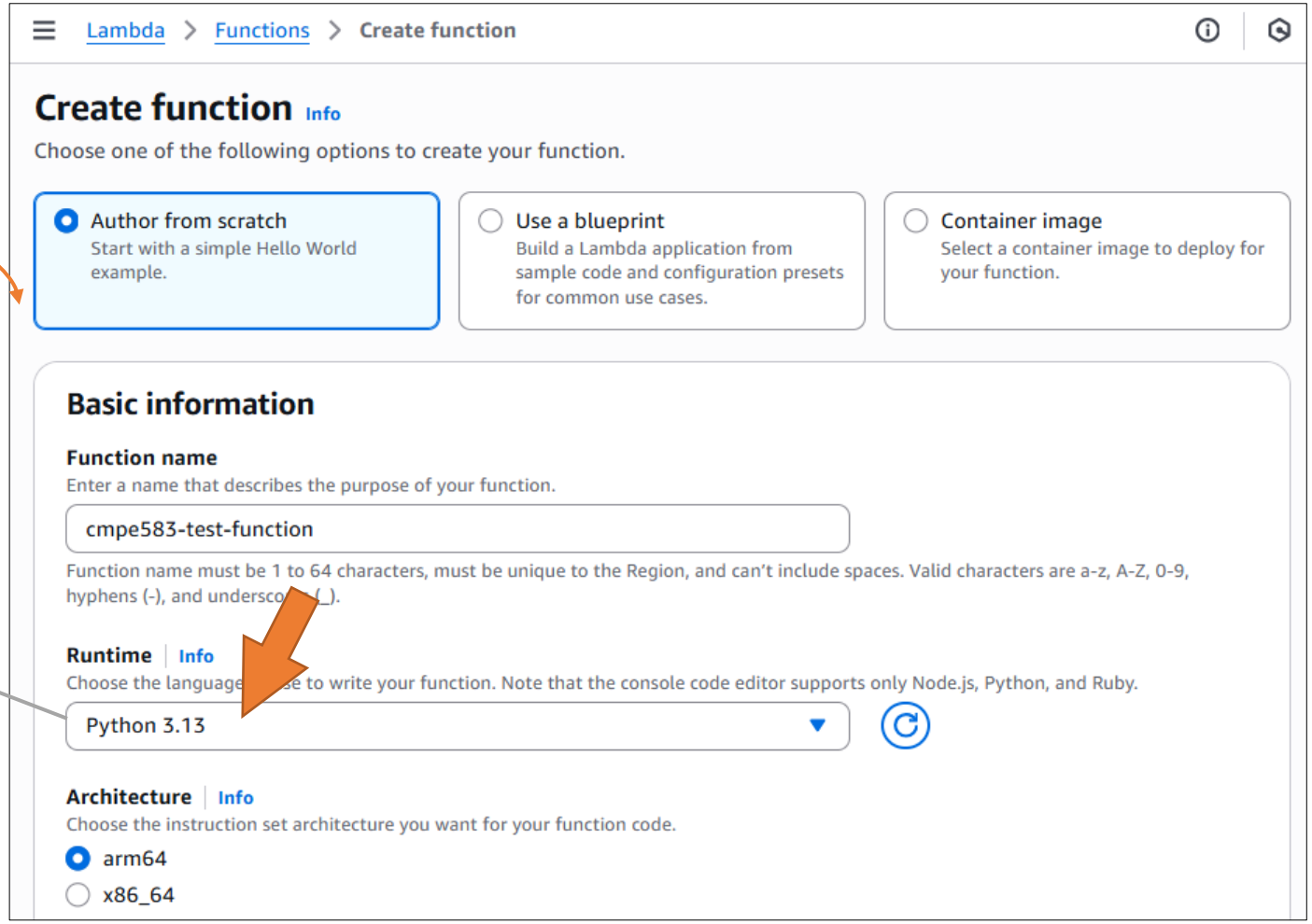
Steps for Deploying a Component Using Lambda

1. Create an AWS Lambda function and publish it.
2. Create a custom component using the Lambda function.
3. Deploy your Greengrass component and define a trigger function.
4. Verify and test the Greengrass component.

Step 1: Create AWS Lambda Function

Via AWS Lambda (Console)





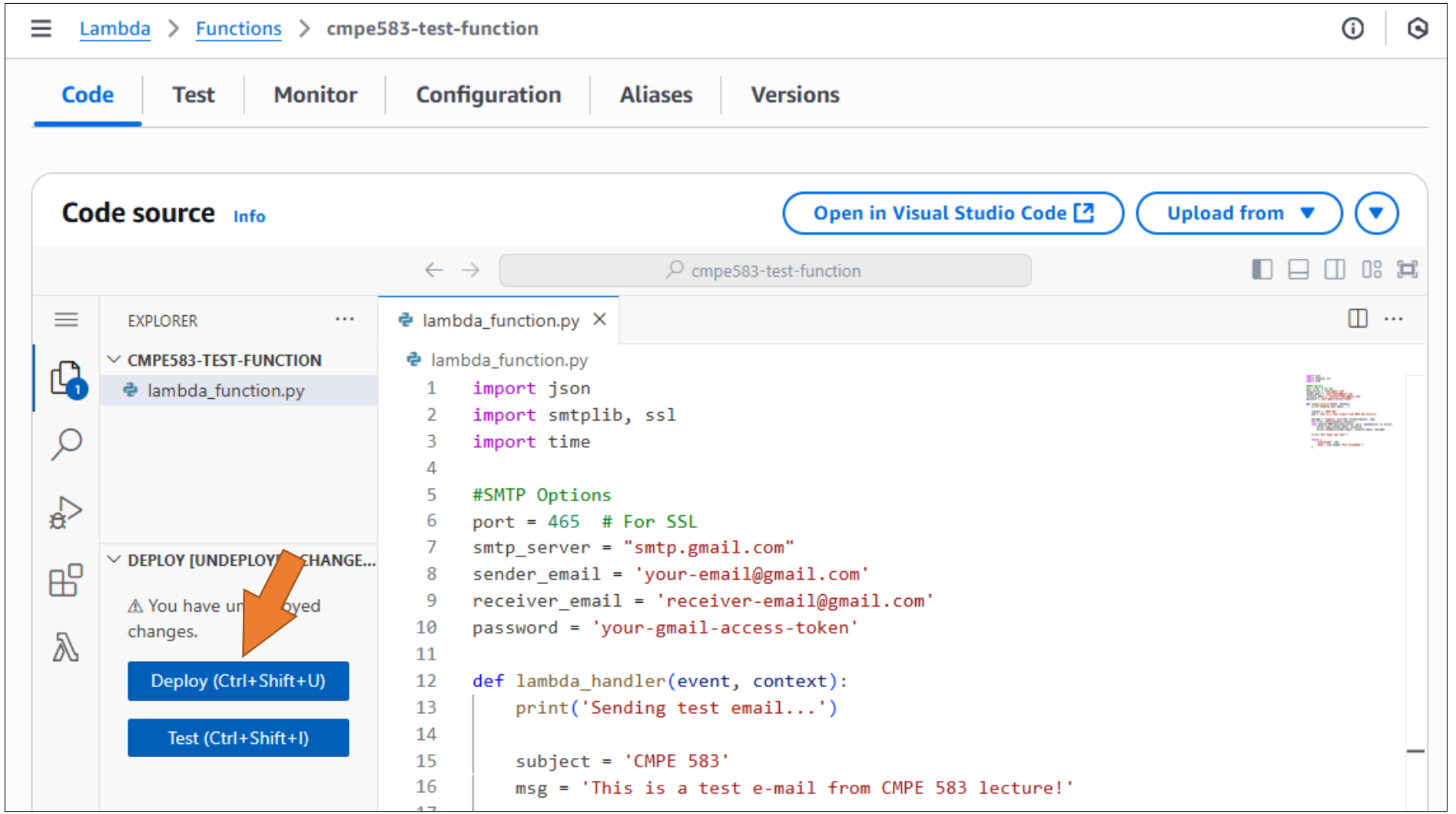
Potential Issues:

Runtime Version Mismatch

Try to use same Runtime and Architecture on AWS and your Core Device! Otherwise use FunctionRuntimeOverride in your Greengrass configuration

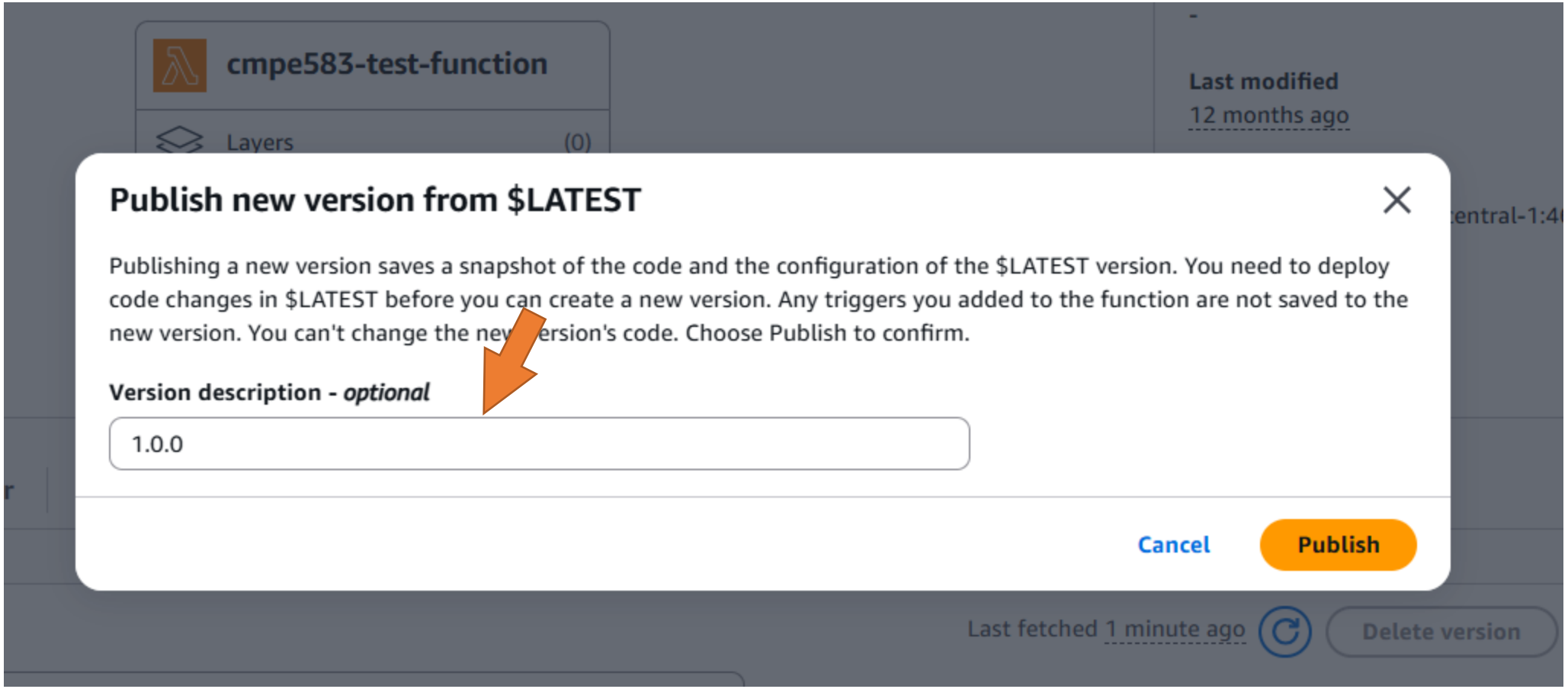
Step 1: Create AWS Lambda Function

Deploy the Lambda Function



Step 1: Create AWS Lambda Function

Publish the First Version

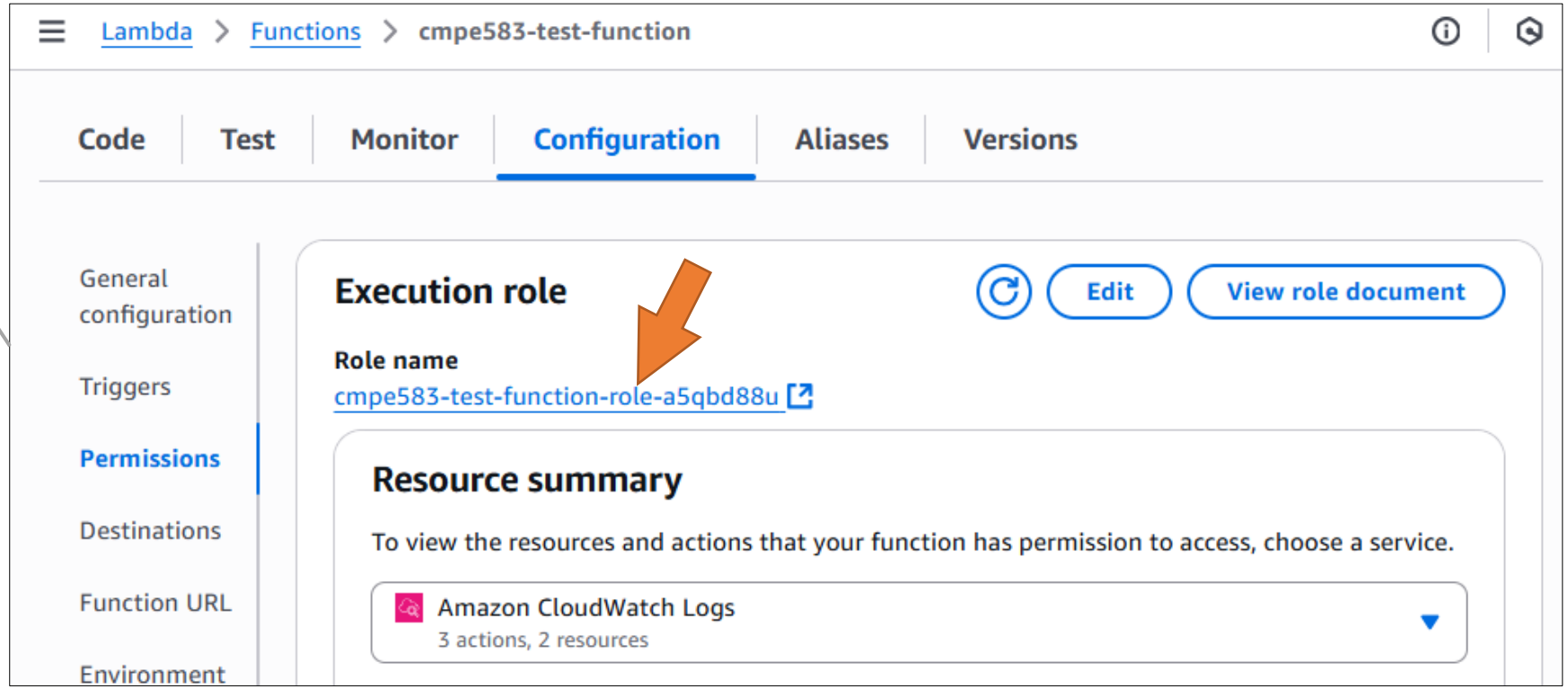


Step 1: Create AWS Lambda Function

Configure Lambda Role

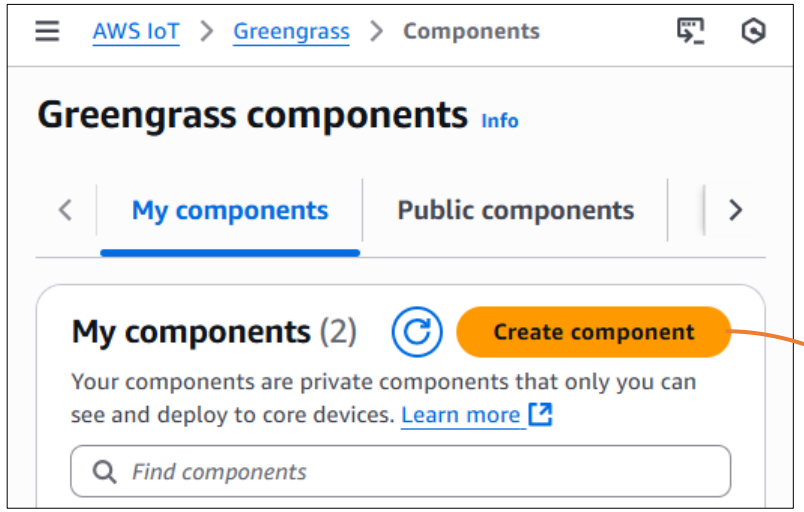
- Our lambda function is very simple for testing purpose, but if you have a more complex function that needs to access AWS resources (server, S3 file, database, etc.), you will need to configure the lambda's role accordingly.

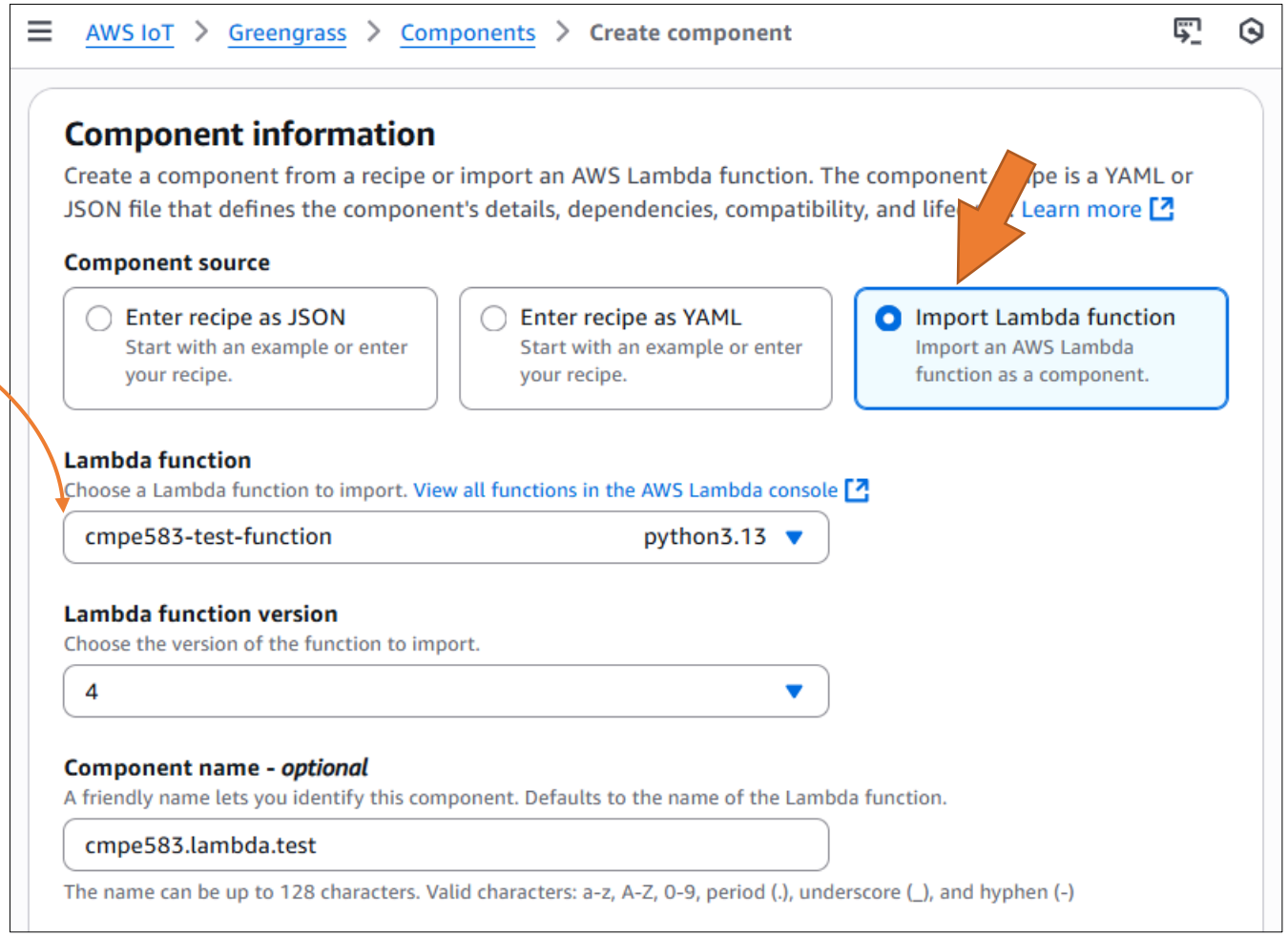
Notice:
Cloud Lambda uses its execution role, while Lambda functions running on Greengrass use the core device's token exchange role to access AWS services.



Step 2: Create a Custom Component

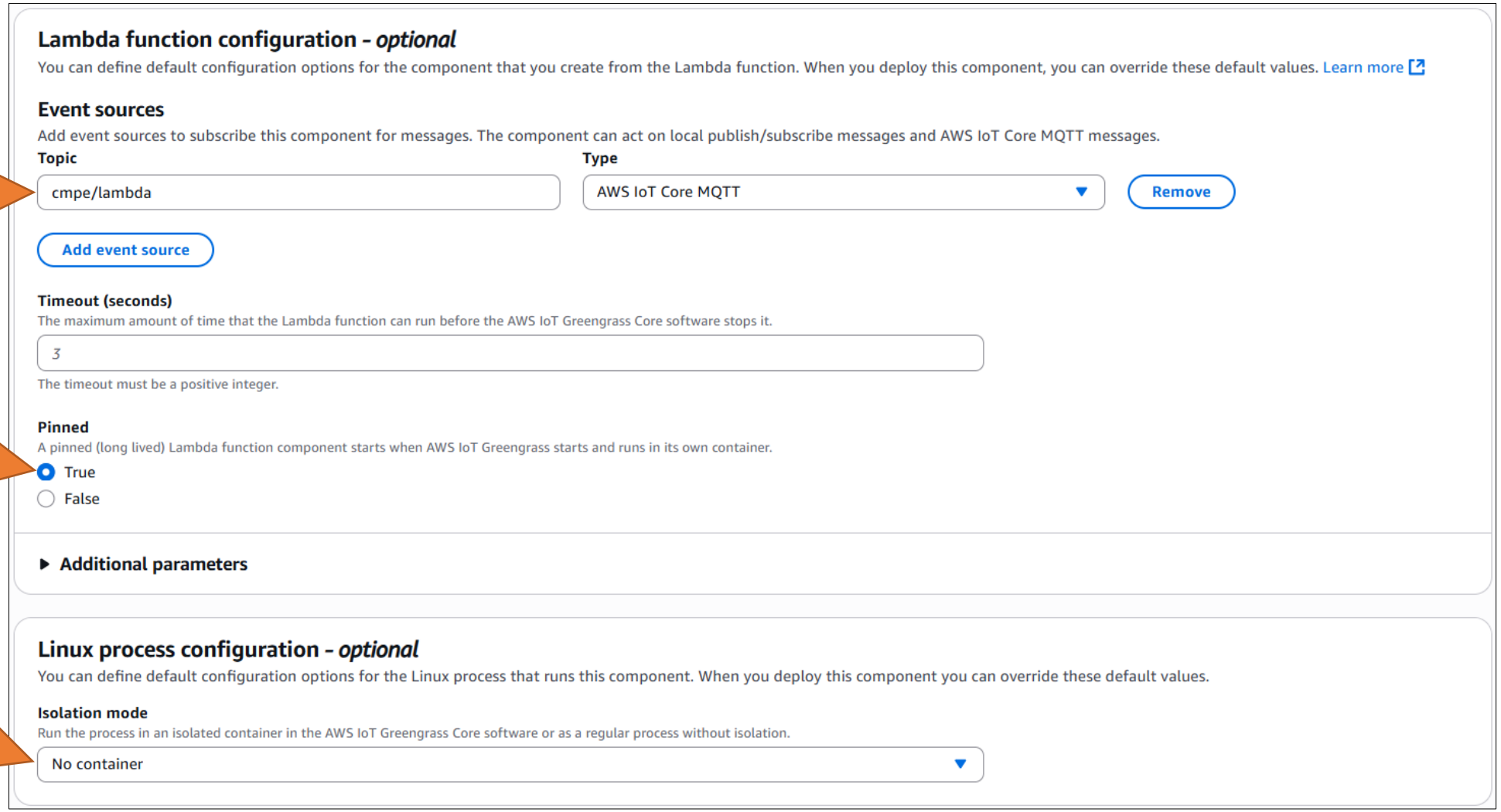
Import Lambda Function





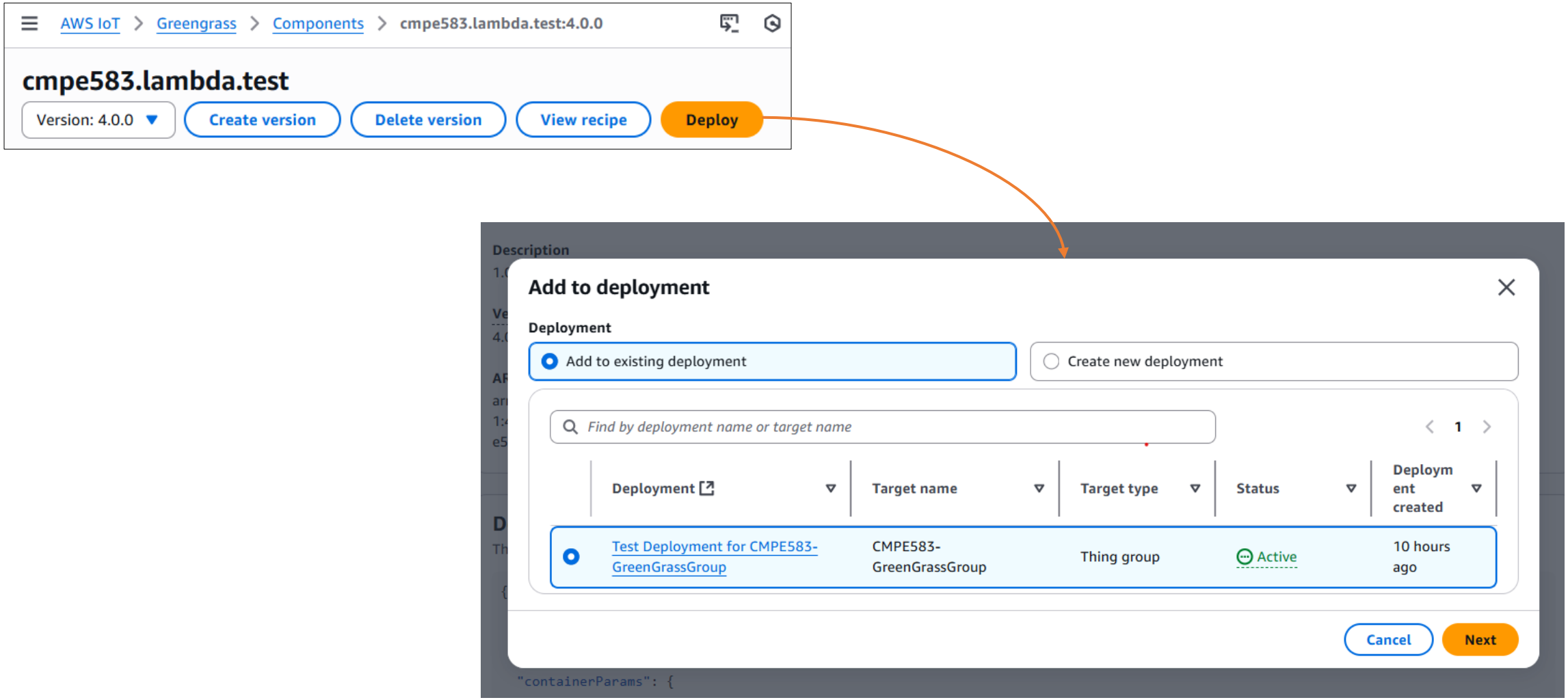
Step 2: Create a Custom Component

Configure Event Source (to Trigger Function) and Lifecycle



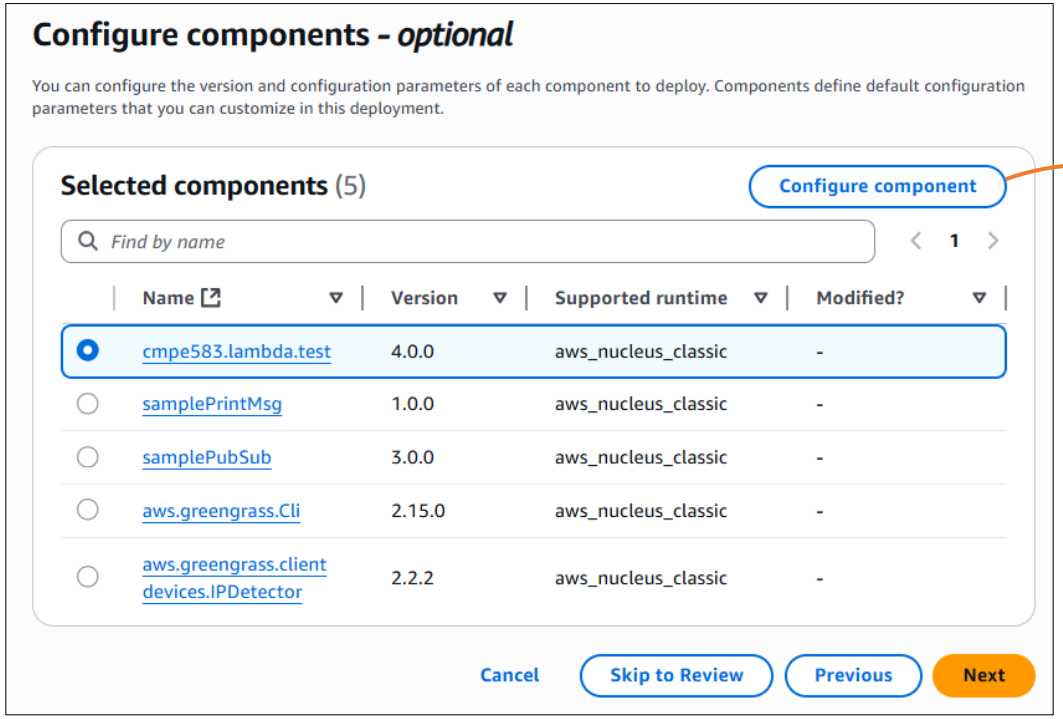
Step 3: Deploy Your Component

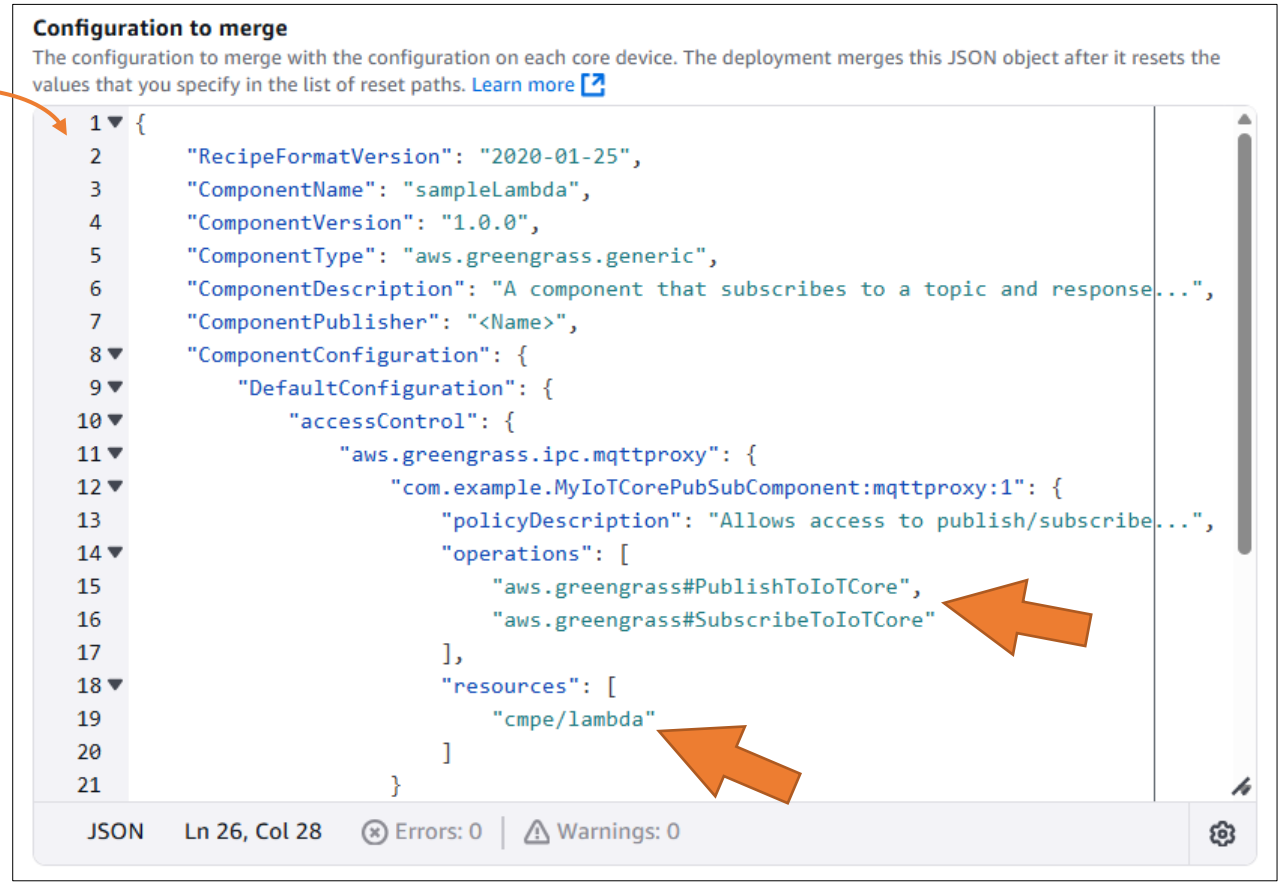
Add Component to Existing Deployment



Step 3: Deploy Your Component

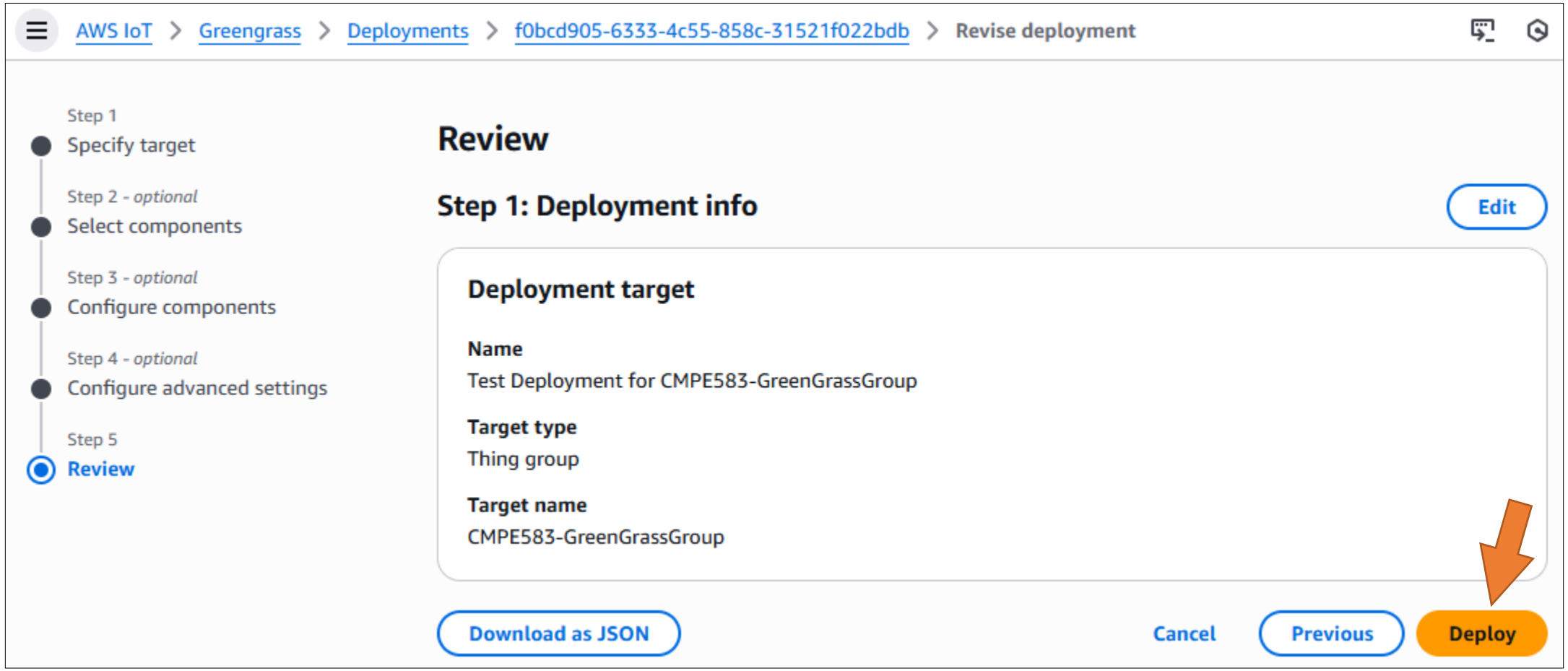
Configure Component to Merge Recipe





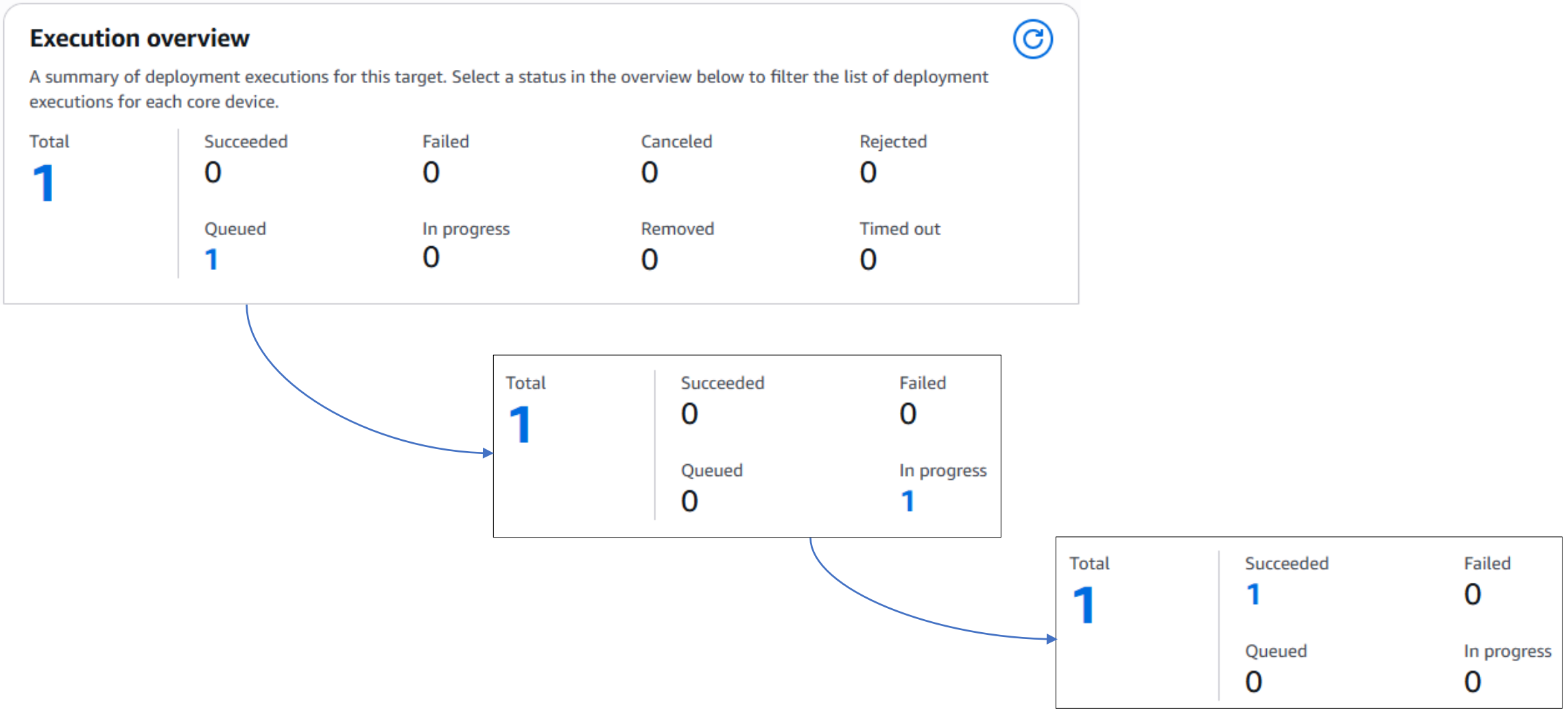
Step 3: Deploy the Component

Review & Deploy



Step 4: Verify the Component

Check AWS IoT Greengrass Console



Test Your Deployment

Via AWS MQTT Test Client

MQTT test client Info

You can use the MQTT test client to monitor the MQTT messages being passed in your AWS account. Devices publish MQTT messages that are identified by topics to communicate their state to AWS IoT. AWS IoT also publishes MQTT messages to inform devices and apps of changes and events. You can subscribe to MQTT message topics and publish MQTT messages to topics by using the MQTT test client.

Connection details Connected — Disconnect

To disconnect from the MQTT test client, choose Disconnect. To re-establish a connection, you can update the details and choose Connect.

Subscribe to a topic | Publish to a topic

Topic name

The topic name identifies the message. The message payload will be published to this topic with a Quality of Service (QoS) of 0.

cmpe/lambda

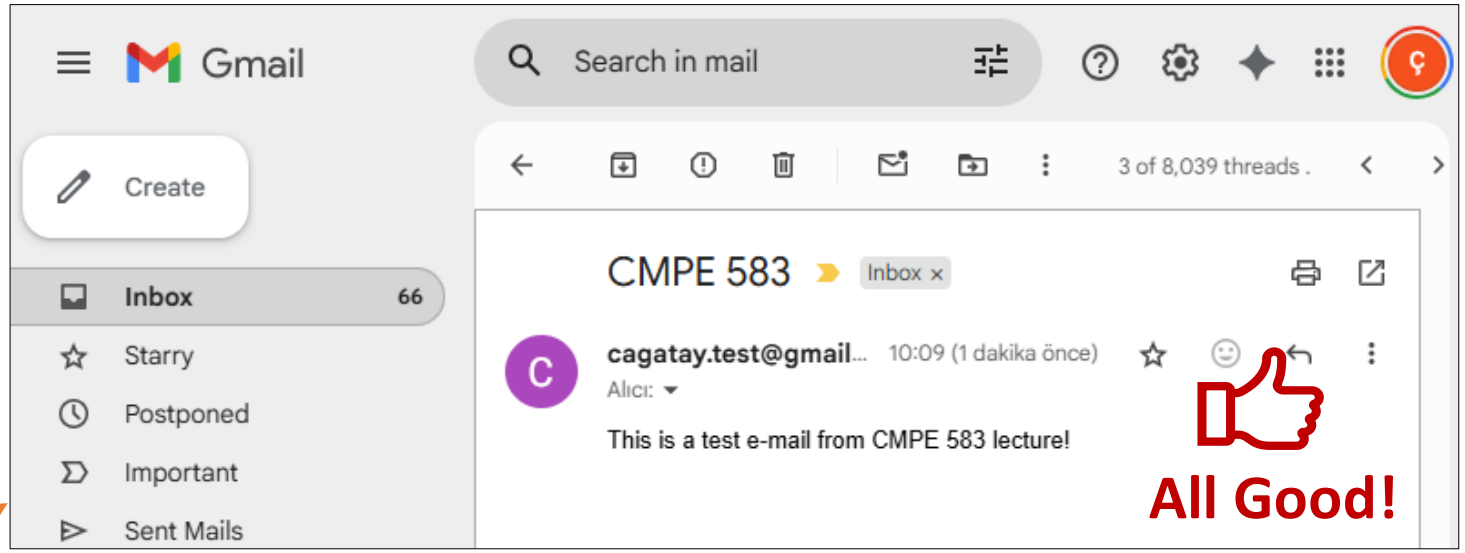
Message payload

```
{
  "message": "test"
}
```

Additional configuration

Publish

Payload will be send to *lambda_handler* function as an *event* parameter.



Test Your Deployment

Verify From The Core Device's Logs

```
root@kubuntu:/home/cagatay/Projects/greengrass-demo# tail -n 100 -f /greengrass/v2/logs/cmpe583.lamda.test.log
2023-10-29T14:51:24.654Z [INFO] (pool-2-thread-52) cmpe583.lamda.test: shell-runner-start. {scriptName=services.cmpe583.lamda.test.lifecycle.startup.script, serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=STARTING, command=["/greengrass/v2/packages/artifacts/aws.greengrass.LambdaLauncher/2.0.12/lambda-..."]}
2023-10-29T14:51:24.821Z [INFO] (Copier) cmpe583.lamda.test: stdout. Started process: 3385. {scriptName=services.cmpe583.lamda.test.lifecycle.startup.script, serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=STARTING}
2023-10-29T14:51:24.824Z [INFO] (Copier) cmpe583.lamda.test: Startup script exited. {exitCode=0, serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=STARTING}
2023-10-29T14:51:24.924Z [INFO] (pool-2-thread-55) cmpe583.lamda.test: lambda_runtime.py:402,Status thread started. {serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=RUNNING}
2023-10-29T14:51:24.931Z [INFO] (pool-2-thread-55) cmpe583.lamda.test: lambda_runtime.py:154,Running [arn:aws:lambda:eu-central-1:465822364162:function:cmpe583-test-function:3]. {serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=RUNNING}
2023-10-29T14:51:25.052Z [INFO] (pool-2-thread-55) cmpe583.lamda.test: lambda_function.py:13,Sending test email.... {serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=RUNNING}
2023-10-29T14:51:26.757Z [INFO] (pool-2-thread-55) cmpe583.lamda.test: lambda_function.py:24,test email was sent!. {serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=RUNNING}
2023-10-29T14:51:54.581Z [INFO] (pool-2-thread-60) cmpe583.lamda.test: shell-runner-start. {scriptName=services.cmpe583.lamda.test.lifecycle.shutdown.script, serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=STOPPING, command=["/greengrass/v2/packages/artifacts/aws.greengrass.LambdaLauncher/2.0.12/lambda-..."]}
2023-10-29T14:51:54.740Z [INFO] (pool-2-thread-55) cmpe583.lamda.test: lambda_runtime.py:370,Caught signal 15. Stopping runtime.. {serviceInstance=1, serviceName=cmpe583.lamda.test, currentState=STOPPING}
```

Interact with Local IoT Devices

Local IoT Devices

- **Client devices** are local IoT devices that connect to and communicate with a Greengrass **core device** over MQTT.

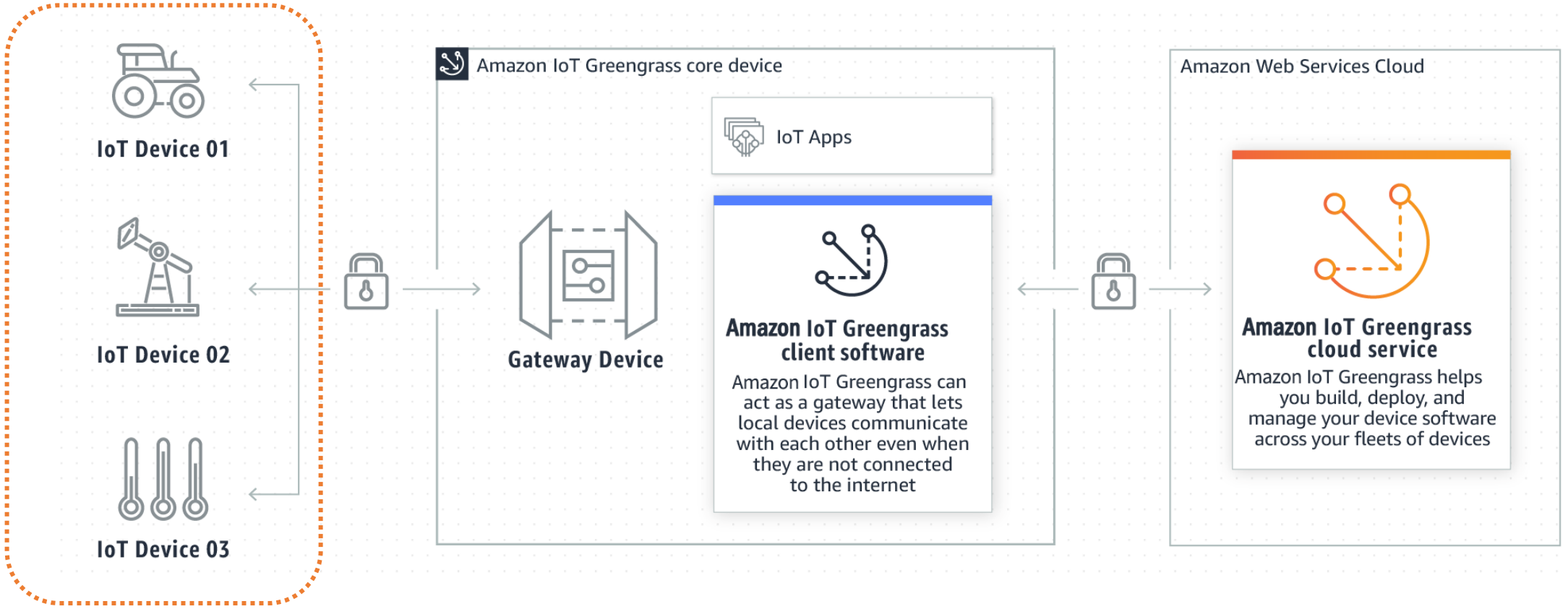


image source: https://aws.amazon.com/greengrass/

Connect Client Devices to Core Devices

- You must configure **cloud discovery** to connect client devices to core devices.
- Associate client devices with a core device so that they can discover the core device.
- Use the AWS IoT Device SDK on **client devices** to discover, connect, and communicate with a core device.
- You can perform both local (edge) and remote (cloud) operations:
  - Local (Edge) Operations:
    - Local MQTT messaging with Greengrass components and client devices.
    - Interact with client device (local) shadows in Greengrass components.
  - Remote (Cloud) Operations:
    - Relay MQTT messages between client devices and AWS IoT Core.
    - Sync client devices (remote) shadows with AWS IoT Core.

AWS-Provided Client Device Components

- AWS provides the following public components that you can deploy to **core devices**.
- These components enable **client devices** to connect & communicate with core devices.

Component	Description	Component type	Supported OS	Open source
Client device auth	Enables local IoT devices, called client devices, to connect to the core device.	Plugin	Linux, Windows	Yes
IP detector	Reports MQTT broker connectivity information to AWS IoT Greengrass, so client devices can discover how to connect.	Plugin	Linux, Windows	Yes
MQTT bridge	Relays MQTT messages between client devices, local AWS IoT Greengrass publish/subscribe, and AWS IoT Core.	Plugin	Linux, Windows	Yes
MQTT 3.1.1 broker (Moquette)	Runs an MQTT 3.1.1 broker that handles messages between client devices and the core device.	Plugin	Linux, Windows	Yes
MQTT 5 broker (EMQX)	Runs an MQTT 5 broker that handles messages between client devices and the core device.	Generic	Linux, Windows	No
Shadow manager	Enables interaction with shadows on the core device. It manages shadow document storage and also the synchronization of local shadow states with the AWS IoT Device Shadow service.	Plugin	Linux, Windows	Yes

Creating Client Devices

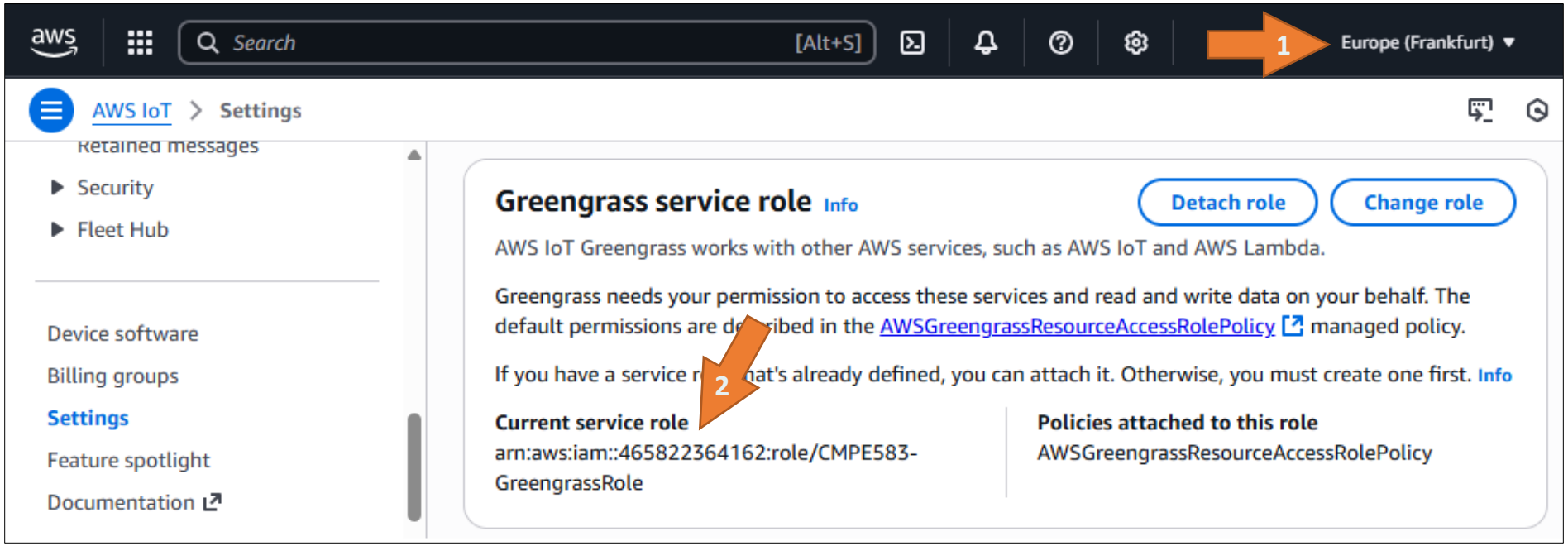
Steps for Creating a Client Device

1. Configure the Greengrass Service Role.
2. Configure the AWS IoT Thing Policy.
3. Create the Client Device (Thing) via AWS IoT Console.
4. Download Certificates and Keys (needed for secure auth. and communication).

Step 1: Configure the Greengrass Service Role

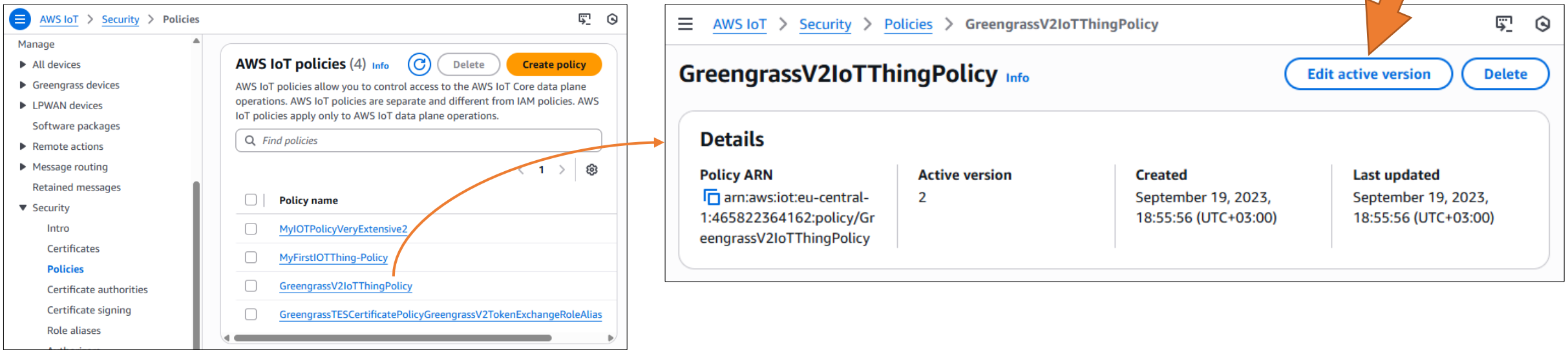
Verify If Greengrass Service Role is Attached

- The Greengrass service role is an IAM service role that authorizes AWS IoT Greengrass to access resources from AWS services on your behalf.
- Check whether the Greengrass service role is set up in AWS IoT console's settings.



Step 2: Configure the AWS IoT Thing Policy

Review the Policy

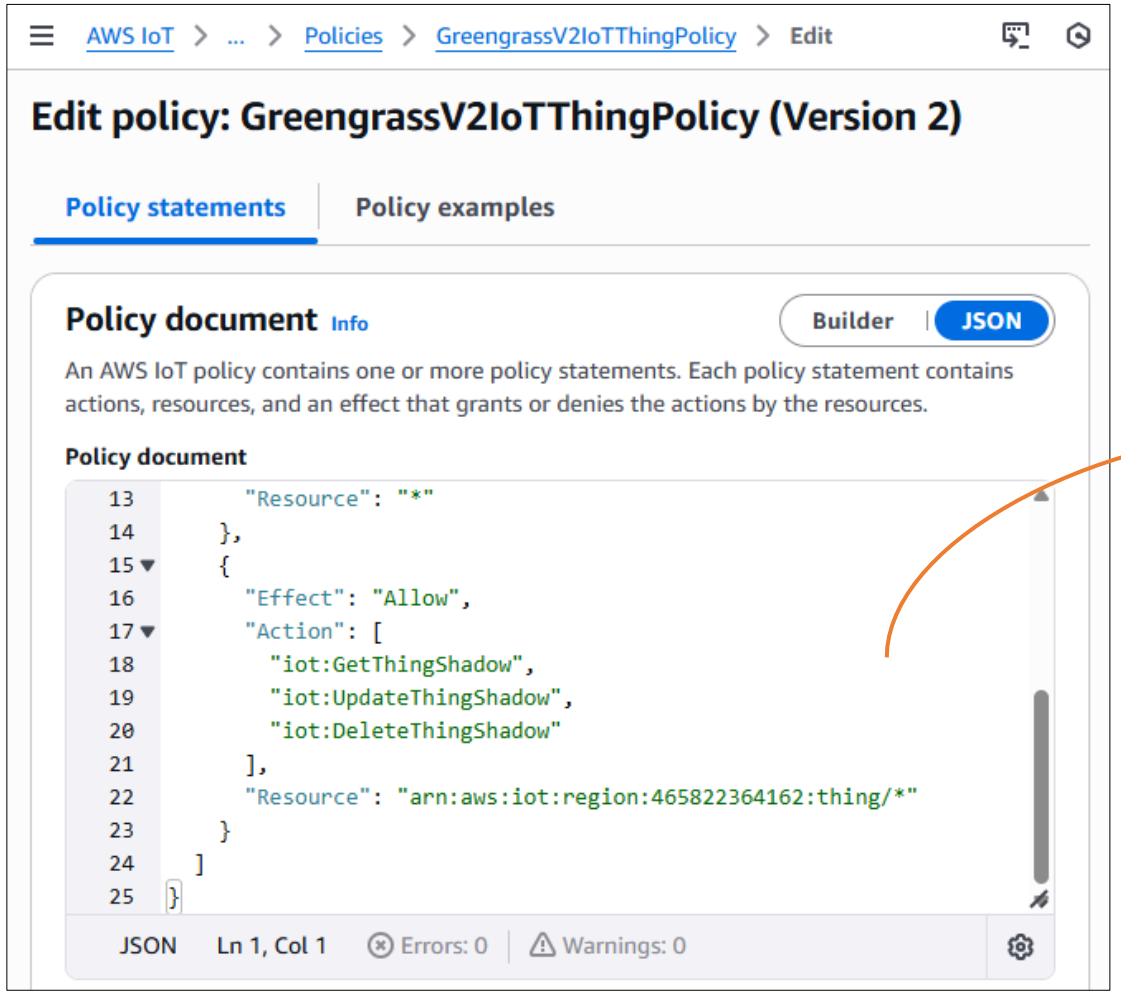


➢Review the policy for required permissions, and add any required permissions that are missing:

- greengrass:PutCertificateAuthorities
- greengrass:VerifyClientDeviceIdentity
- greengrass:VerifyClientDeviceIoTCertificateAssociation
- greengrass:GetConnectivityInfo

Step 2: Configure the AWS IoT Thing Policy

Optional: Allow Core Device to Sync Shadows

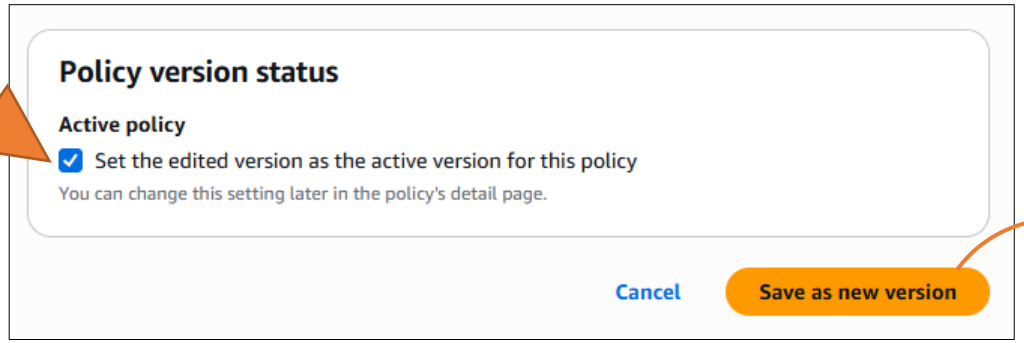


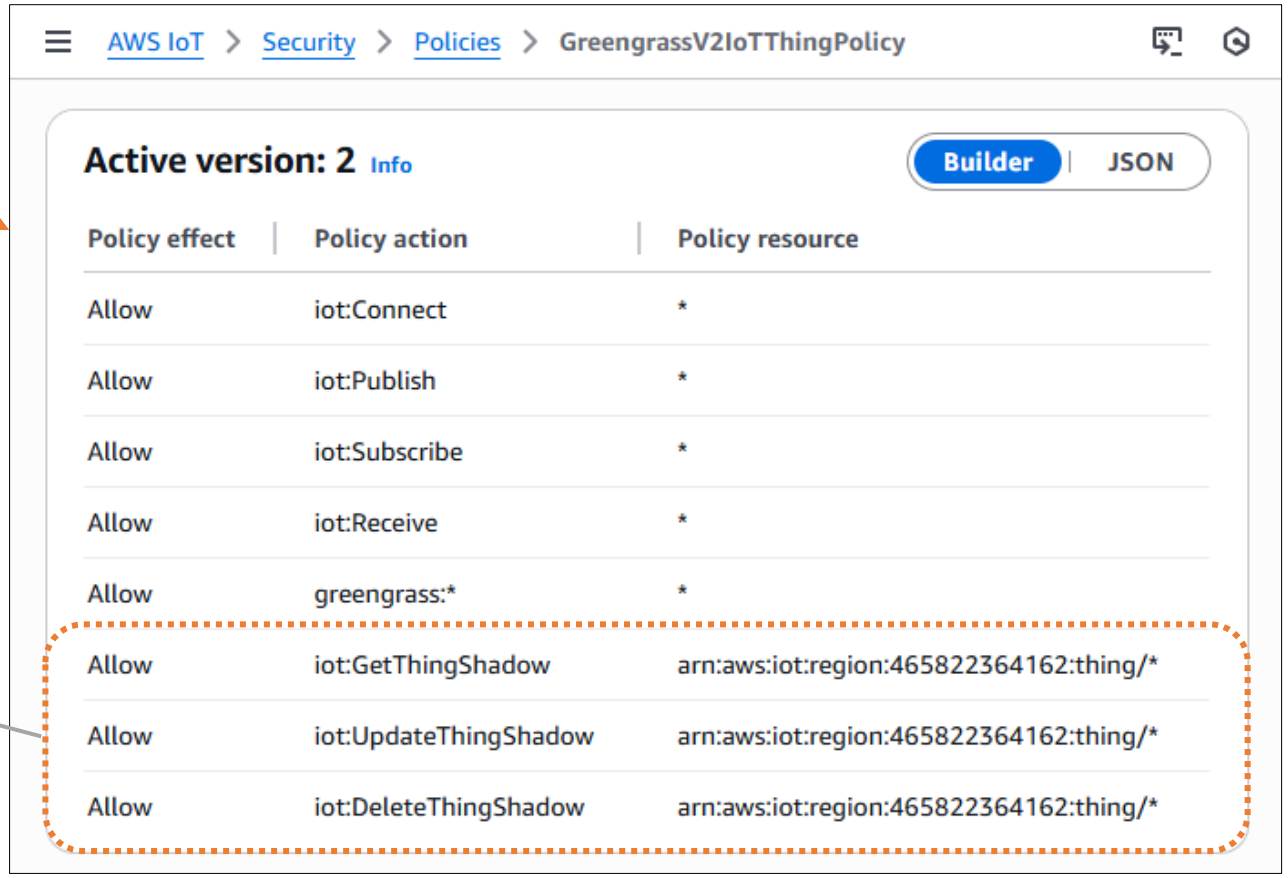
To allow the core device to sync shadows with AWS IoT Core, add the following statement to the policy:

```
{
   "Effect": "Allow",
   "Action": [
      "iot:GetThingShadow",
      "iot:UpdateThingShadow",
      "iot:DeleteThingShadow" ],
   "Resource": [
      "arn:aws:iot:region:account-id:thing/*"
   ]
}
```

Step 2: Configure the AWS IoT Thing Policy

Update & Verify Policy

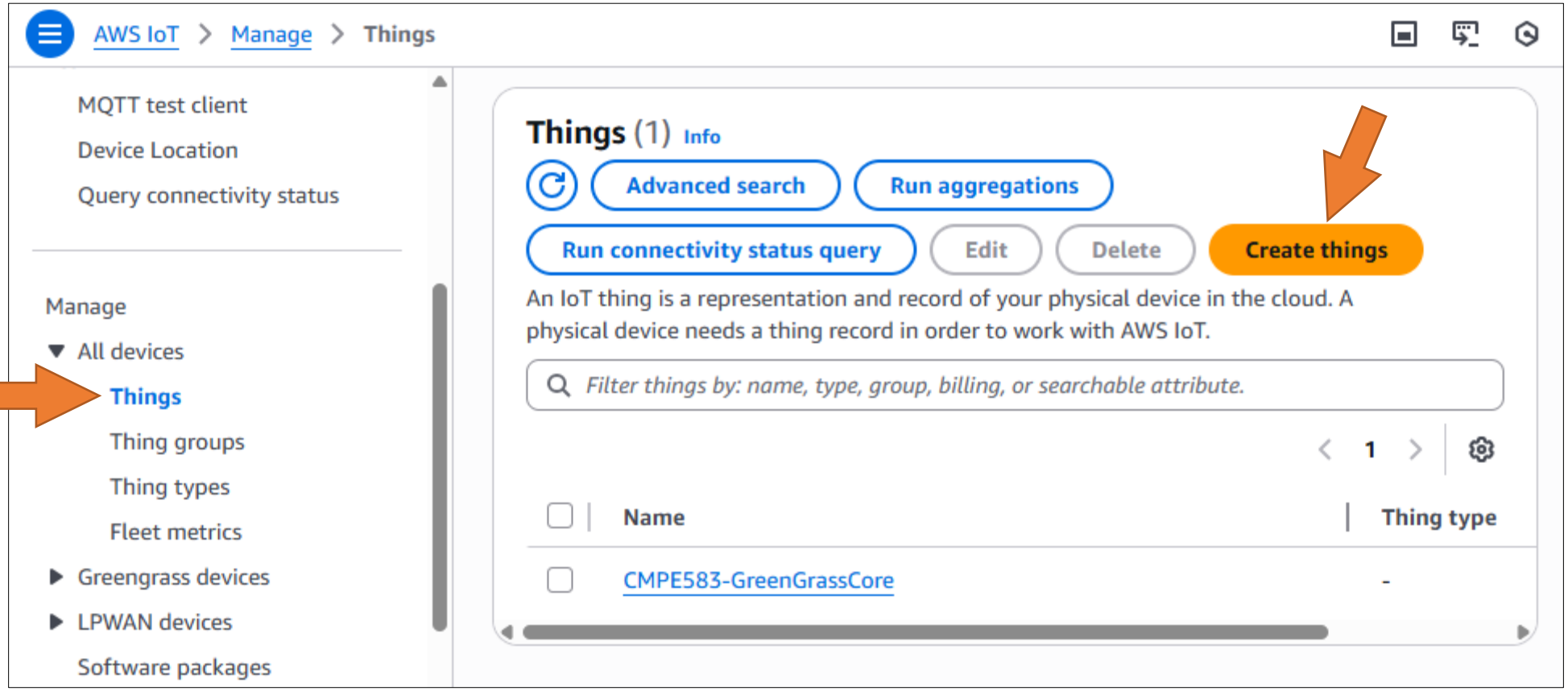




🚨 **Security Best Practice: Principle of Least Privilege**
Only add these permissions if your application specifically uses the Device Shadow service.

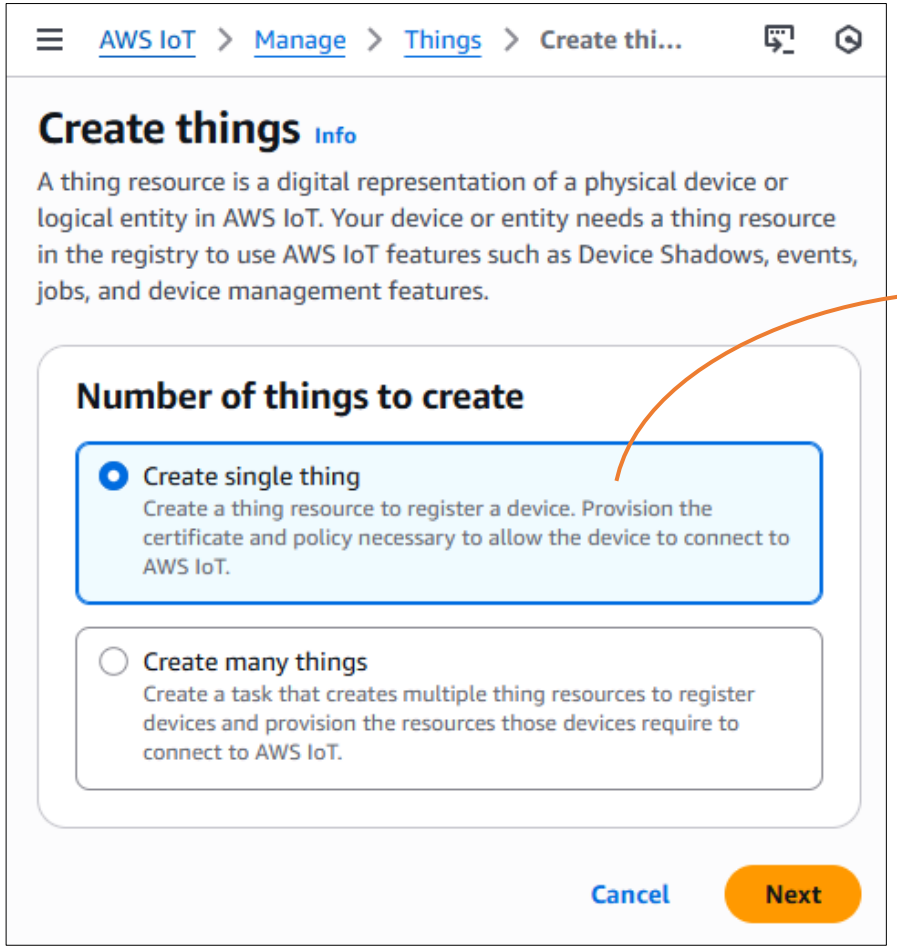
Step 3: Create Client Device (Thing)

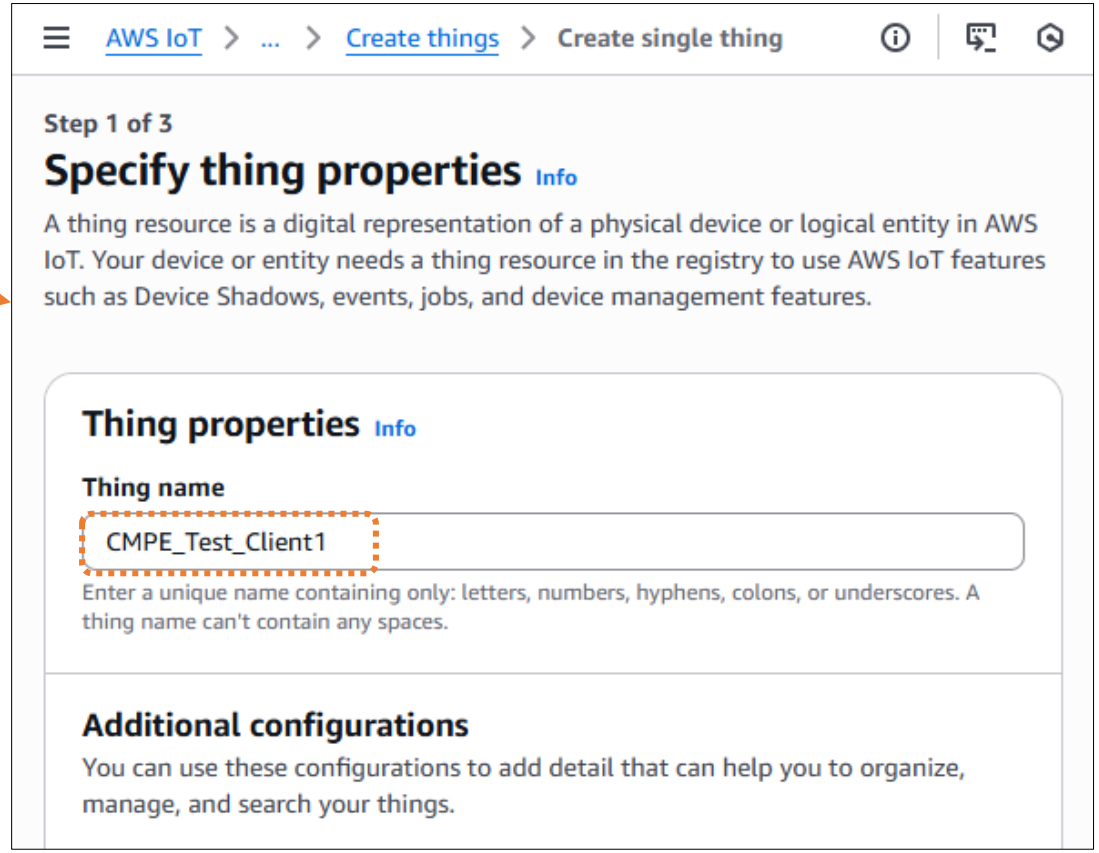
Via AWS IoT Console



Step 3: Create Client Device (Thing)

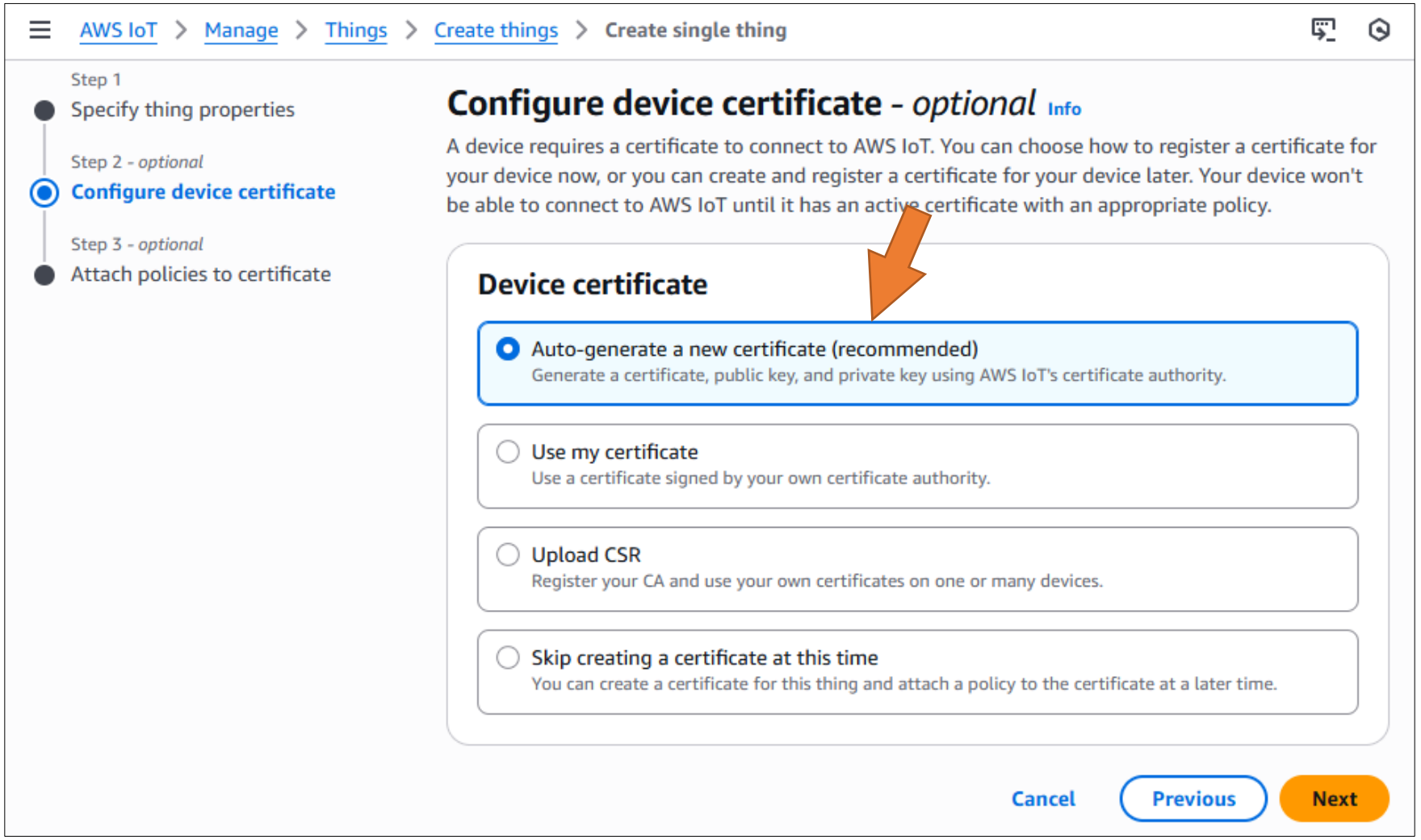
Via AWS IoT Console





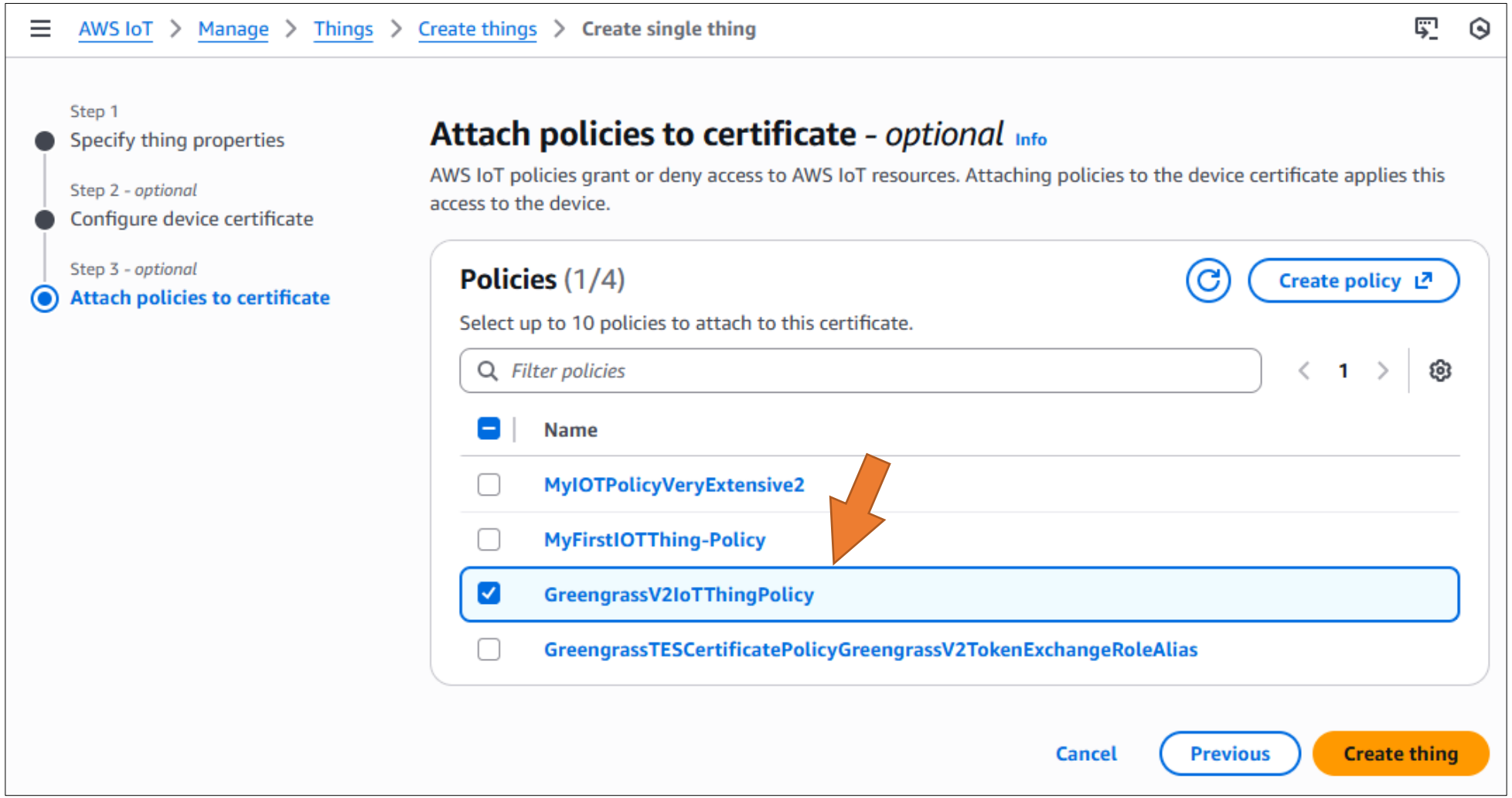
Step 3: Create Client Device (Thing)

Configure Device Certificate

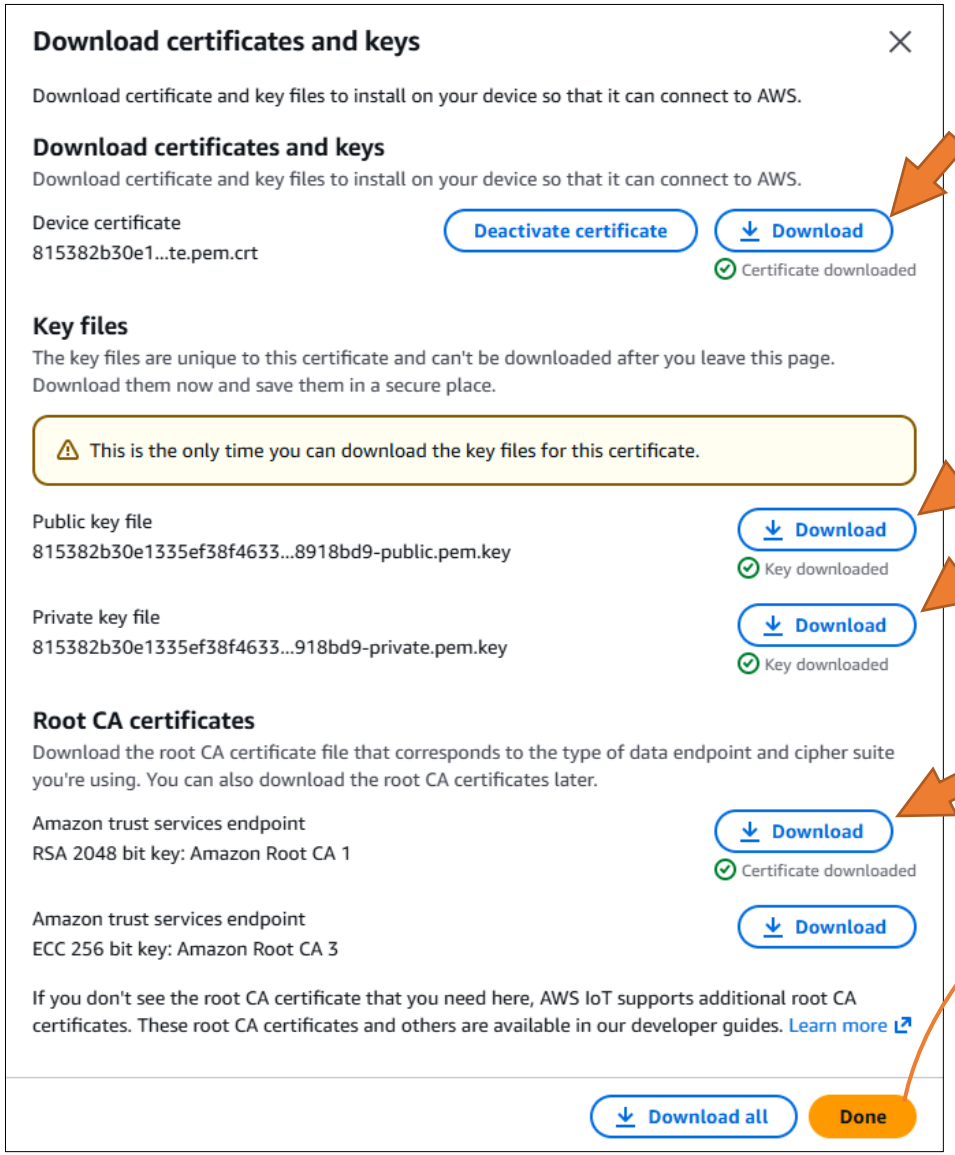


Step 3: Create Client Device (Thing)

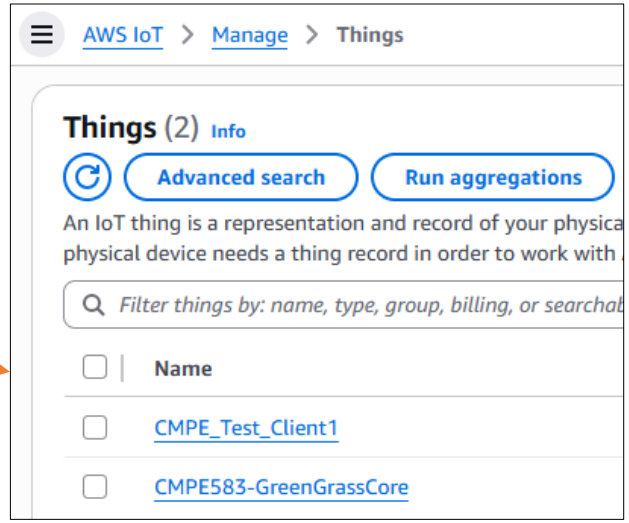
Configure Policy



Step 4: Download Certificates and Keys

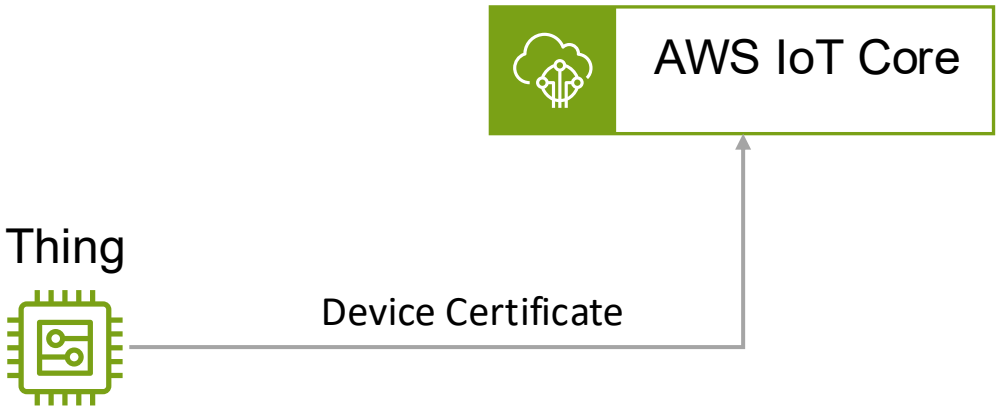


★ Key files must be downloaded in the last step of things creation!



How are Certificate and Keys Used?

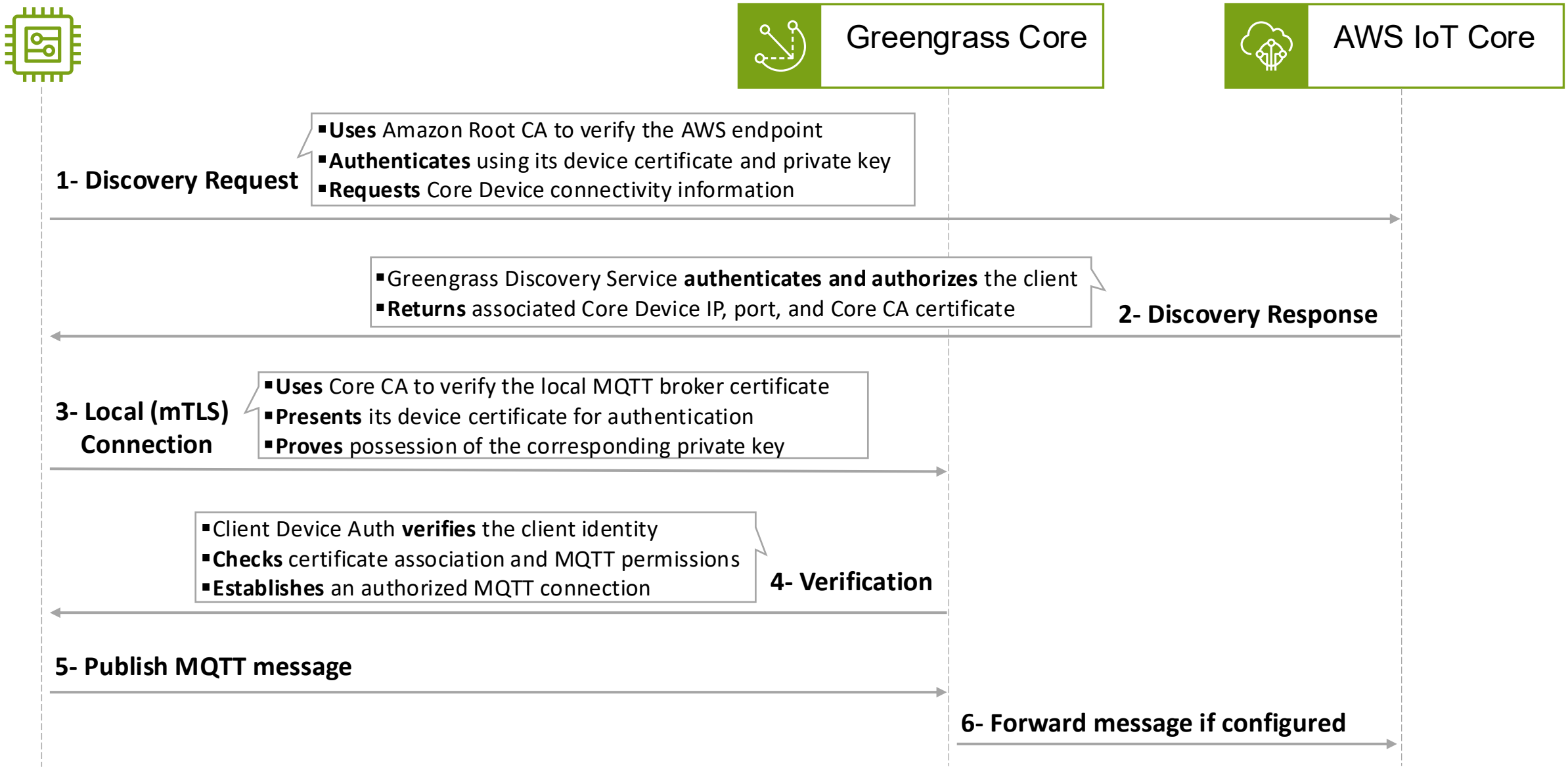
- **Private Key (Your Device's Secret)**: Should be stored only on the device, it is used to create a digital signature that authenticates the device's identity and prevents unauthorized access.
- **Public Key:** Not directly used, because it is already embedded in the device certificate.
- **Device Certificate (Your Device's Public ID)**: Contains the device's public key and identifying information, signed by AWS Root CA. This allows the server to verify the device's authenticity during TLS/mTLS connections.
- **Amazon Root CA Certificate (The Trusted Notary)**: Device stores Root CA to authenticate AWS IoT Core.



- **Validates** the device certificate using the AWS CA chain.
- **Authenticates** the Thing during the mutual TLS handshake by verifying its digital signature using the public key in the device's certificate.

- **Uses** its private key to authenticate itself during the TLS/mTLS handshake.
- **Verifies** AWS IoT Core's identity using the Amazon Root CA.

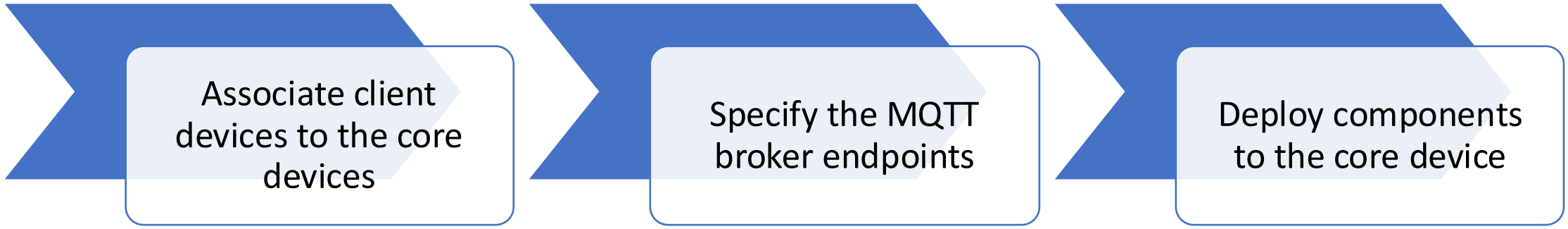
How are Certificate and Keys Used? Cont.



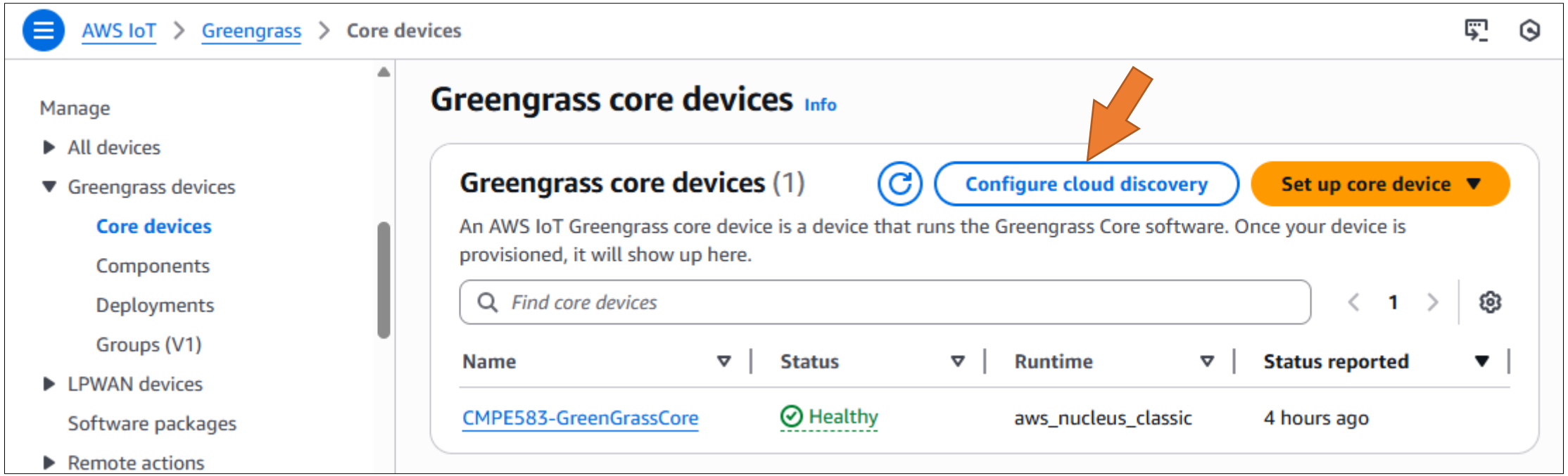
Configuring Cloud Discovery

Cloud Discovery Configuration

- You should configure cloud discovery to connect client devices to core devices.
- When you configure cloud discovery, client devices can connect to the AWS IoT Greengrass cloud service to retrieve information about core devices.
- Then, the client devices can attempt to connect to each core device until they successfully connect.
- To use cloud discovery, you must do the following:

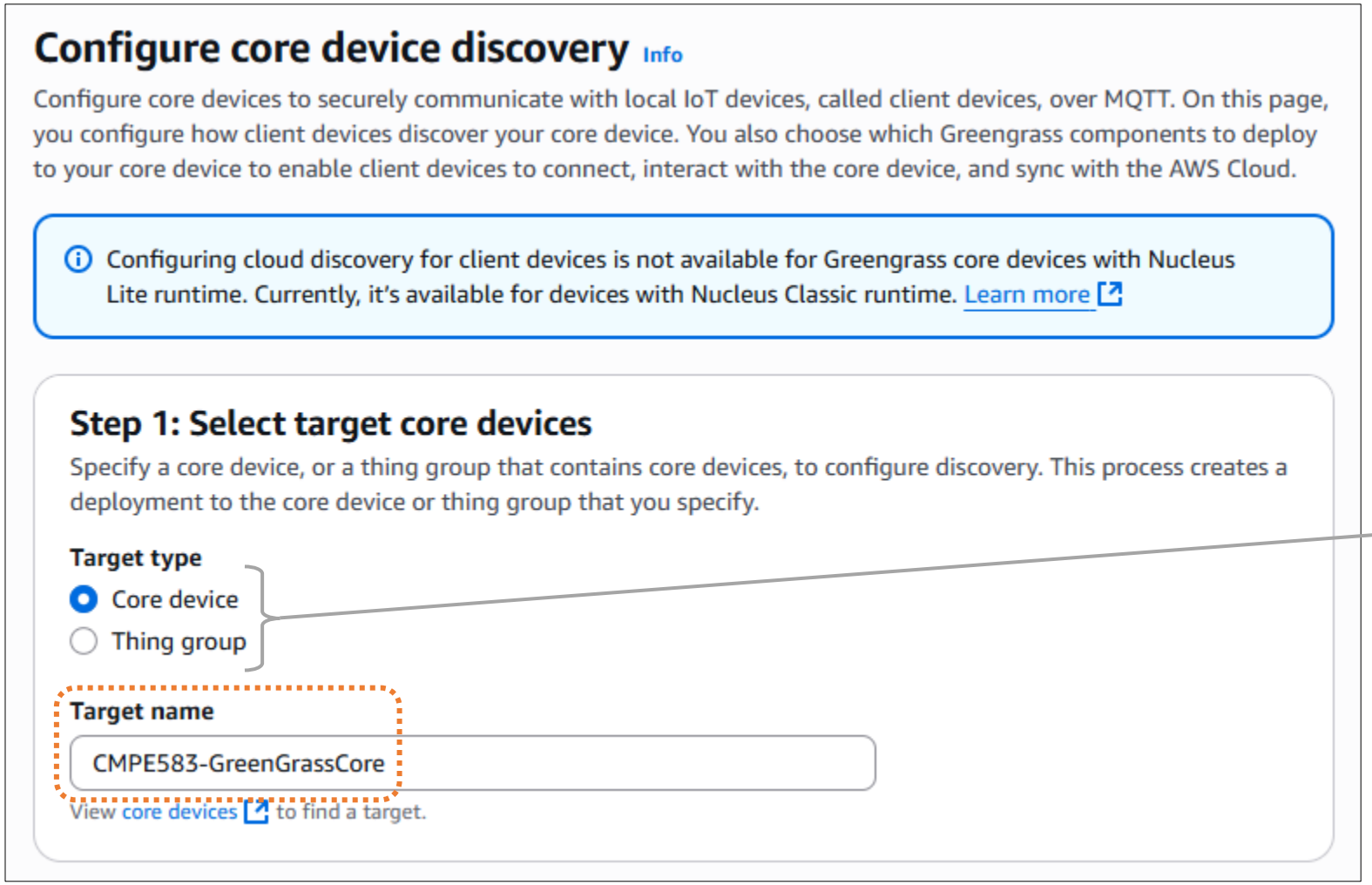


Step 1: Configure Cloud Discovery



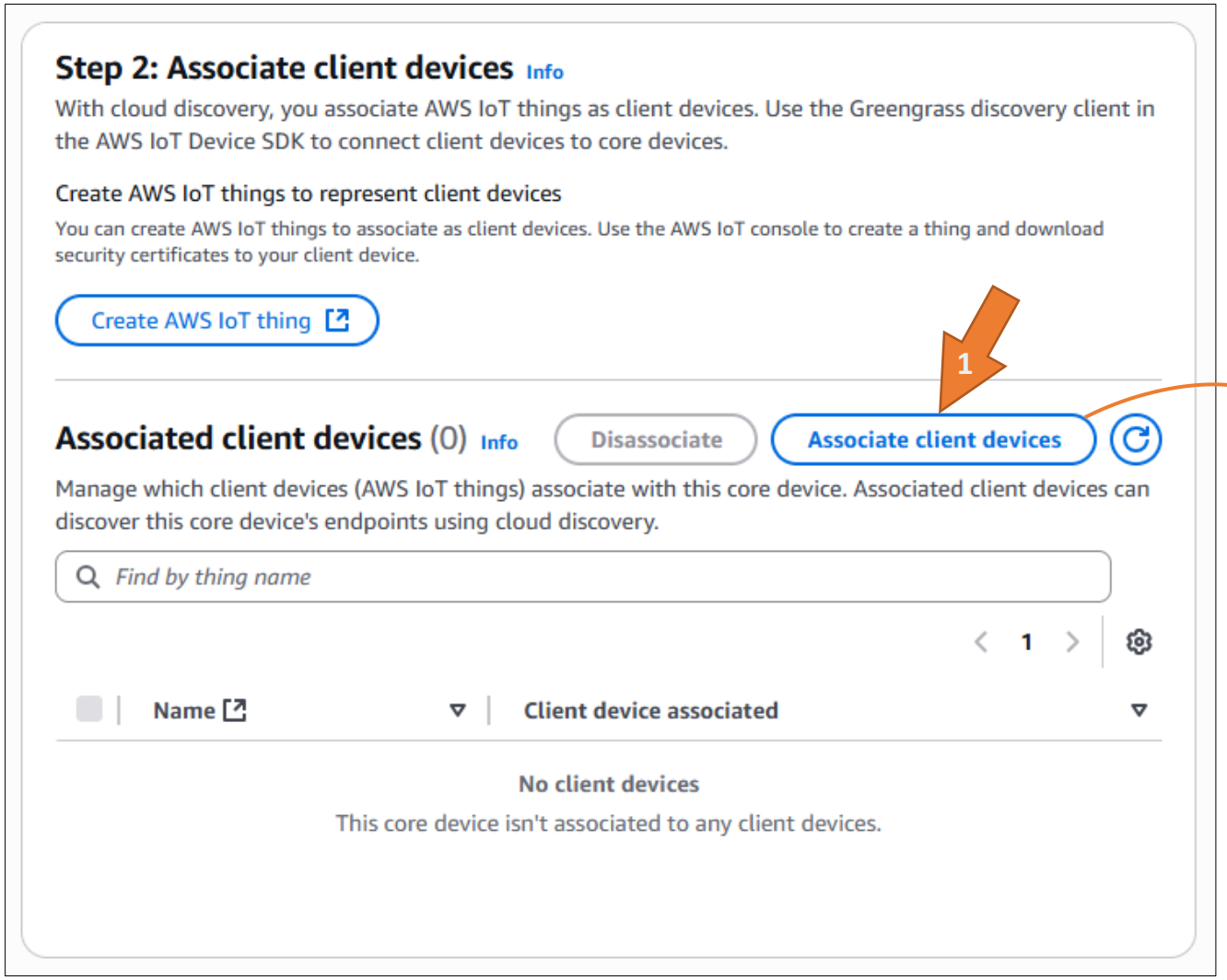
Step 1: Configure Cloud Discovery

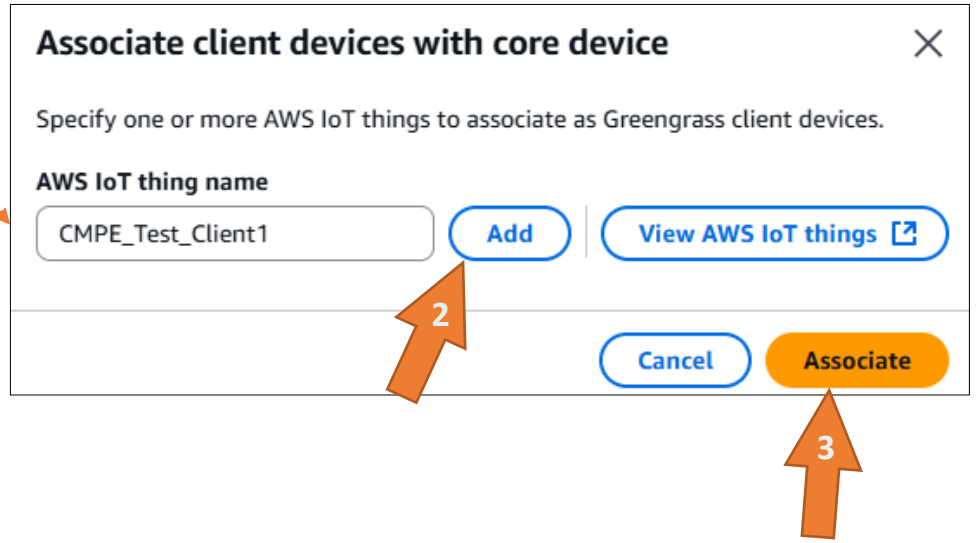
Select Target Core Devices



You can configure core device discovery for a core device or a thing group.

Step 2: Associate Client Devices



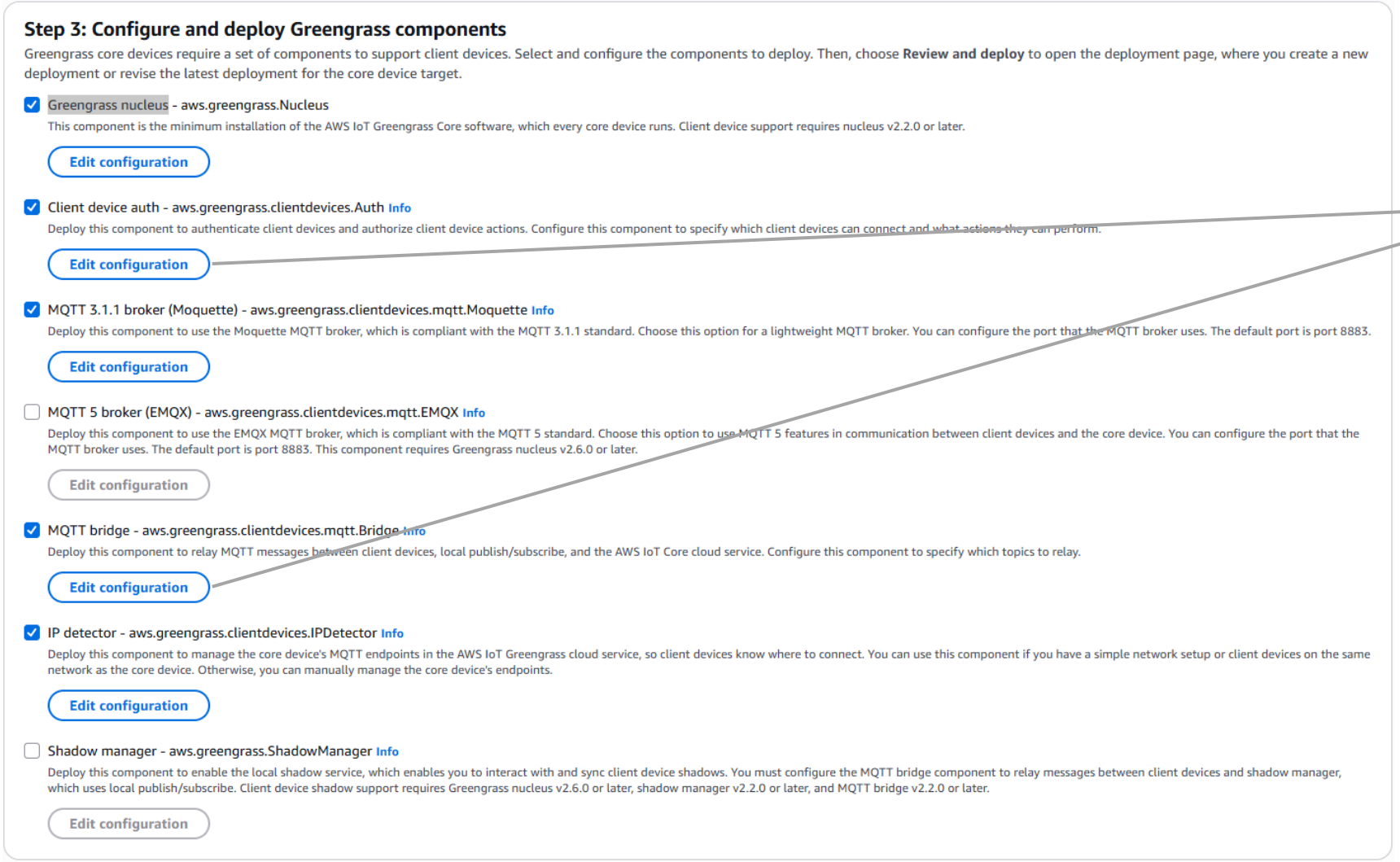


Step 3: Deploy Required Greengrass Components

- The following Greengrass components are required:
  - Greengrass nucleus (aws.greengrass.Nucleus)
  - Client device auth (aws.greengrass.clientdevices.Auth)
  - IP detector (aws.greengrass.clientdevices.IPDetector)
- You must also deploy ONE of the following local brokers:
  - MQTT 3.1.1 broker (Moquette) (aws.greengrass.clientdevices.mqtt.Moquette)
  - MQTT 5 broker (EMQX) (aws.greengrass.clientdevices.mqtt.EMQX)
- The following components are optional:
  - MQTT bridge (aws.greengrass.clientdevices.mqtt.Bridge)
  - Shadow manager (aws.greengrass.ShadowManager)

Step 3: Deploy Required Greengrass Components

Configure Required Greengrass Components

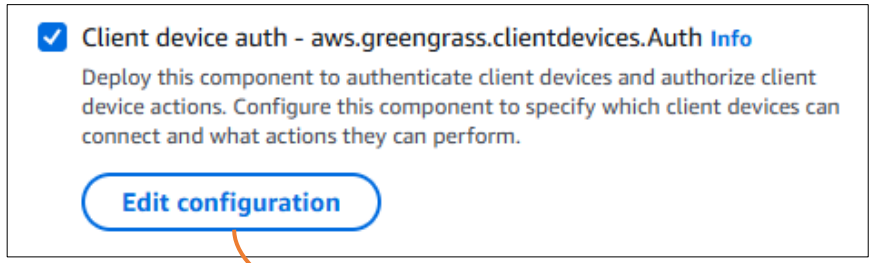


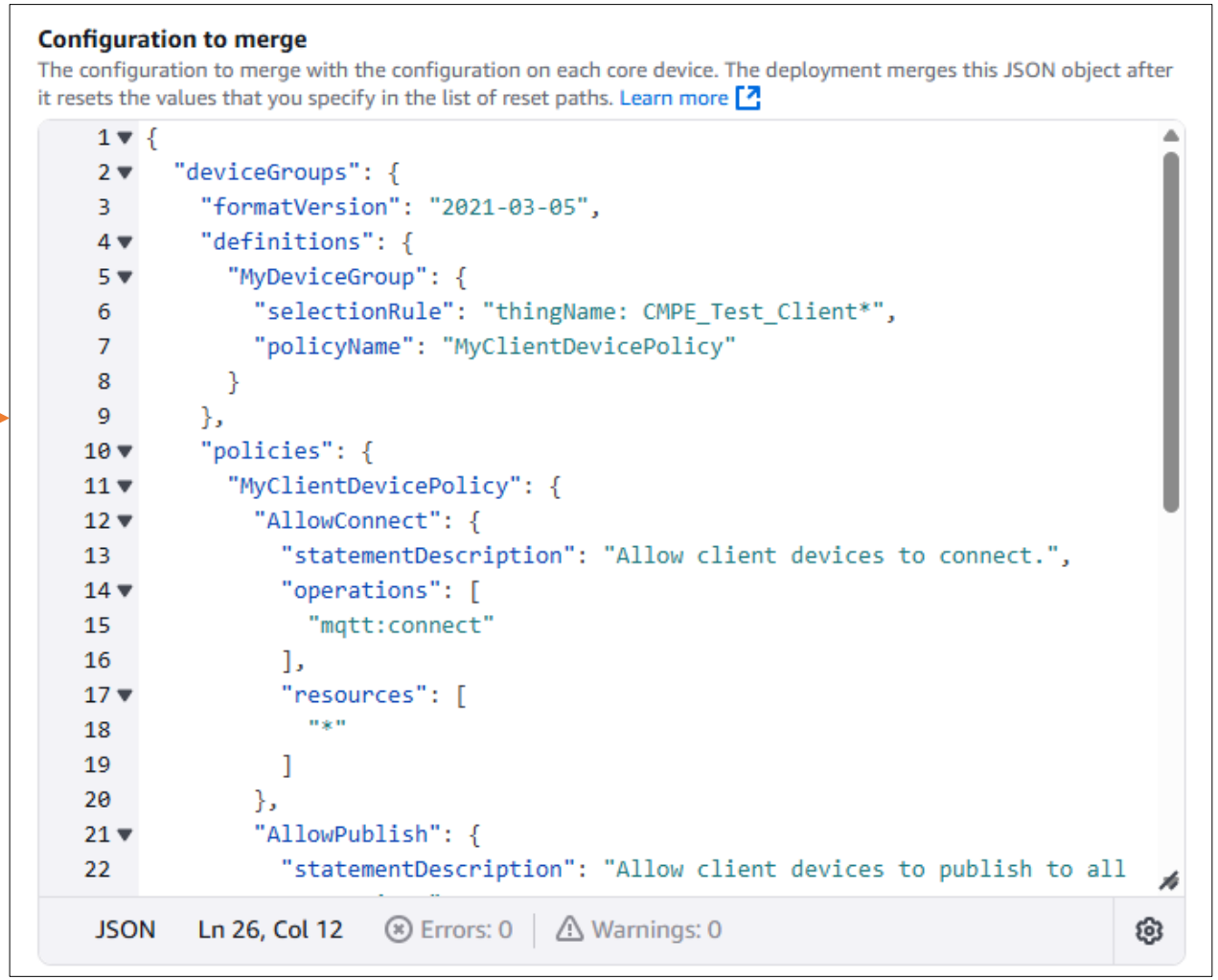
MQTT bridge and client device auth components should be configured before the deployment!

★ It is recommended to deploy only **one** MQTT broker component. If you deploy multiple MQTT broker components, you must configure them to use **different** ports!

Step 3: Deploy Required Greengrass Components

Customize aws.greengrass.clientdevices.Auth

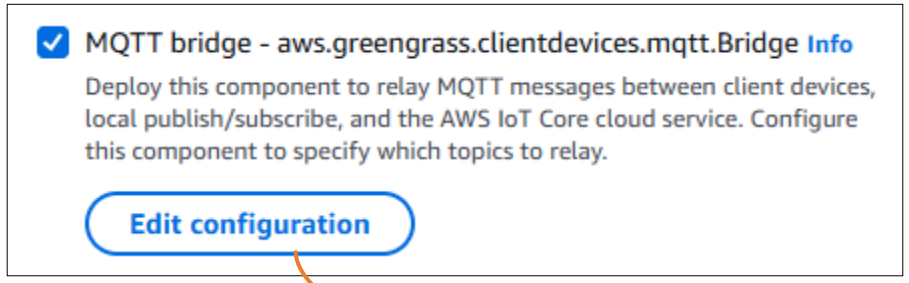


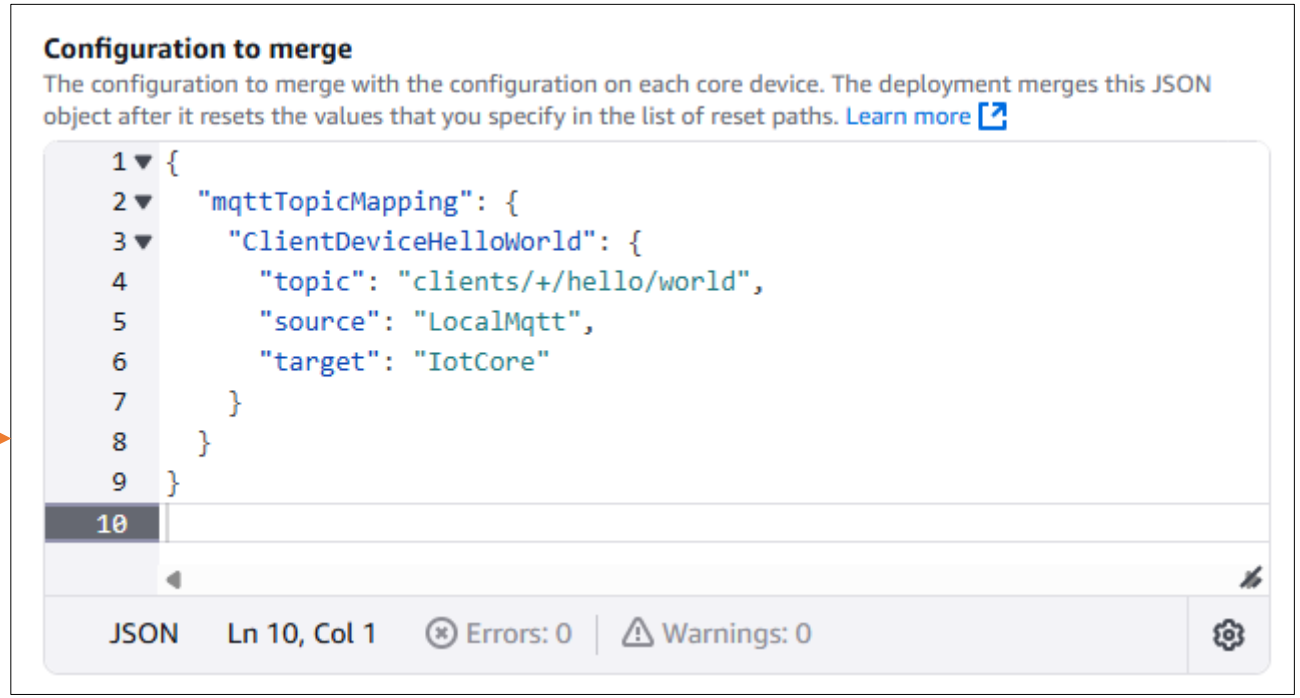


In this example, permissive policy is used to allow all operations. To use a more restrictive policy, check out the policy file in github: ***cmpe583-ClientDeviceAuth.Json***

Step 3: Deploy Required Greengrass Components

Customize aws.greengrass.clientdevices.mqtt.Bridge

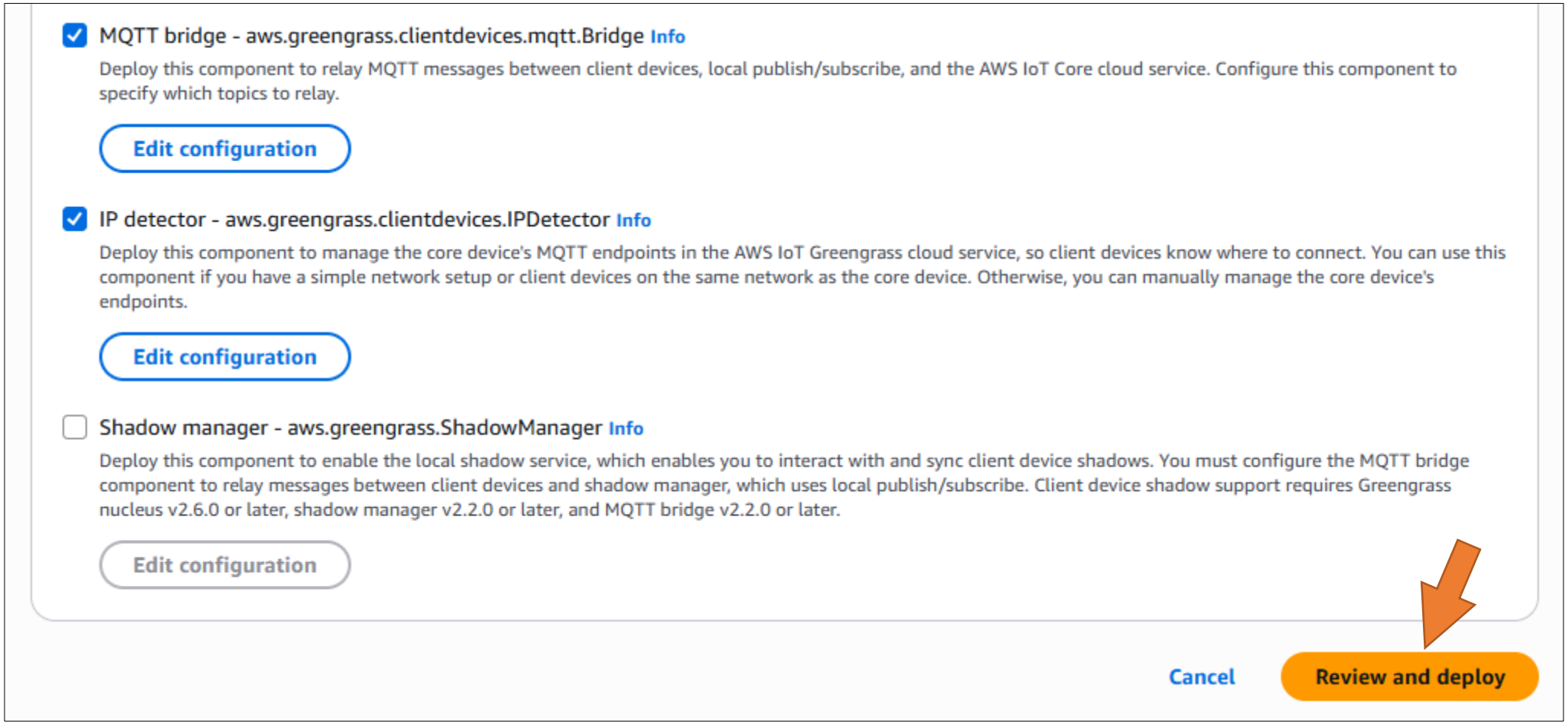




- Relay MQTT messages on the clients/+/hello/world topic filter **from client** devices **to the AWS IoT Core** cloud service.
- For example, this topic filter matches the clients/CMPE_Test_Client1/hello/world topic.

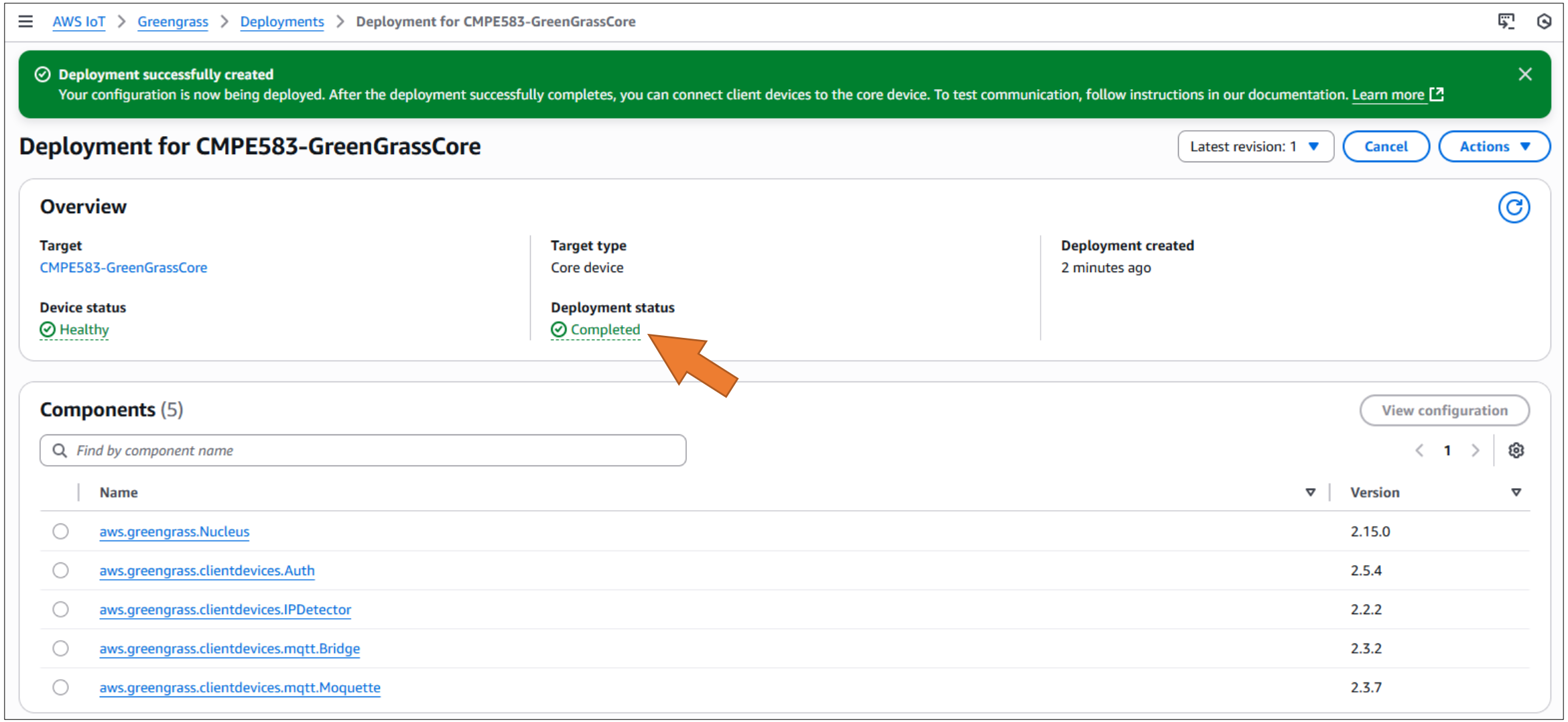
Step 3: Deploy Required Greengrass Components

Review & Deploy Greengrass Components



Step 3: Deploy Required Greengrass Components

Check Deployment Status



Onboarding Client Devices

Step 1: Install AWS IoT Device SDKs

Programming Languages

- Client devices can use the AWS IoT Device SDK to discover, connect, and communicate with a core device.
- AWS IoT Device SDKs
  - AWS IoT C++ Device SDK
  - AWS IoT Device SDK for Python
  - AWS IoT Device SDK for JavaScript
  - AWS IoT Device SDK for Java
- AWS Mobile SDKs
  - AWS Mobile SDK for Android
  - AWS Mobile SDK for iOS

Step 1: Install AWS IoT Device SDKs

AWS IoT Device SDK v2 for Python

- AWS IoT Device SDK is open source and publicly available in GitHub!

Clone the AWS IoT Device SDK v2 for Python repository to download it.

```
git clone https://github.com/aws/aws-iot-device-sdk-python-v2.git
```

Install the AWS IoT Device SDK v2 for Python.

```
python3 -m venv venvclient
source venvclient/bin/activate

(venvclient)$ python3 -m pip install ./aws-iot-device-sdk-python-v2
```

★ It is recommended to install & run aws-iot-device-sdk in virtual environment to avoid making any changes that affect the entire system!

Step 2: Run Sample Device Discovery

With samples/basic_discovery.py

- ✓ The discovery sample application sends the message 10 times and disconnects. It also subscribes to the same topic where it publishes messages.

```
(venvclient)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient)$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/hello/world' \
    --message 'Hello World!' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1 \
    --max_pub_ops 10
```

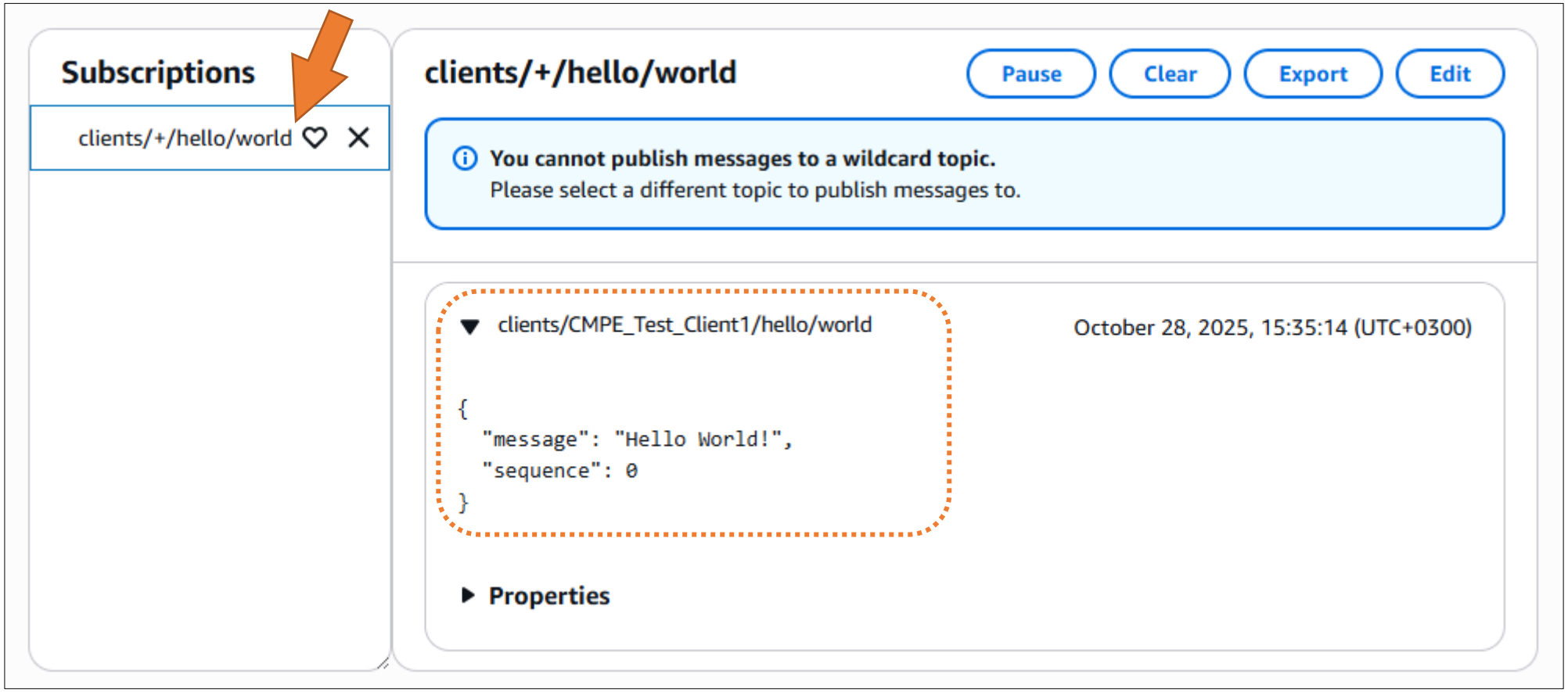
Step 2: Run Sample Device Discovery

Verify Client Device Connection From Terminal

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/hello/world' \
    --message 'Hello World!' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate1.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private1.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1
Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Publish received on topic clients/CMPE_Test_Client1/hello/world
b'{"message": "Hello World!", "sequence": 0}'
Successfully published to topic clients/CMPE_Test_Client1/hello/world with payload `{"message": "Hello World!", "sequence": 0}`
```

Step 3: Verify Client Device Connection (MQTT Bridge)

Verify From MQTT Test Client



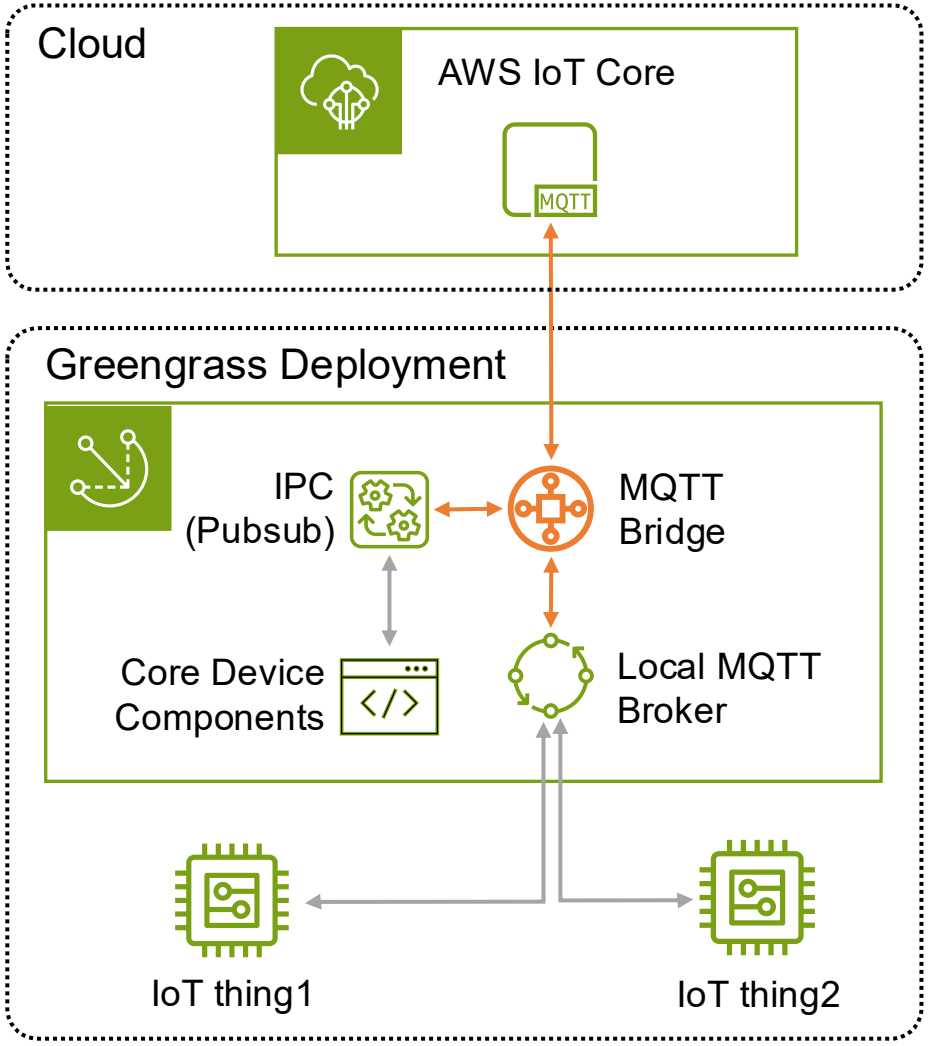
Communicate with Client Devices

MQTT Bridge: Understanding the Brokers

- The *MQTT bridge* component relays messages between three distinct systems. Before configuring the bridge, it's important to understand these systems:

1. Local MQTT ("**LocalMqtt**")
   - The **local** broker (e.g., Moquette) that client devices (like IoT sensors) connect to.
   - Handles messages between client devices and the core device.
2. Local Publish/Subscribe ("**Pubsub**")
   - The **internal** Greengrass message broker.
   - Handles messages between different components running inside the core device.
3. AWS IoT Core ("**IotCore**")
   - The **cloud**-based MQTT broker.
   - Handles messages between the Greengrass core device and the AWS Cloud.

MQTT Bridge: Topic Mapping for Different Cases



Communication Scenario	*mqttTopicMapping* Settings of MQTT Bridge
Thing to Thing	**Not required***
Component to Component	**Not required****
Thing to Core Device	"source": "LocalMqtt" "target": "Pubsub"
Core Device to Thing	"source": "Pubsub" "target": "LocalMqtt"
Thing to IoT Core	"source": "LocalMqtt" "target": "IoTCore"
IoT Core to Thing	"source": "IotCore" "target": "LocalMqtt"

* Things can exchange messages directly via the Local MQTT broker; the MQTT Bridge is not required.

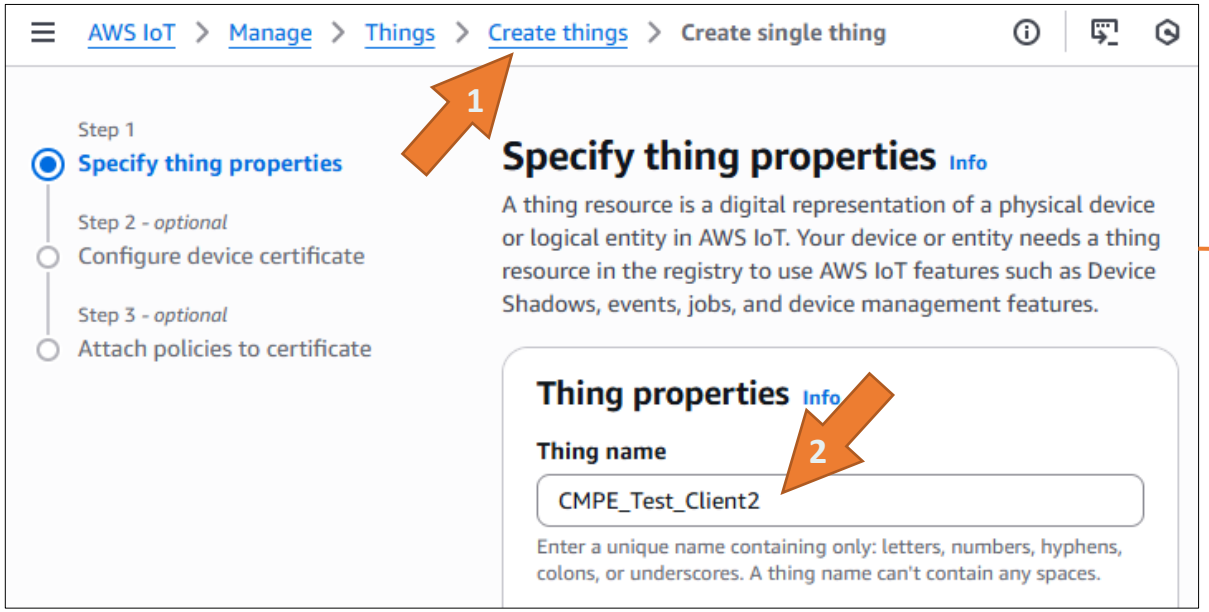
** Components can exchange messages directly via the Local IPC (Pubsub).

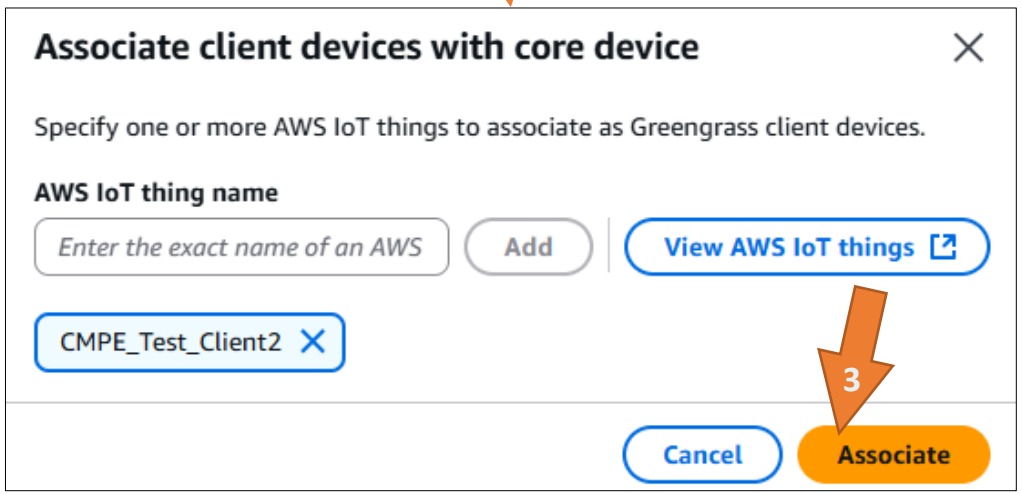
Steps for Client Device Communication

1. Create multiple IoT client devices.
2. Configure cloud discovery to set up an MQTT bridge for topic mapping.
3. Create custom component (with Python) to subscribe to MQTT topics.
4. Deploy and review your component.
5. Use the AWS IoT Device SDK to implement different use cases.

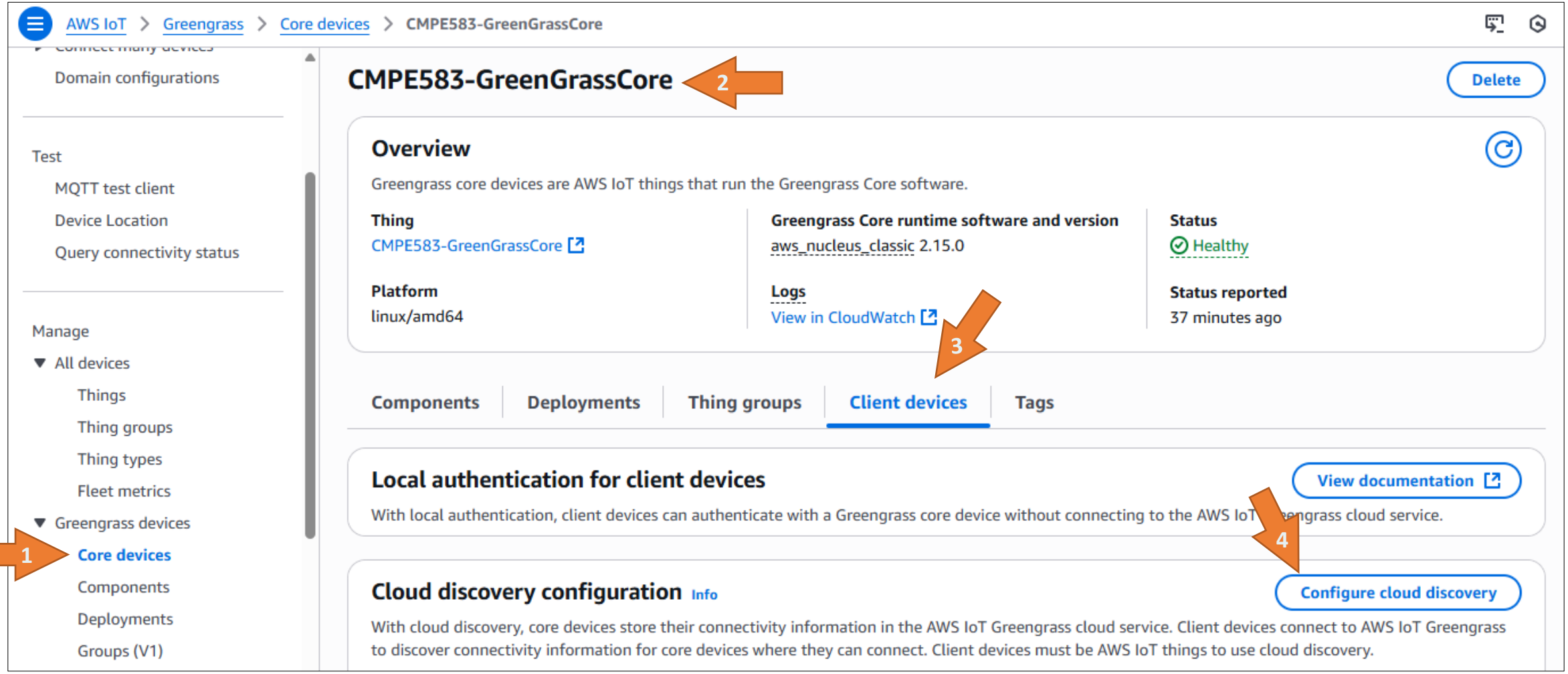
Step 1: Create More IoT Clients

Create Another Client and Associate It With Core Device



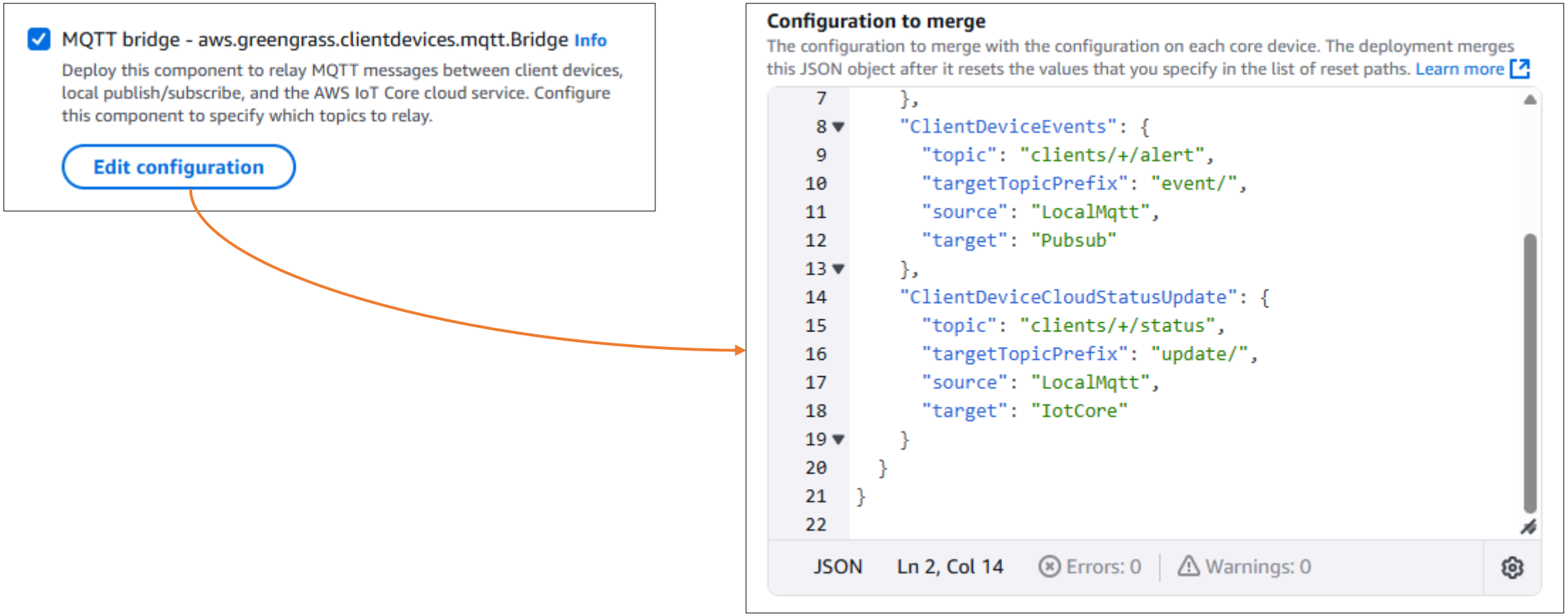


Step 2: Reconfigure Cloud Discovery



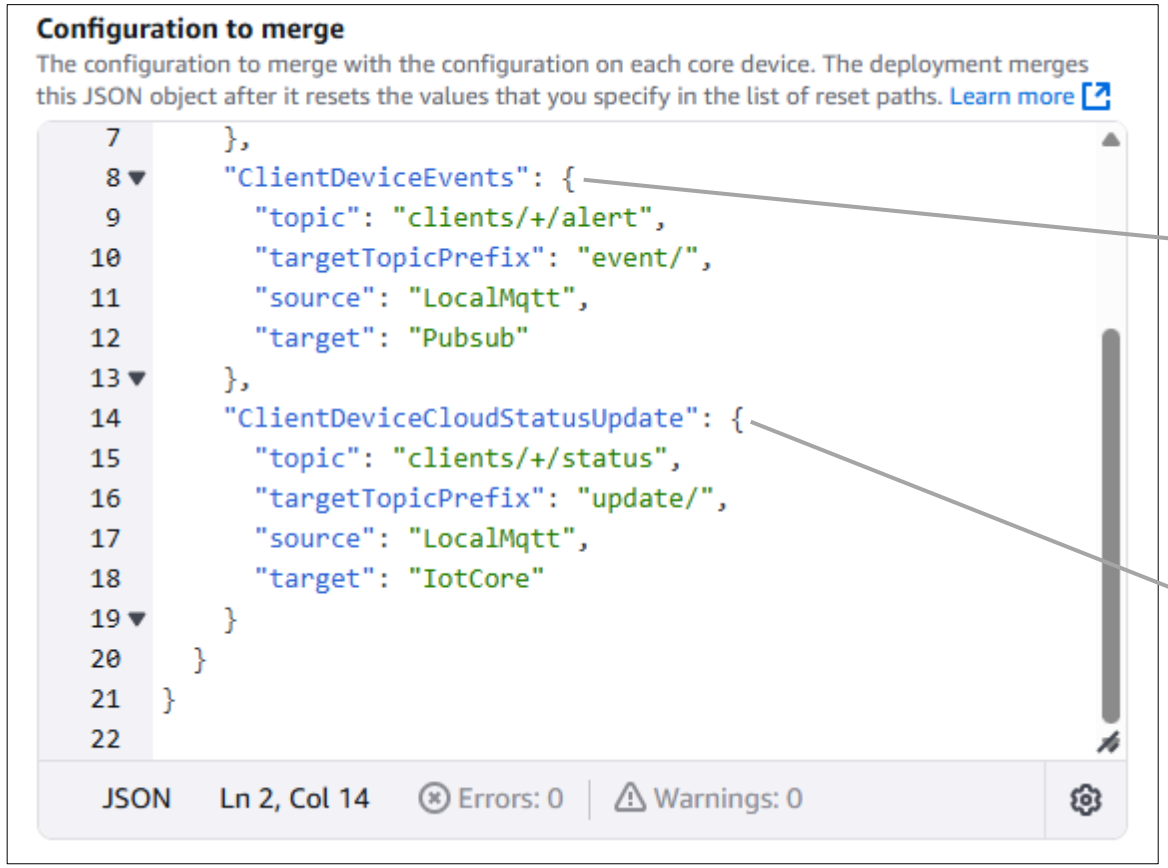
Step 2: Reconfigure Cloud Discovery

Reconfigure MQTT Bridge



Step 2: Reconfigure Cloud Discovery

New MQTT Topic Mapping

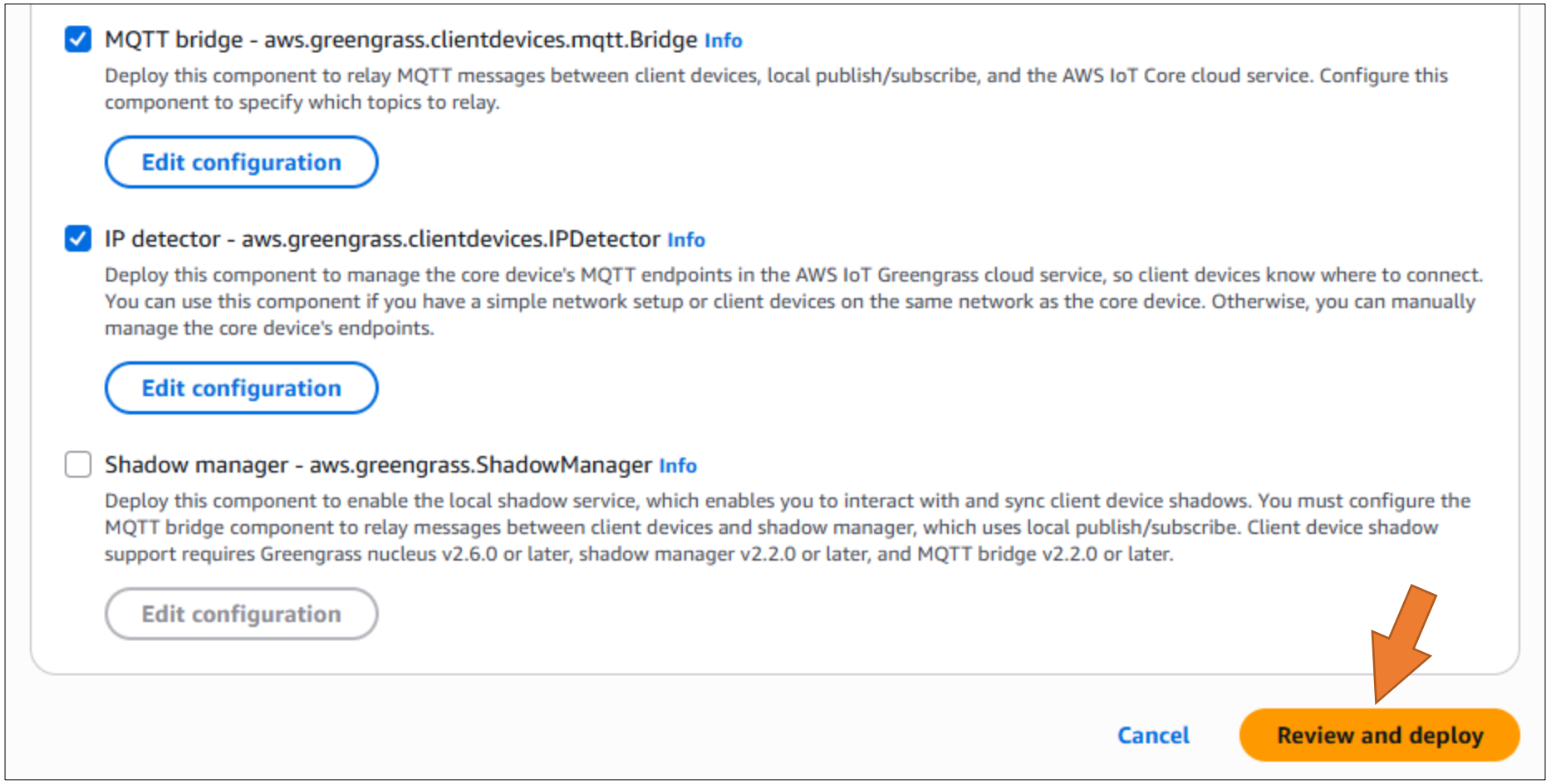


Relay messages from client devices to local IPC (Pubsub) through MQTT Bridge with **clients/+/alert** topic filter and add the *events/* prefix to the target topic. Core device components can receive this message with **events/clients/+/alert** topic.

Relay messages from client devices to AWS IoT Core through MQTT Bridge with clients/+/status topic filter and add *update/* prefix to the target topic. IoT Core can receive this message with **update/clients/+/status** topic.

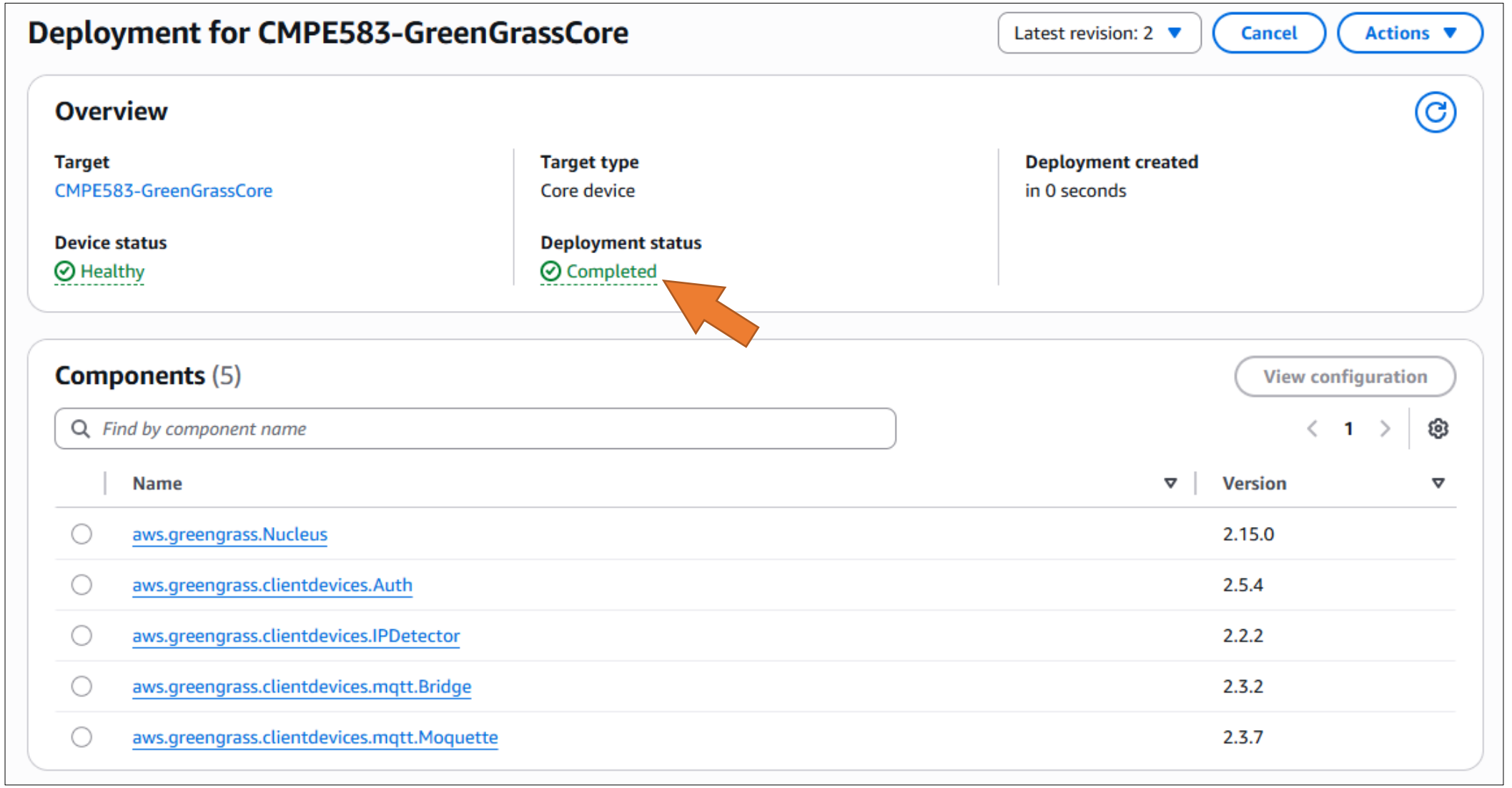
Step 2: Reconfigure Cloud Discovery

Review & Deploy the New Configuration



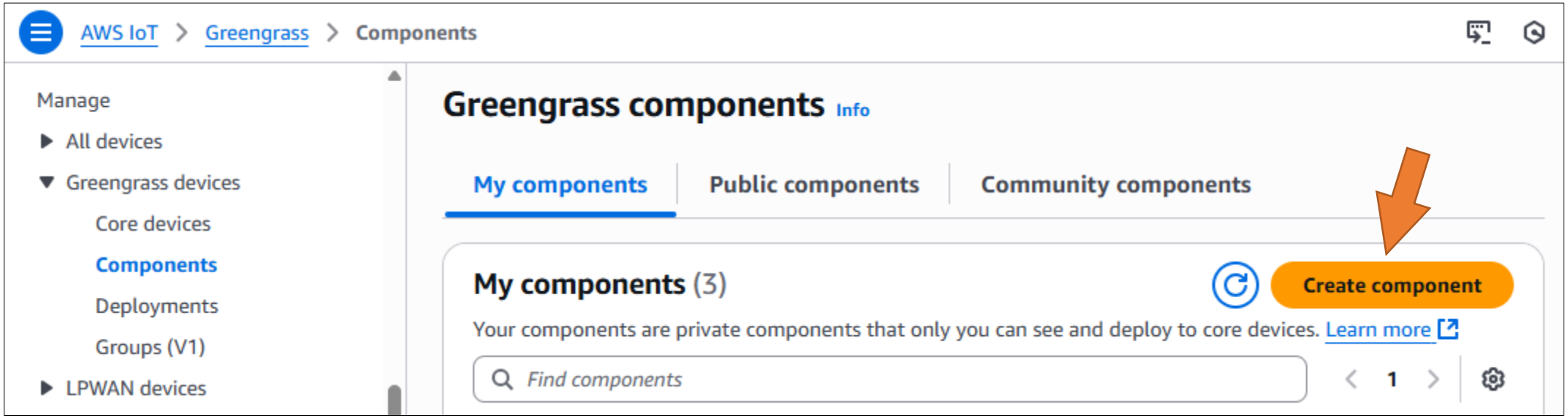
Step 2: Reconfigure Cloud Discovery

Check Deployment Status



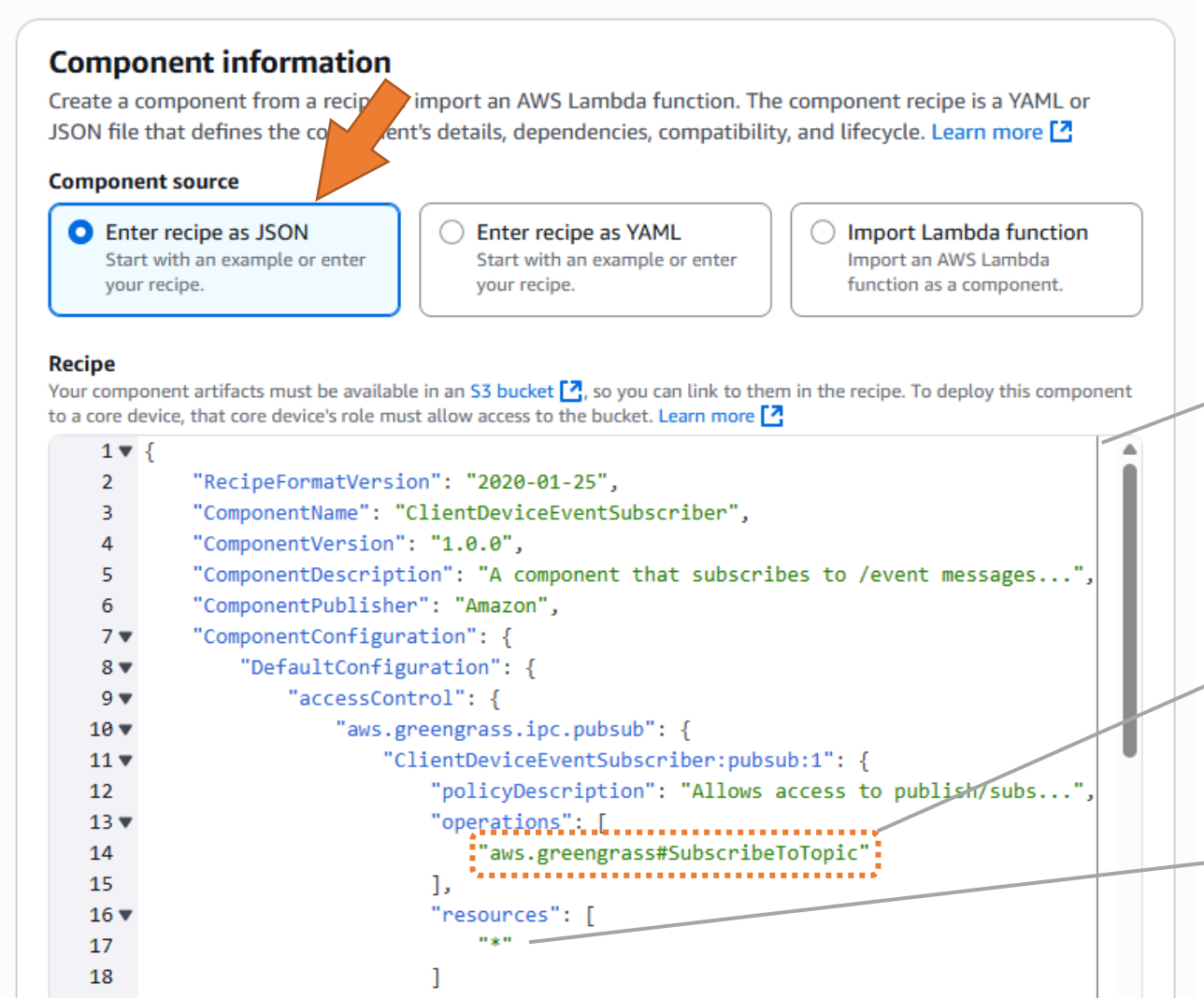
Step 3: Create a Custom Component

To Suscribe Local MQTT Events



Step 3: Create a Custom Component

Configure accessControl for Local Messages



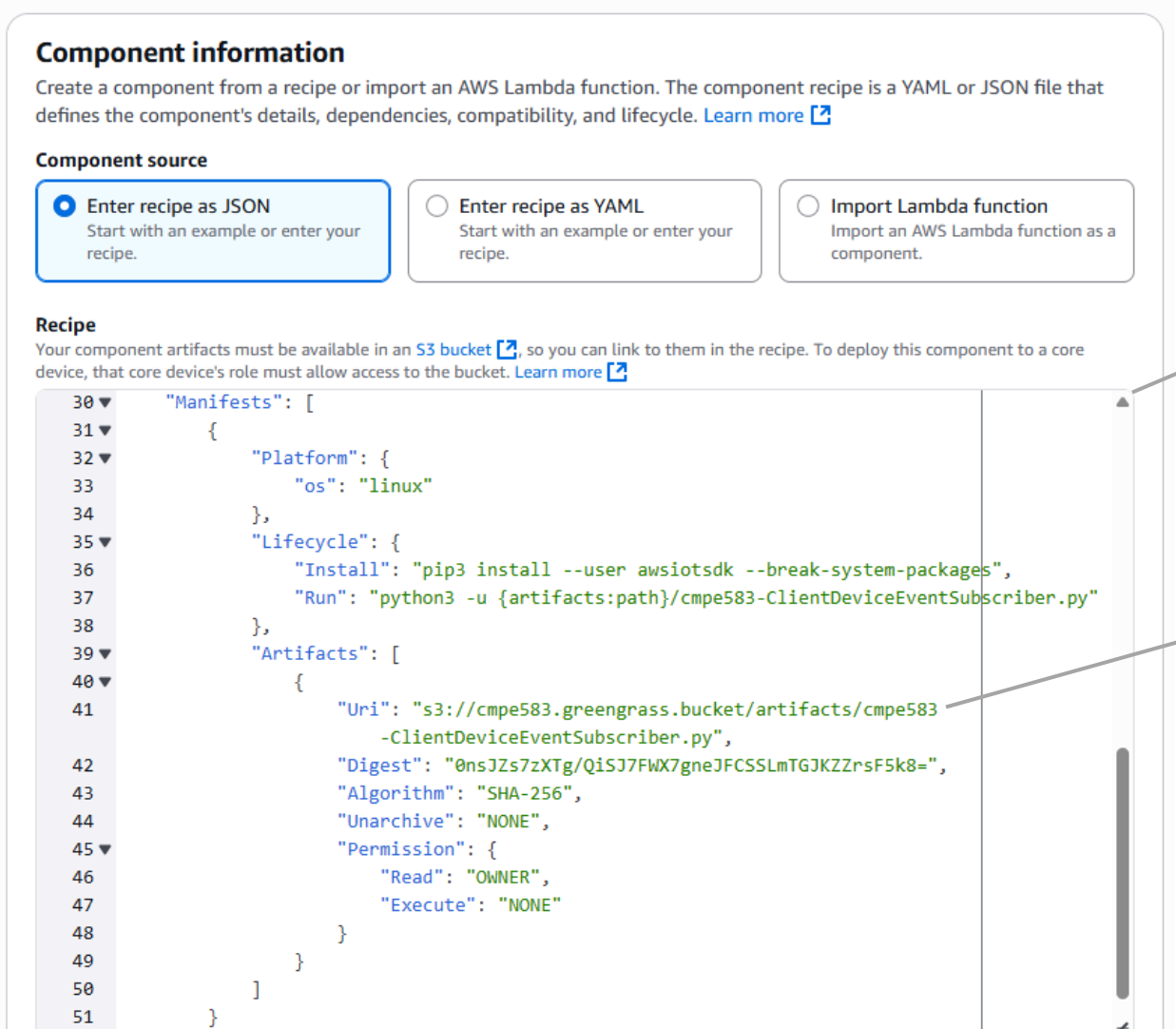
Check GitHub to see whole contents in ***cmpe583-ClientDeviceEventSubscriber.json***

Allow the component to subscribe to messages from local IoT Devices.

Allow access to all topics.

Step 3: Create a Custom Component

Configure Artifacts in the Recipe



Component information

Create a component from a recipe or import an AWS Lambda function. The component recipe is a YAML or JSON file that defines the component's details, dependencies, compatibility, and lifecycle. Learn more

Component source

- Enter recipe as JSON — Start with an example or enter your recipe.
- Enter recipe as YAML — Start with an example or enter your recipe.
- Import Lambda function — Import an AWS Lambda function as a component.

Recipe

Your component artifacts must be available in an S3 bucket, so you can link to them in the recipe. To deploy this component to a core device, that core device's role must allow access to the bucket. Learn more

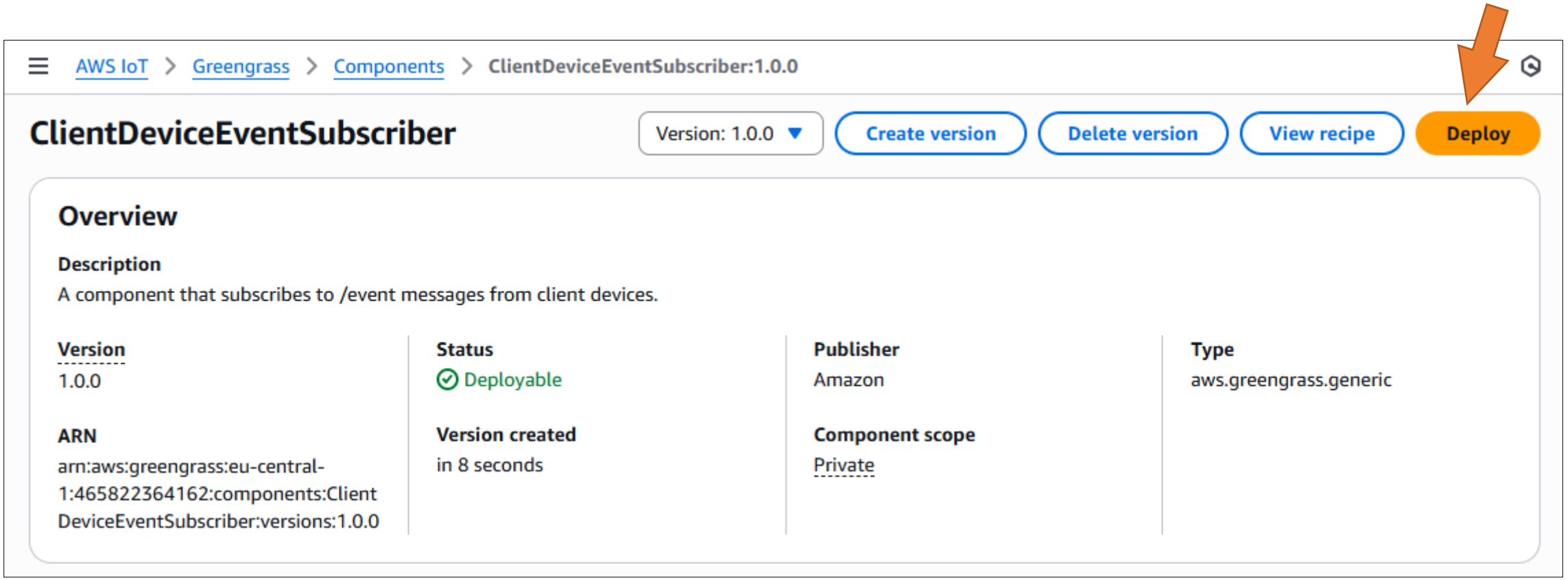
```
"Manifests": [
    {
        "Platform": {
            "os": "linux"
        },
        "Lifecycle": {
            "Install": "pip3 install --user awsiotsdk --break-system-packages",
            "Run": "python3 -u {artifacts:path}/cmpe583-ClientDeviceEventSubscriber.py"
        },
        "Artifacts": [
            {
                "Uri": "s3://cmpe583.greengrass.bucket/artifacts/cmpe583
                    -ClientDeviceEventSubscriber.py",
                "Digest": "0nsJZs7zXTg/QiSJ7FWX7gneJFCSSLmTGJKZZrsF5k8=",
                "Algorithm": "SHA-256",
                "Unarchive": "NONE",
                "Permission": {
                    "Read": "OWNER",
                    "Execute": "NONE"
                }
            }
        ]
    }
```

Check GitHub to see source code in python file: ***cmpe583-ClientDeviceEventSubscriber.py***

Upload python file to S3 bucket and be sure that the Core Device has proper access rights to related file.

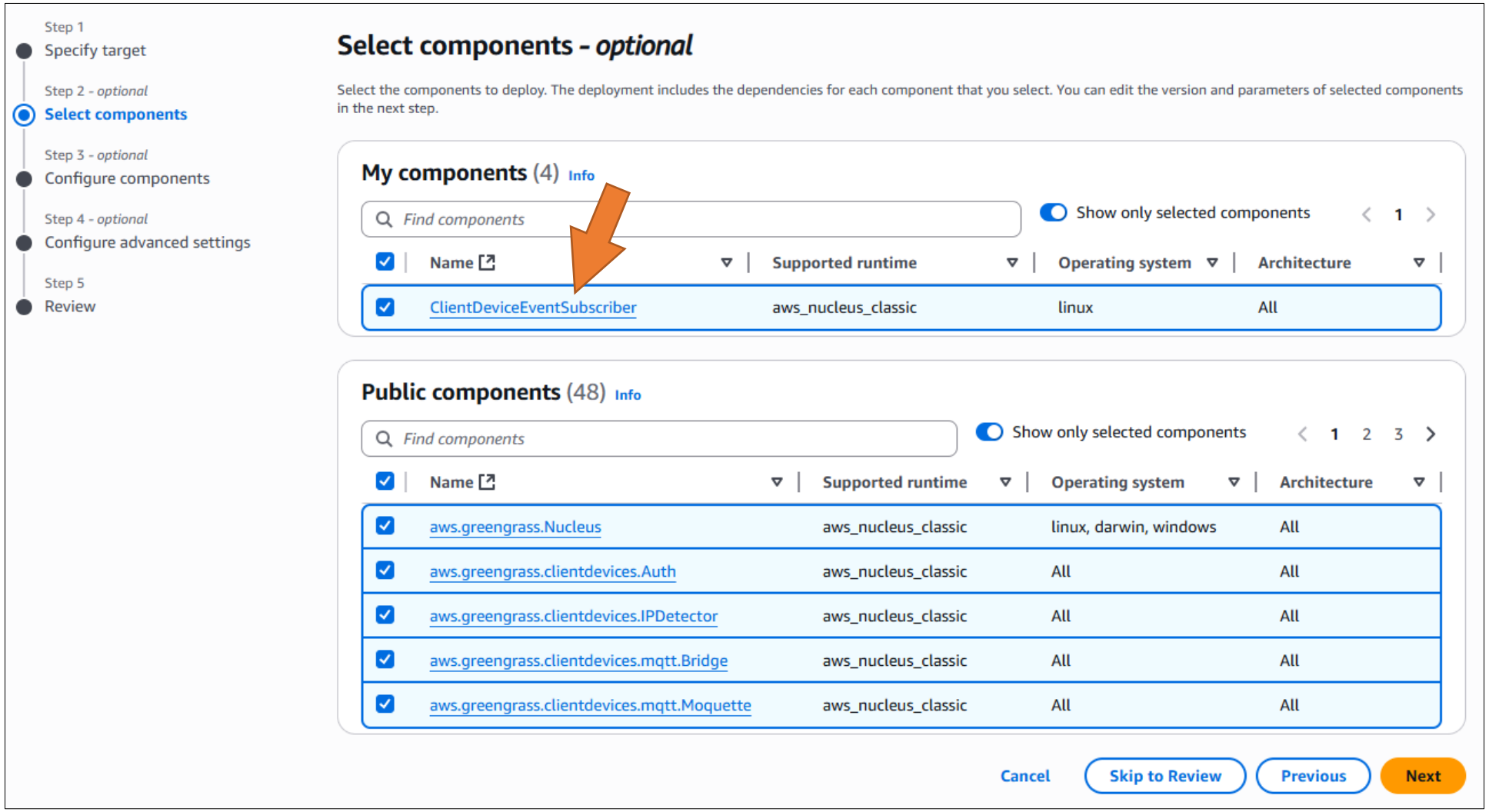
Step 4: Deploy Your Component

Via AWS IoT Greengrass (Console)



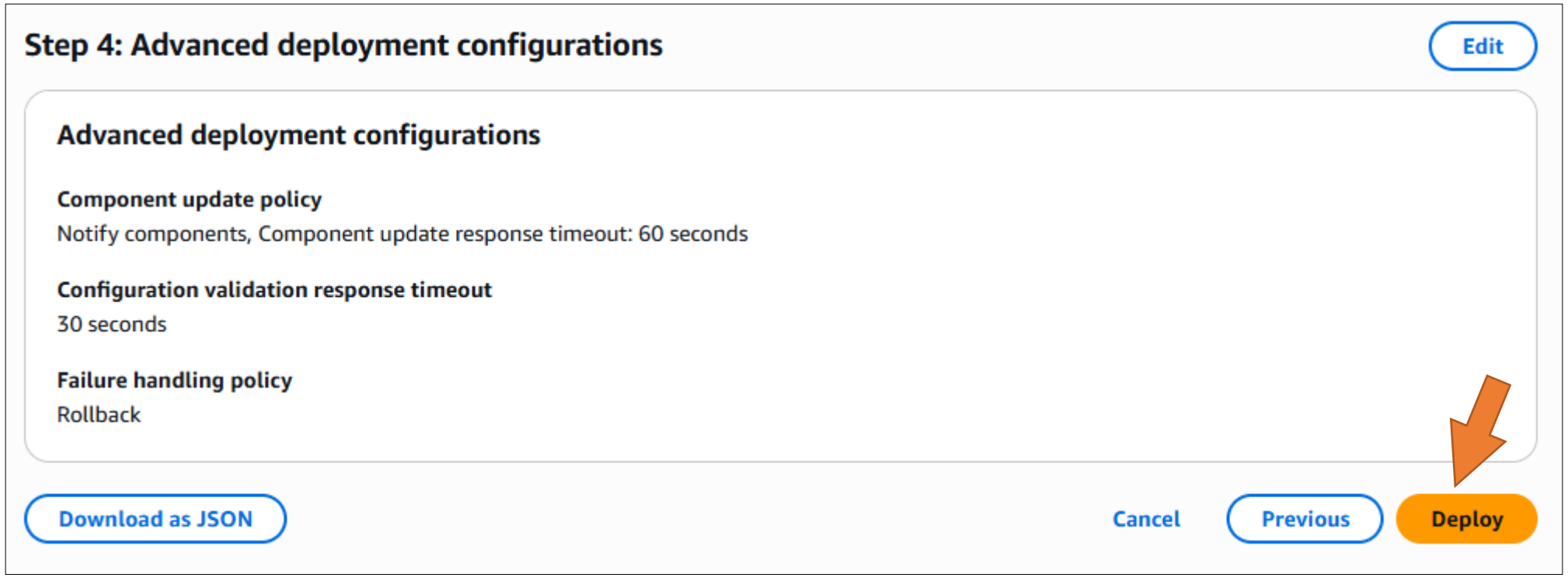
Step 4: Deploy Your Component

Select the Component



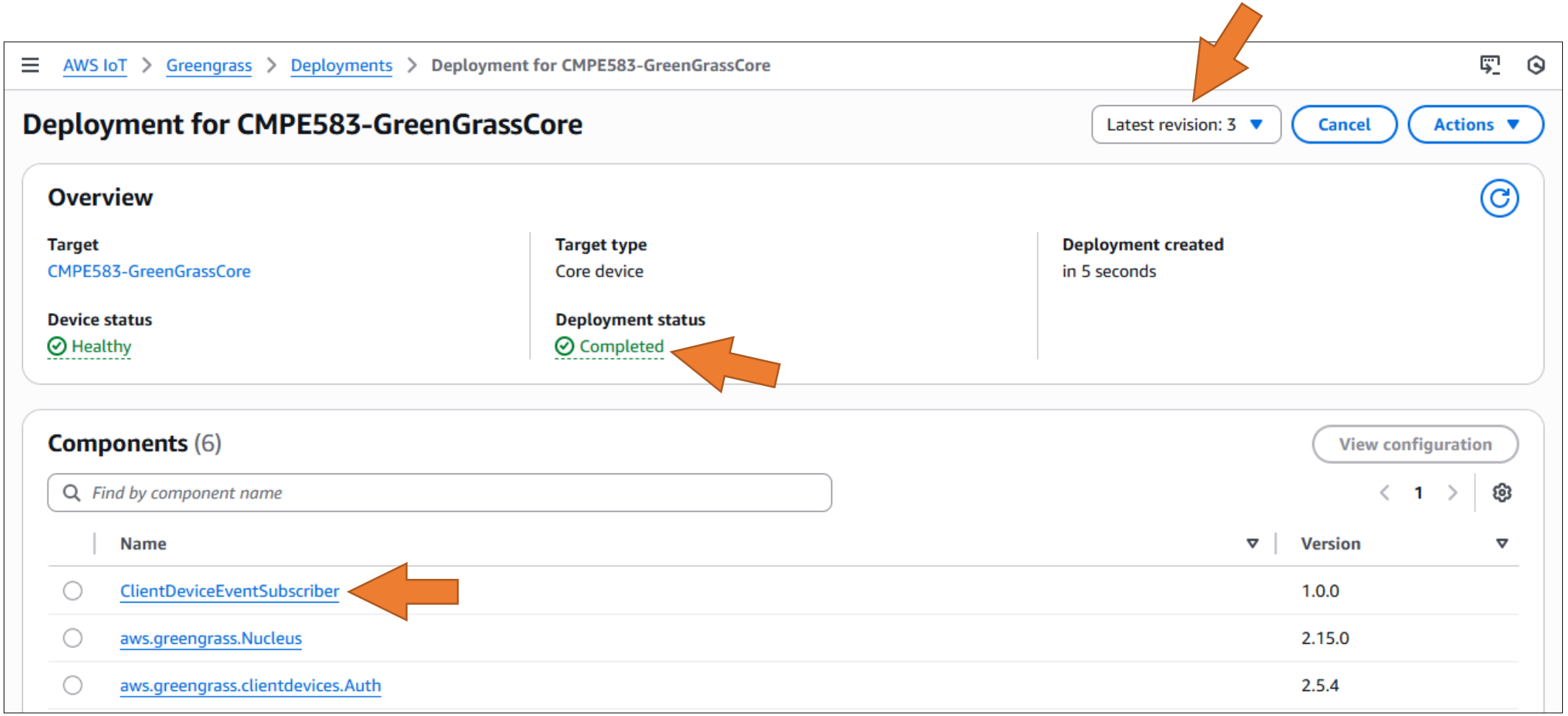
Step 4: Deploy Your Component

Review & Deploy



Step 4: Deploy Your Component

Check Deployment Status on AWS IoT Console



Step 4: Deploy Your Component

Check Deployment Status on Core Device

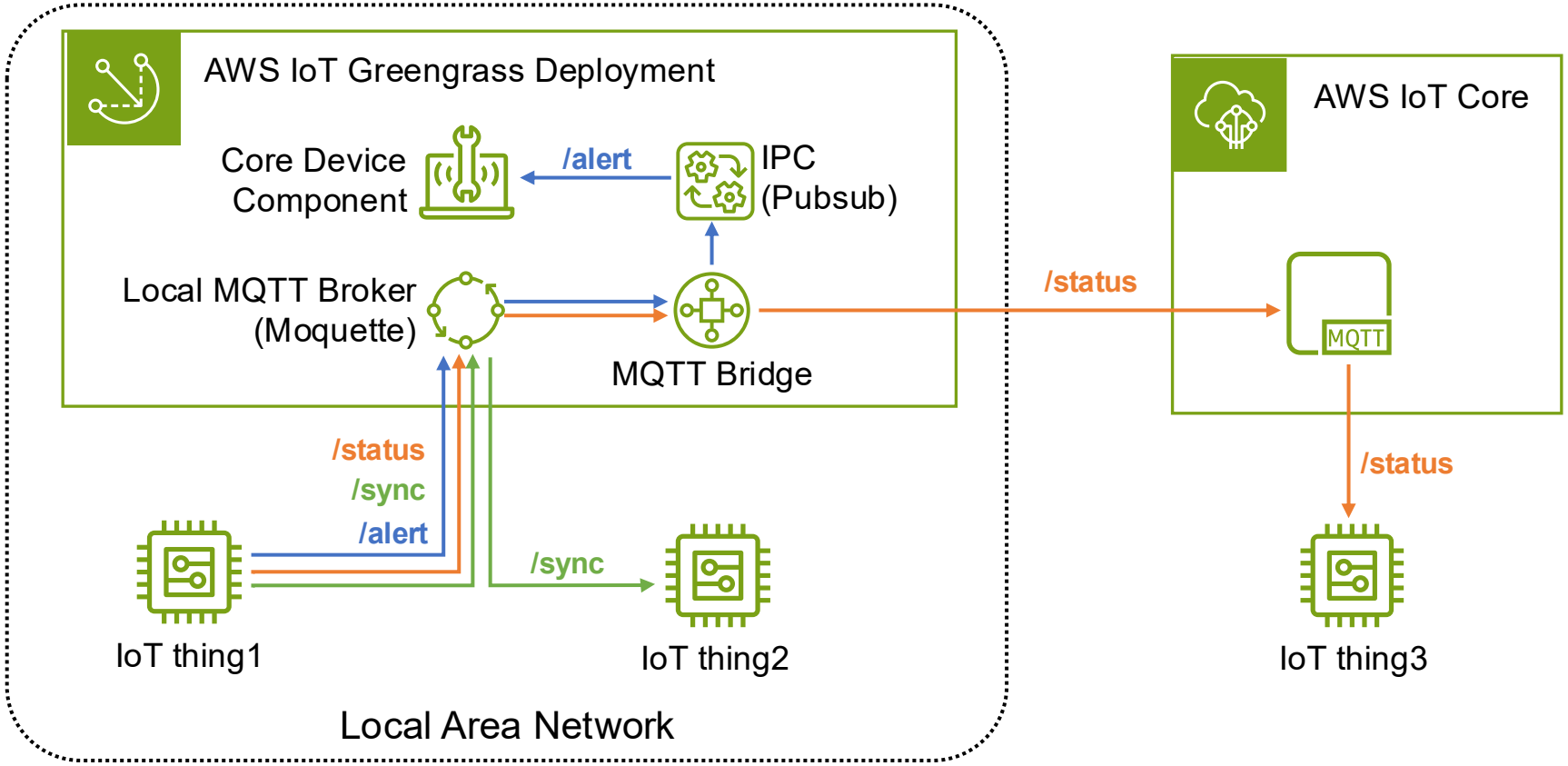
- Check ClientDeviceEventSubscriber.log file if the deployed component runs without error.
- If yes, you will see that the Core Device was subscribed to the topic successfully.

```
root@cagatay:/home/cagatay# tail -n 100 /greengrass/v2/logs/ClientDeviceEventSubscriber.log
2025-10-28T13:23:57.755Z [INFO] (pool-3-thread-79) ClientDeviceEventSubscriber: shell-runner-start. {scriptName=services.ClientDeviceEv
entSubscriber.lifecycle.Install, serviceName=ClientDeviceEventSubscriber, currentState=NEW, command=["pip3 install --user awsiotsdk --b
reak-system-packages"]}
2025-10-28T13:23:59.243Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Requirement already satisfied: awsiotsdk in /home/ggc_use
r/.local/lib/python3.12/site-packages (1.26.0). {scriptName=services.ClientDeviceEventSubscriber.lifecycle.Install, serviceName=ClientD
eviceEventSubscriber, currentState=NEW}
2025-10-28T13:23:59.243Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Requirement already satisfied: awscrt==0.28.1 in /home/gg
c_user/.local/lib/python3.12/site-packages (from awsiotsdk) (0.28.1). {scriptName=services.ClientDeviceEventSubscriber.lifecycle.Instal
l, serviceName=ClientDeviceEventSubscriber, currentState=NEW}
2025-10-28T13:23:59.743Z [INFO] (pool-3-thread-79) ClientDeviceEventSubscriber: shell-runner-start. {scriptName=services.ClientDeviceEv
entSubscriber.lifecycle.Run, serviceName=ClientDeviceEventSubscriber, currentState=STARTING, command=["python3 -u /greengrass/v2/packag
es/artifacts/ClientDeviceEventSubscriber/1.0.0..."]}
2025-10-28T13:24:00.110Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Successfully subscribed to topic: event/clients/+/alert.
{scriptName=services.ClientDeviceEventSubscriber.lifecycle.Run, serviceName=ClientDeviceEventSubscriber, currentState=RUNNING}
root@cagatay:/home/cagatay#
```

Four Different Device Communication Scenarios

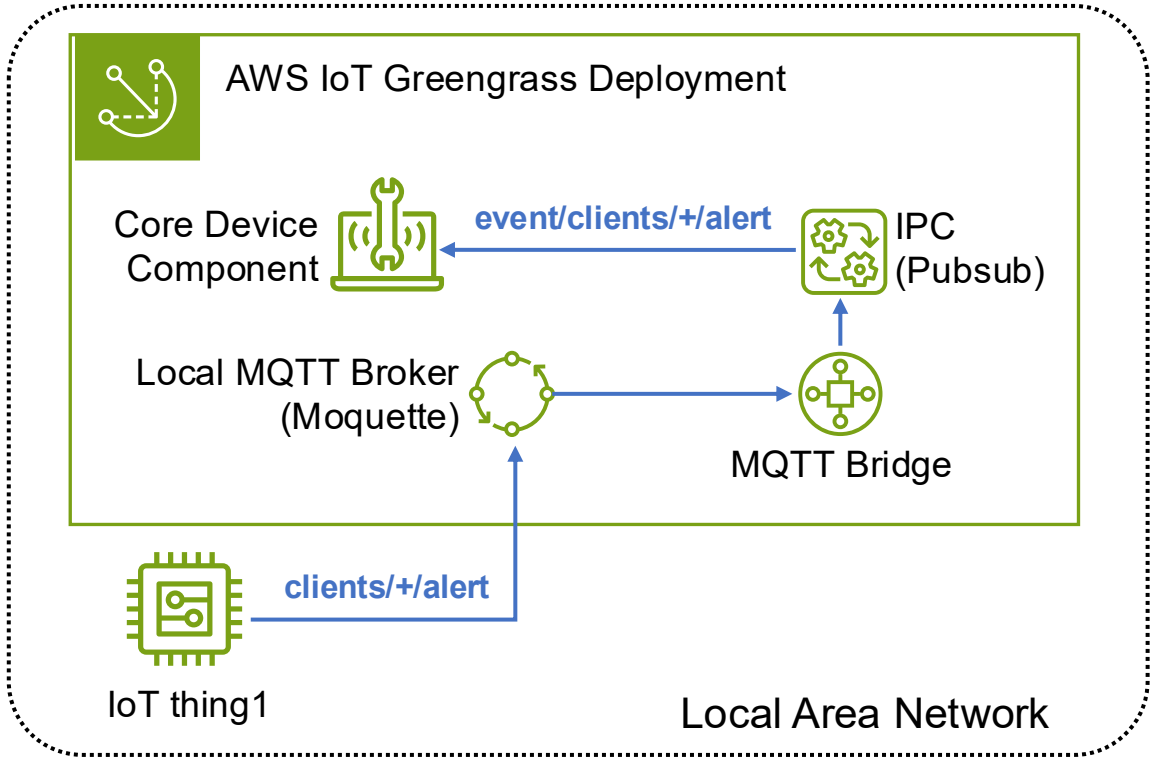
Communicating with Client Devices

- In this section, we will examine 4 different communication scenario where we will send events from the IoT device to the core device and the cloud using the **MQTT Bridge**, and to other local client devices directly via the **Local MQTT Broker**.



Case 1: IoT Client to Core Device Communication

- Publish 'clients/CMPE_Test_Client1/alert' Topic From IoT Client1
- Receive '**event**/clients/+/alert' on Core Device Component



Step 1: Publish 'clients/+/alert' Topic From IoT Client

Publish Topic to Trigger Subscribers (Core Device)

- Publish the topic with basic_discovery.py script.

```
(venvclient)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient)$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/alert' \
    --message 'Production Anomaly Detected. Check the Production Plant!' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish
```

This message will be relayed to Core Device

```
"ClientDeviceEvents": {
    "topic": "clients/+/alert",
    "targetTopicPrefix": "event/",
    "source": "LocalMqtt",
    "target": "Pubsub"
},
```

Step 1: Publish 'clients/+/alert' Topic From IoT Client

Verify Process on Terminal

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/alert' \
    --message 'Production Anomaly Detected. Check the Production Plant!' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate1.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private1.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish
Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Successfully published to topic clients/CMPE_Test_Client1/alert with payload `{"message": "Production Anomaly Detected.
Check the Production Plant!", "sequence": 0}`
```

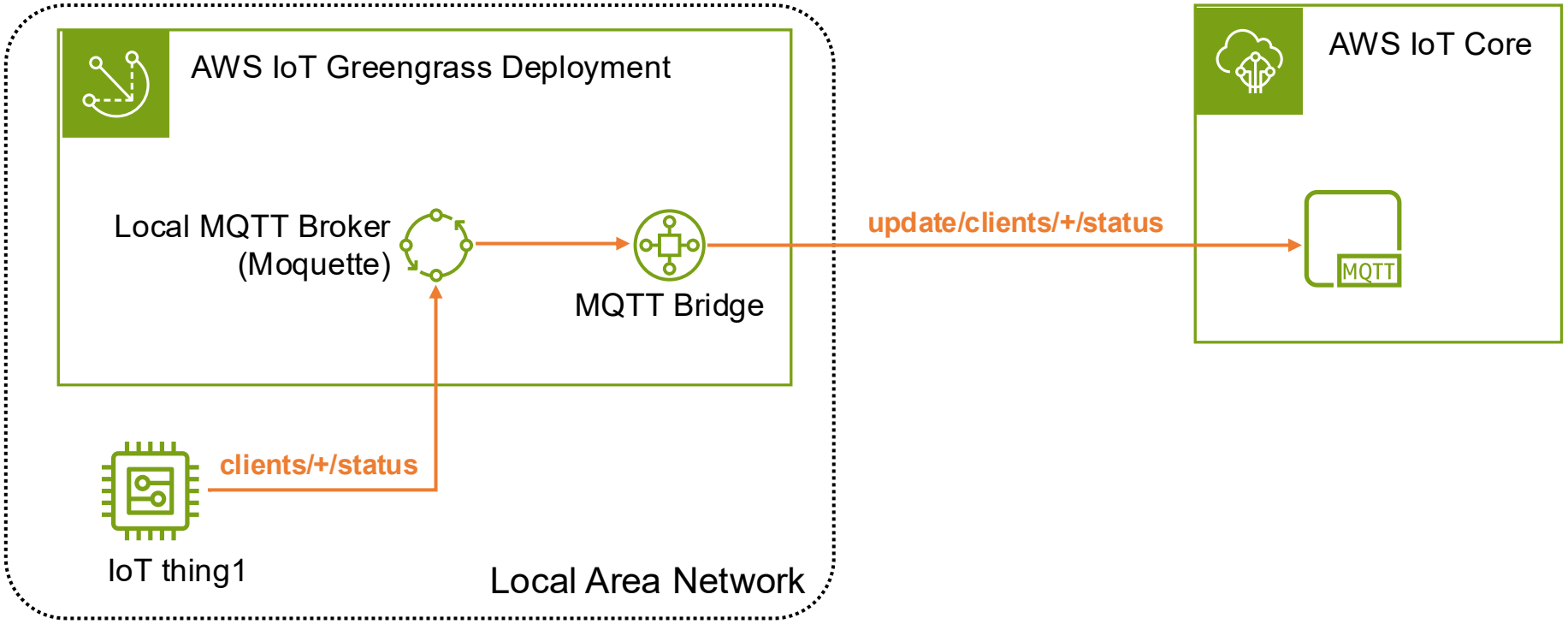
Step 2: Receive 'event/clients/+/alert' on Component

Verify Process on Core Device's Terminal

```
root@cagatay:/home/cagatay#
root@cagatay:/home/cagatay# tail -n 100 /greengrass/v2/logs/ClientDeviceEventSubscriber.log
2025-10-28T13:23:57.755Z [INFO] (pool-3-thread-79) ClientDeviceEventSubscriber: shell-runner-start. {scriptName=services
.ClientDeviceEventSubscriber.lifecycle.Install, serviceName=ClientDeviceEventSubscriber, currentState=NEW, command=["pip
3 install --user awsiotsdk --break-system-packages"]}
2025-10-28T13:23:59.243Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Requirement already satisfied: awsiotsdk i
n /home/ggc_user/.local/lib/python3.12/site-packages (1.26.0). {scriptName=services.ClientDeviceEventSubscriber.lifecycl
e.Install, serviceName=ClientDeviceEventSubscriber, currentState=NEW}
2025-10-28T13:23:59.243Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Requirement already satisfied: awscrt==0.2
8.1 in /home/ggc_user/.local/lib/python3.12/site-packages (from awsiotsdk) (0.28.1). {scriptName=services.ClientDeviceEv
entSubscriber.lifecycle.Install, serviceName=ClientDeviceEventSubscriber, currentState=NEW}
2025-10-28T13:23:59.743Z [INFO] (pool-3-thread-79) ClientDeviceEventSubscriber: shell-runner-start. {scriptName=services
.ClientDeviceEventSubscriber.lifecycle.Run, serviceName=ClientDeviceEventSubscriber, currentState=STARTING, command=["py
thon3 -u /greengrass/v2/packages/artifacts/ClientDeviceEventSubscriber/1.0.0..."]}
2025-10-28T13:24:00.110Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Successfully subscribed to topic: event/cl
ients/+/alert. {scriptName=services.ClientDeviceEventSubscriber.lifecycle.Run, serviceName=ClientDeviceEventSubscriber,
currentState=RUNNING}
2025-10-28T13:58:19.828Z [INFO] (Copier) ClientDeviceEventSubscriber: stdout. Received new event from IoT Client: {"mess
age": "Production Anomaly Detected. Check the Production Plant!", "sequence": 0}. {scriptName=services.ClientDeviceEvent
Subscriber.lifecycle.Run, serviceName=ClientDeviceEventSubscriber, currentState=RUNNING}
root@cagatay:/home/cagatay#
```

Case 2: IoT Client to IoT Core Communication

- Publish 'clients/CMPE_Test_Client1/status' Topic From IoT Client1
- Receive '**update**/clients/+/status' Topic on AWS IoT Core



Step 1: Publish 'clients/+/status' Topic From IoT Client

Publish Topic to Trigger Subscribers (AWS IoT Core)

- Publish the topic with basic_discovery.py script.

```
(venvclient)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient)$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/status' \
    --message 'Client device is running for 2 days without errror!' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish
```

This message will be relayed to AWS IoT Core

```
"ClientDeviceCloudStatusUpdate": {
    "topic": "clients/+/status",
    "targetTopicPrefix": "update/",
    "source": "LocalMqtt",
    "target": "IotCore"
}
```

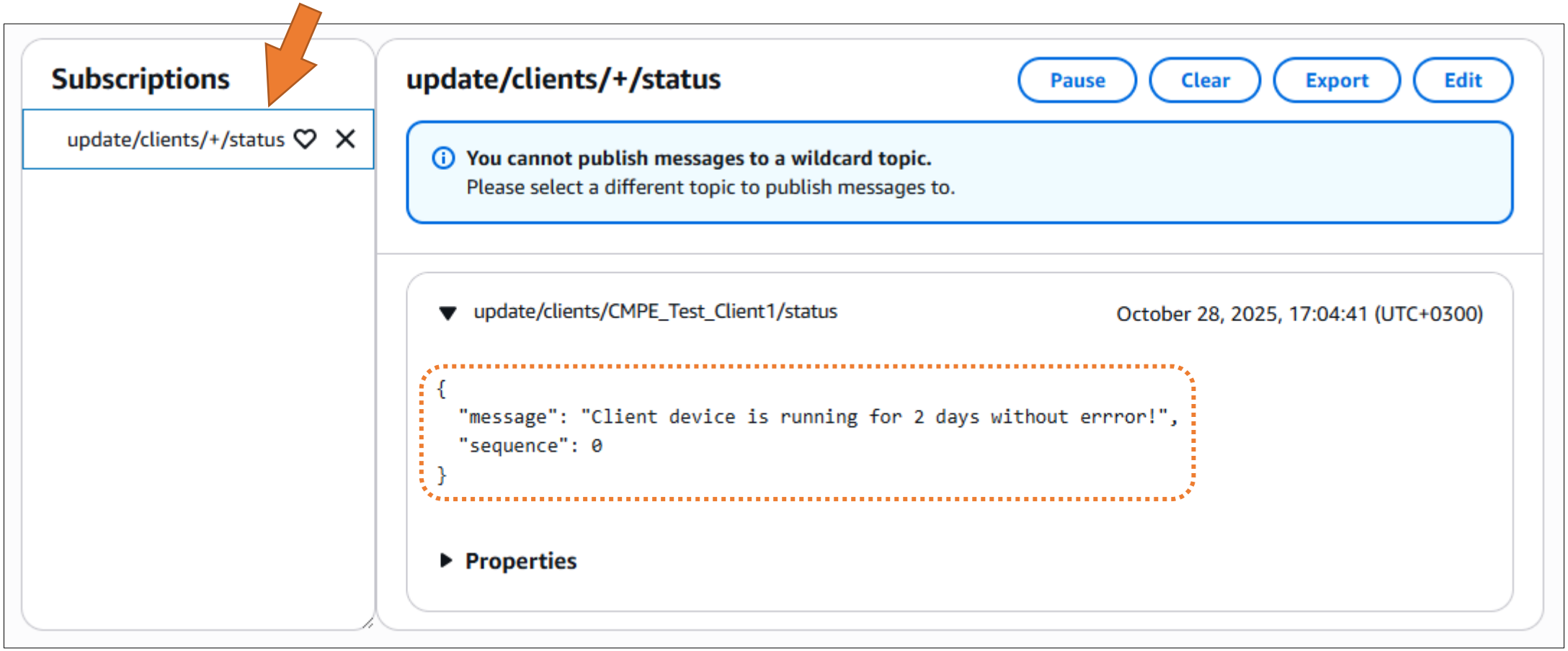
Step 1: Publish 'clients/+/status' Topic From IoT Client

Verify Process on Terminal

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/status' \
    --message 'Client device is running for 2 days without errror!' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate1.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private1.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish
Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Successfully published to topic clients/CMPE_Test_Client1/status with payload `{"message": "Client device is running for 2 days without errror!"
, "sequence": 0}`
```

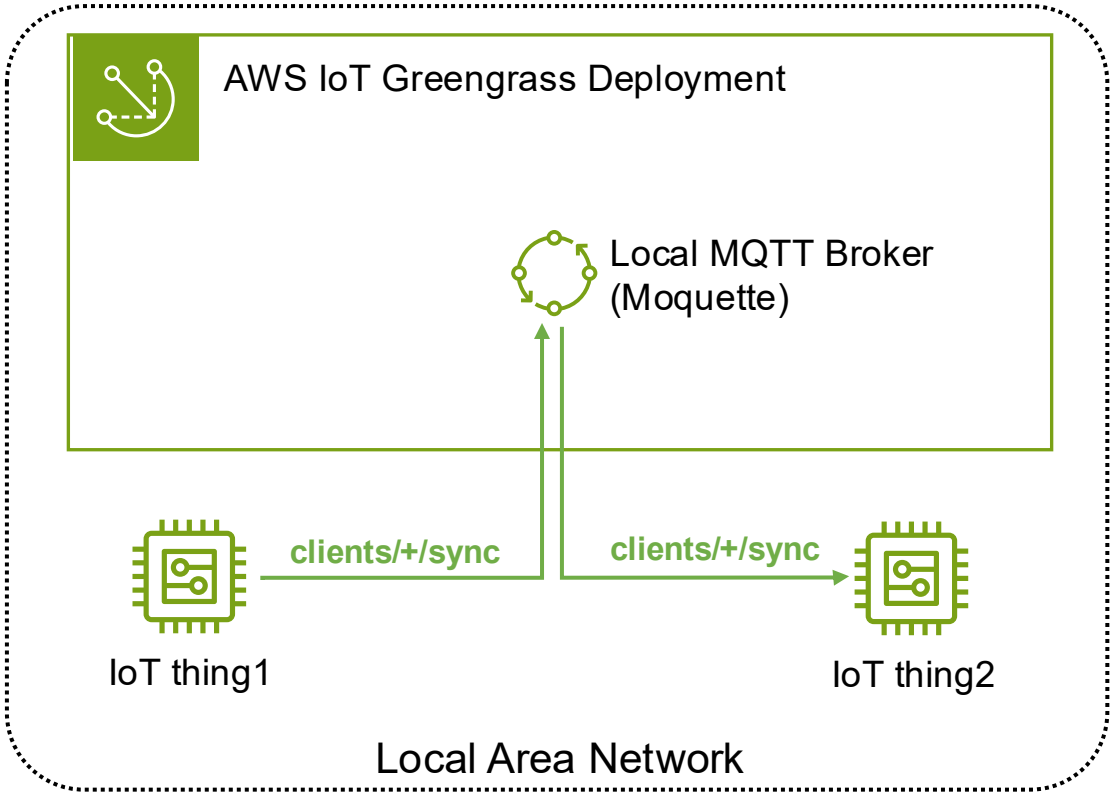
Step 2: Receive 'update/clients/+/status' on IoT Core

Verify Process on MQTT Test Client



Case 3: IoT Client to IoT Client Communication

- Publish 'clients/+/sync' Topic From IoT Client1 to IoT Client2



Step 1: Subscribe 'clients/+/sync' Topic on IoT Client2

Subscribe Topic From Test Client 2

- Subscribe the topic using the '*subscribe*' --mode argument of basic_discovery.py script.

```
python3 -m venv venvclient2
source venvclient2/bin/activate
(venvclient2)$ python3 -m pip install ./aws-iot-device-sdk-python-v2
(venvclient2)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient2)$ python3 basic_discovery.py \
   --thing_name 'CMPE_Test_Client2' \
   --topic 'clients/+/sync' \
   --ca_file ~/AmazonRootCA1.pem \
   --cert ~/certificate.pem.crt \
   --key ~/private.pem.key \
   --region eu-central-1 \
   --verbosity Warn \
   --mode subscribe
```

Step 2: Publish 'clients/+/sync' Topic From IoT Client1

Publish Topic to Trigger Subscribers

- Publish the topic using the '*publish*' --mode argument of basic_discovery.py script.

```
(venvclient)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient)$ python3 basic_discovery.py \
    --thing_name 'CMPE_Test_Client1' \
    --topic 'clients/CMPE_Test_Client1/sync' \
    --message 'Hello from IoT Client 1!' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1 \
    --verbosity Warn \
   --max_pub_ops 1 \
    --mode publish
```

This message will be relayed to test client 2 through local Greengrass core MQTT broker

★ Things can exchange messages directly via the Local MQTT broker; the MQTT Bridge is not required.

Step 2: Publish 'clients/+/sync' Topic From IoT Client1

Verify Topic Published

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name 'CMPE_Test_Client1' \
    --topic 'clients/CMPE_Test_Client1/sync' \
    --message 'Hello from IoT Client 1!' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate1.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private1.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish

Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Successfully published to topic clients/CMPE_Test_Client1/sync with payload `{"message": "Hello from IoT Client 1!",
"sequence": 0}`

(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$
```

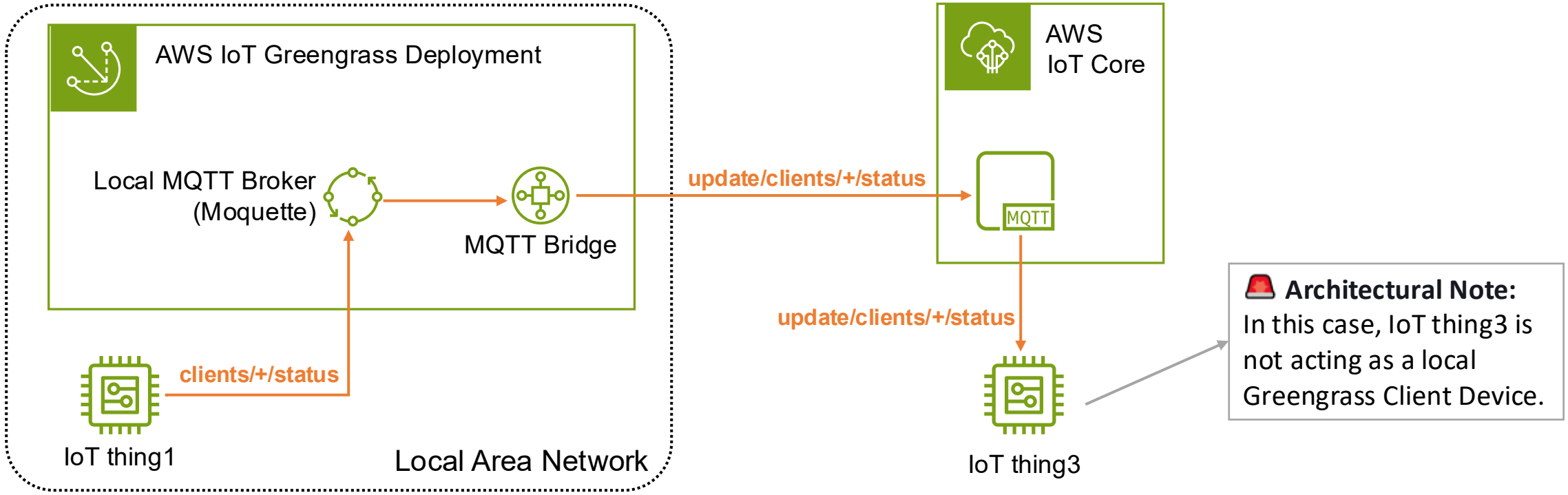
Step 2: Receive 'clients/+/sync' Topic on IoT Client2

Verify Topic Received

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name 'CMPE_Test_Client2' \
    --topic 'clients/+/sync' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate2.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private2.pem.key \
    --region eu-central-1 \
    --mode subscribe
Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Publish received on topic clients/CMPE_Test_Client1/sync
b'{"message": "Hello from IoT Client 1!", "sequence": 0}'
```

Case 4: IoT Core to IoT Client Communication

- Publish 'clients/+/status' Topic from IoT Client1 to IoT Core
- Receive '**update**/clients/+/status' topic on IoT Client3



Step 1: Publish 'clients/+/status' Topic From IoT Client1

Publish Topic to Trigger Subscribers (AWS Iot Core -> IoT Client 3)

- Publish the topic using the '*publish*' --mode argument of basic_discovery.py script.

```
(venvclient)$ cd aws-iot-device-sdk-python-v2/samples/greengrass

(venvclient)$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/status' \
    --message 'Status update from Client 1!' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1 \
    --verbosity Warn \
   --max_pub_ops 1 \
    --mode publish
```

This message will be relayed to AWS IoT Core

```
"ClientDeviceCloudStatusUpdate": {
    "topic": "clients/+/status",
    "targetTopicPrefix": "update/",
    "source": "LocalMqtt",
    "target": "IotCore"
}
```

Step 1: Publish 'clients/+/status' Topic From IoT Client1

Verify Topic Published

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples/greengrass$ python3 basic_discovery.py \
    --thing_name CMPE_Test_Client1 \
    --topic 'clients/CMPE_Test_Client1/status' \
    --message 'Client device is running for 2 days without errror!' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate1.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private1.pem.key \
    --region eu-central-1 \
    --max_pub_ops 1 \
    --mode publish

Performing greengrass discovery...
Received a greengrass discovery result! Not showing result for possible data sensitivity.
Trying core arn:aws:iot:eu-central-1:465822364162:thing/CMPE583-GreenGrassCore at host 10.0.2.15 port 8883
Connected!
Successfully published to topic clients/CMPE_Test_Client1/status with payload `{"message": "Client device is running for 2 days
 without errror!", "sequence": 0}`
```

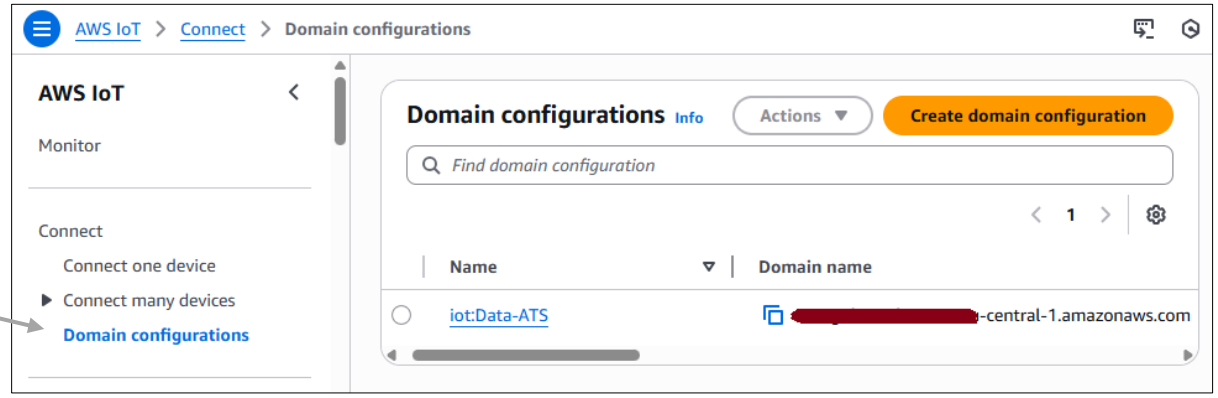
Receiving Events From AWS IoT Core

- Subscribe events published from AWS IoT Cloud with pubsub.py script using an empty --message.
- Don’t forget to modify MQTT Bridge for topic mapping!

✓ Use an empty--message with pubsub.py script to subsribe and receive events on IoT client device.

```
(venvclient)$ python3 pubsub.py \
    --topic 'topic-name' \
    --message ' ' \
    --endpoint 'your-endpoint' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key \
    --region eu-central-1
```

✓ Your account’s endpoint can be found under the domain configuration settings



Step 2: Receive 'update/clients/+/status' on IoT Client3

Subscribe Topic From Test Client 3

- Subscribe **IoT Core events** using empty --message argument with pubsub.py script.

```
(venvclient3)$ cd aws-iot-device-sdk-python-v2/samples

(venvclient3)$ python3 pubsub.py \
    --topic 'update/clients/+/status' \
    --message ' ' \
    --endpoint 'your-endpoint' \
    --ca_file ~/AmazonRootCA1.pem \
    --cert ~/certificate.pem.crt \
    --key ~/private.pem.key
```

Step 2: Receive 'update/clients/+/status' on IoT Client3

Verify Topic Received

```
(venvclient) cagatay@cagatay:~/Projects/aws-iot-device-sdk-python-v2/samples$ python3 pubsub.py \
    --topic 'update/clients/+/status' \
    --message ' ' \
    --endpoint '[redacted].iot.eu-central-1.amazonaws.com' \
    --ca_file /home/cagatay/shared_folder/greengrass/AmazonRootCA1.pem \
    --cert /home/cagatay/shared_folder/greengrass/certificate2.pem.crt \
    --key /home/cagatay/shared_folder/greengrass/private2.pem.key

Starting MQTT5 X509 PubSub Sample

==== Creating MQTT5 Client ====

==== Starting client ====
Lifecycle Connection Attempt
Connecting to endpoint: '[redacted]iot.eu-central-1.amazonaws.com' with client ID'mqtt5-sample-fa241d03'
Lifecycle Connection Success with reason code:<ConnectReasonCode.SUCCESS: 0>

==== Subscribing to topic 'update/clients/+/status' ====
Suback received with reason code:[<SubackReasonCode.GRANTED_QOS_1: 1>]

==== Sending 5 message(s) ====

==== Received message from topic 'update/clients/CMPE_Test_Client1/status': {"message": "Client device is running for 2 days without errror!",
"sequence": 0} ====
```

Step 2: Receive 'update/clients/+/status' on IoT Client3

Troubleshooting

- The pubsub.py sample code by default both publishes and subscribes.
- Remove publishing part from the source code to focus only on receiving messages.

```
146     # Publish
147     if message_count == 0:
148         print("==== Sending messages until program killed ====\n")
149     else:
150         print("==== Sending {} message(s) ====\n".format(message_count))
151
152     publish_count = 1
153     while (publish_count <= message_count) or (message_count == 0):
154         message = f"{message_string} [{publish_count}]"
155         print(f"Publishing message to topic '{message_topic}': {message}")
156         publish_future = client.publish(mqtt5.PublishPacket(
157             topic=message_topic,
158             payload=message,
159             qos=mqtt5.QoS.AT_LEAST_ONCE
160         ))
161         publish_completion_data = publish_future.result(TIMEOUT)
162         print("PubAck received with {}\n".format(repr(publish_completion_data.puback.reason_code)))
163         time.sleep(1.5)
164         publish_count += 1
165
166     received_all_event.wait(TIMEOUT)
167     print("{} message(s) received.\n".format(received_count))
168
```

Thank You